



Leibniz Institute
for high
performance
microelectronics

50 GHz Medium Power Amplifier: Exploring Qucs-s & OpenEMS

Phillip Ferreira Baade–Pedersen (Design Engineer)
Erlangen - 24/02/2026

Projects: BMBF -> FMD-QNC (16ME0831)

Key Takeaways from Yesterday



- 0 Insights on models within the IHP Open PDK
- 0 Building the testbench for two stage OTA
- 0 Building the testbench for the BGR
- 0 Exploring mismatch simulations
- 0 Beginning the layout of two stage OTA

Questions?

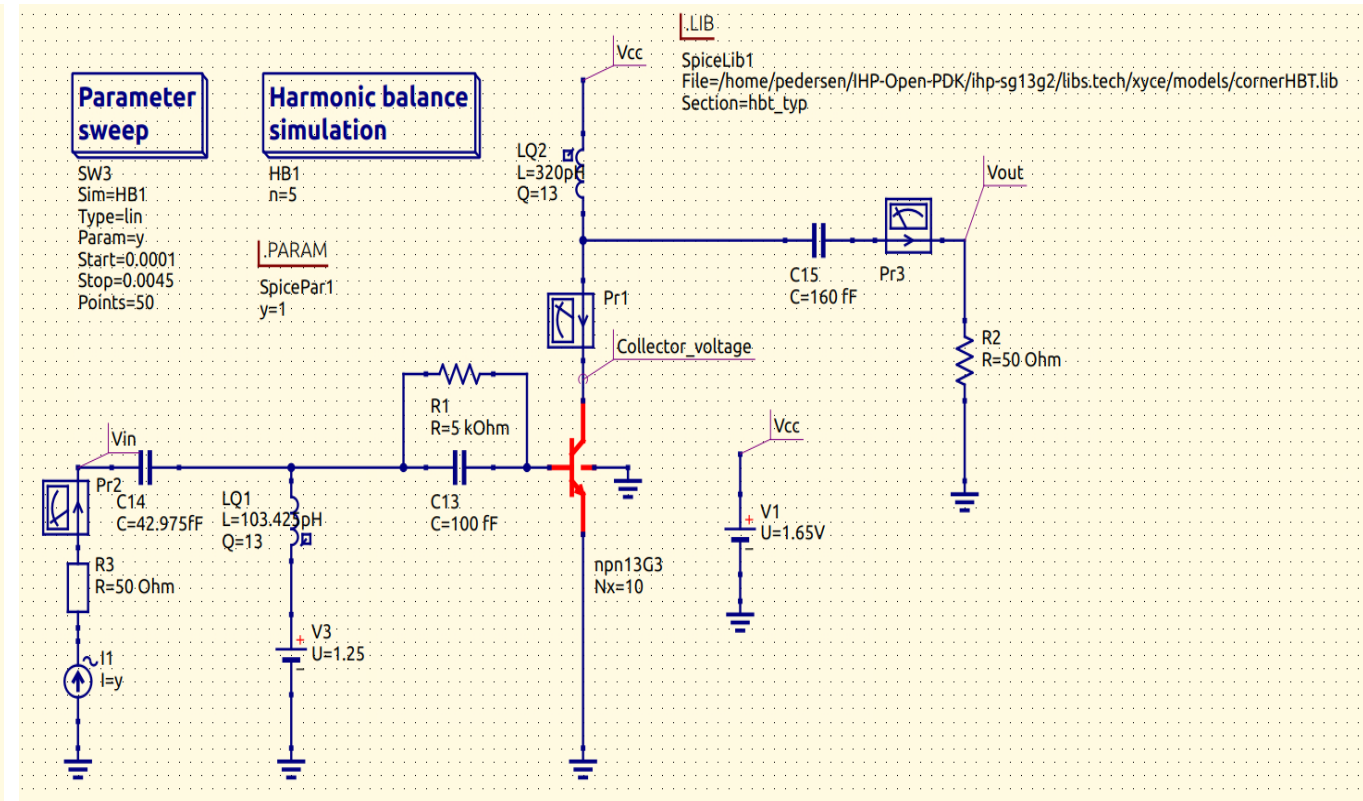
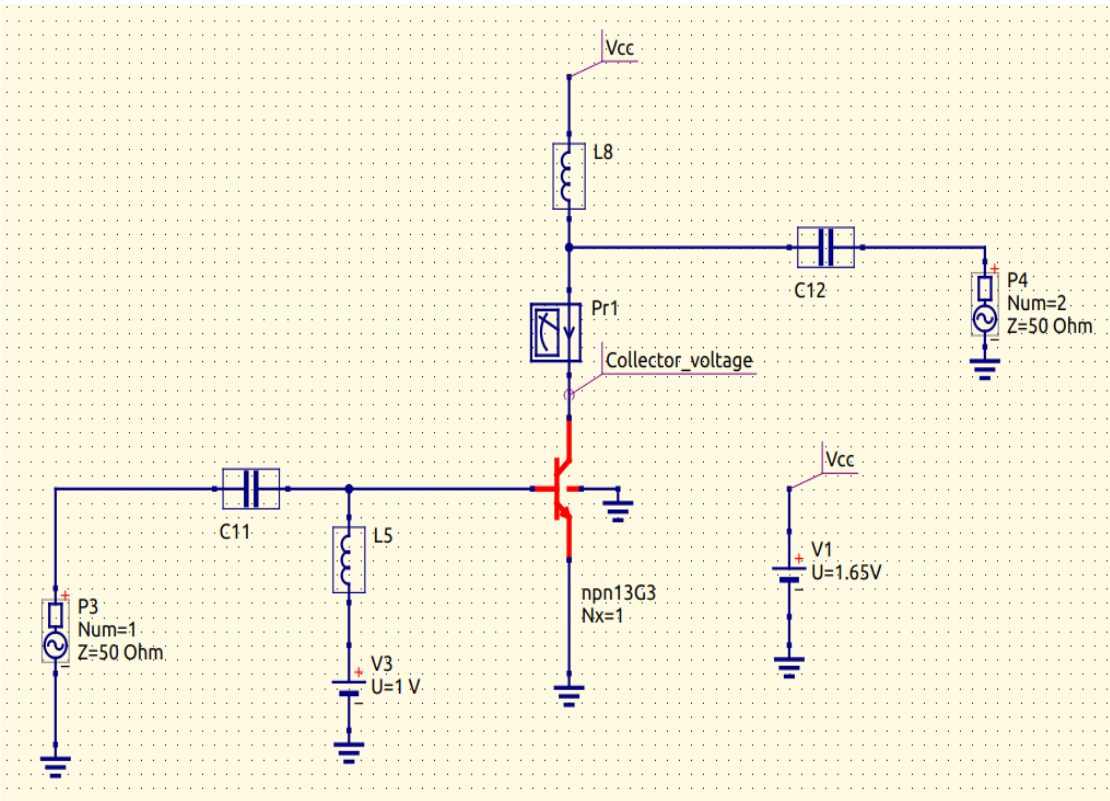
- 0 Was something from yesterday unclear?
- 0 Are we going to fast?
- 0 Do you need more time for the small exercise?

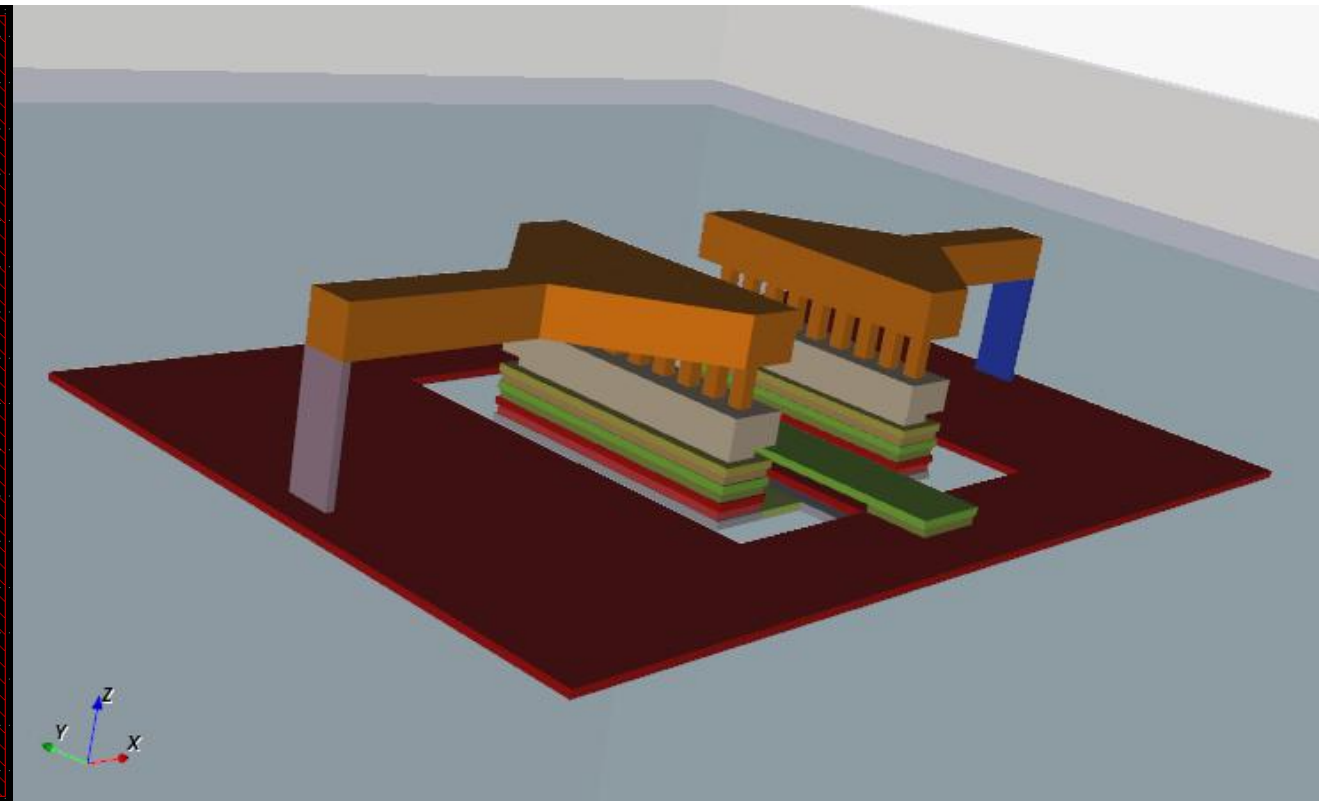
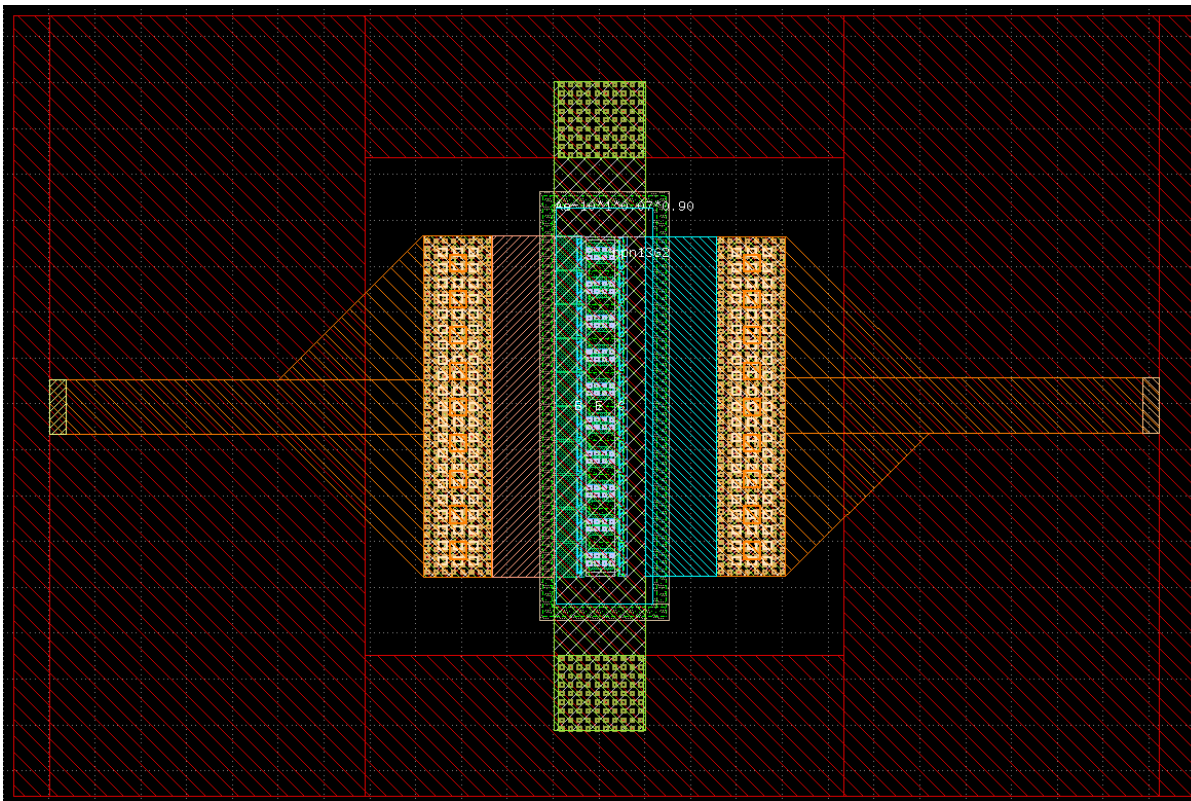
Overview Of Todays Plan



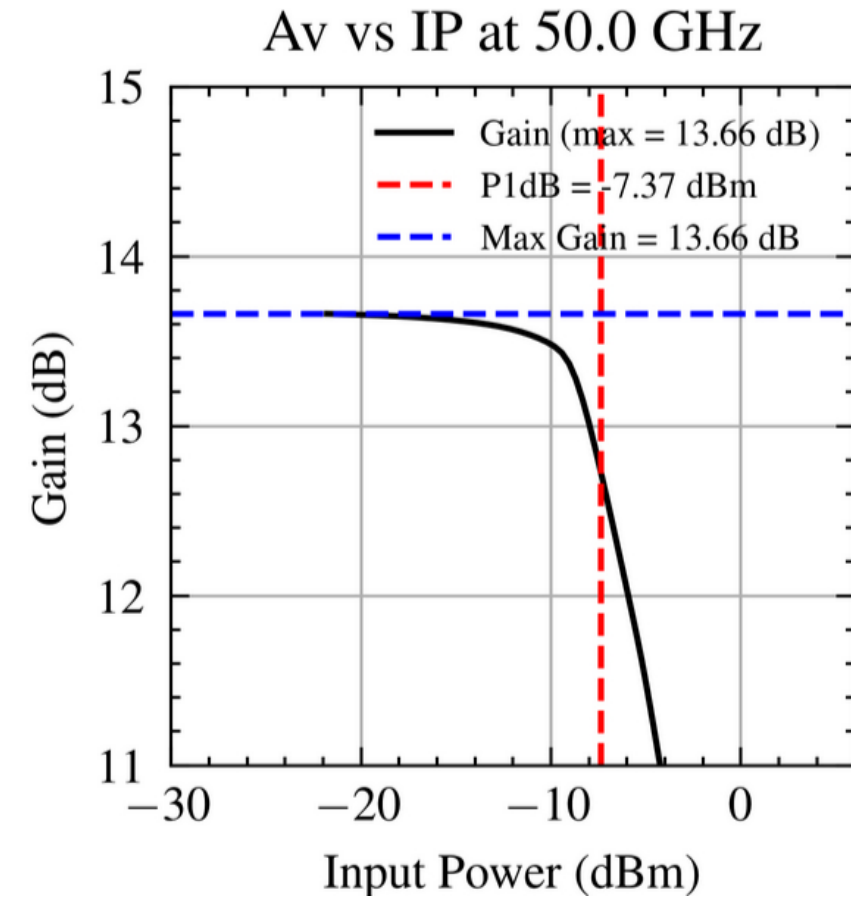
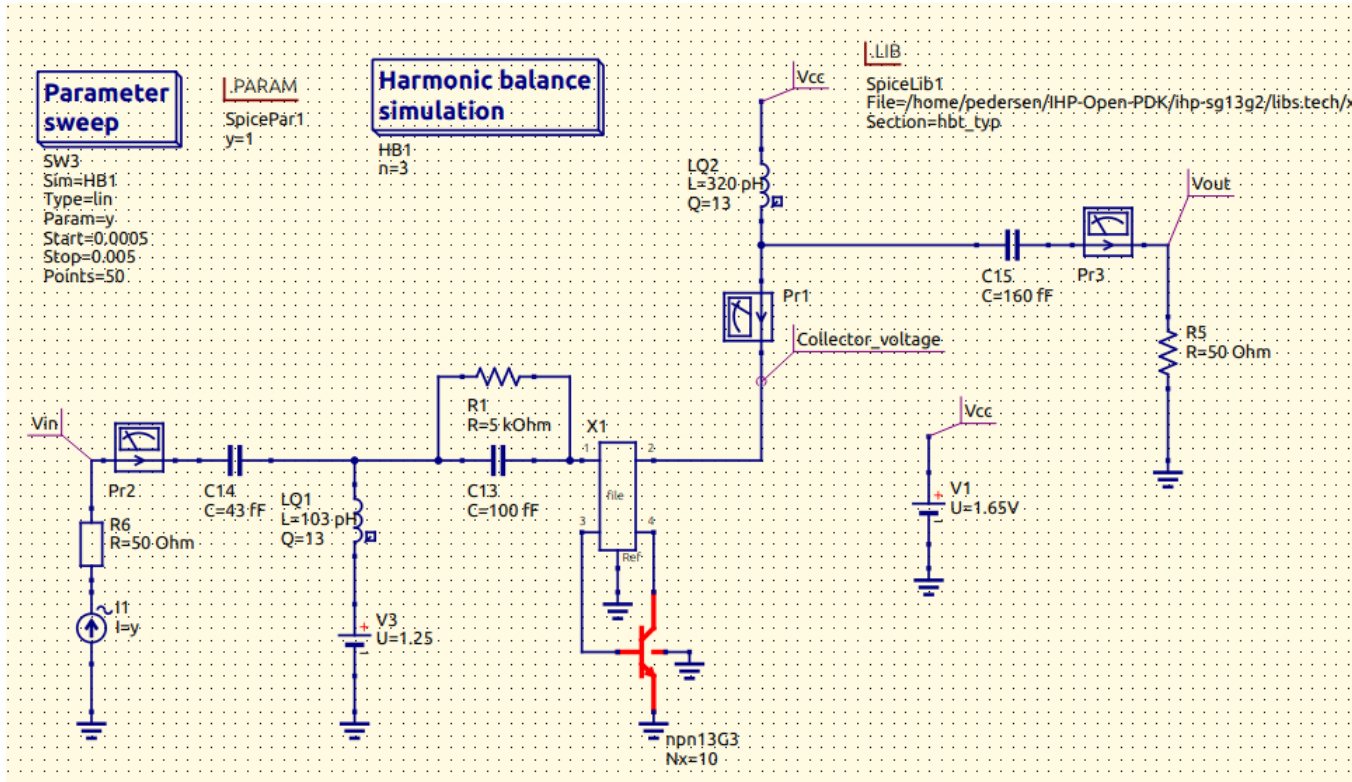
- 0 **Exploring QUCS-S:** Small introduction to Qucs-s
- 0 **MPA Schematic:** Biasing and Initial matching
- 0 **Compression Point:** Non-Linear analysis using xyce to determine compression point
- 0 **OpenEMS & Palace:** Expert talk and workshop by Volker Muehlhaus

Single Stage 50 GHz MPA





Post Processing



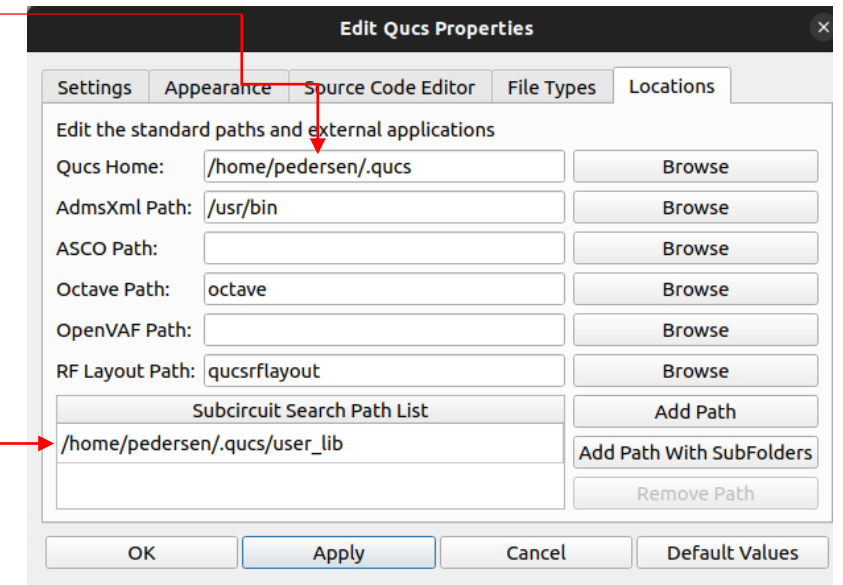
Setting Up Qucs-s



- 0 Launch Qucs by writing the following in the command line: `qucs-s`
- 0 Navigate to the following location in Qucs: *file -> Application settings -> Locations tab*

Change to your own path!

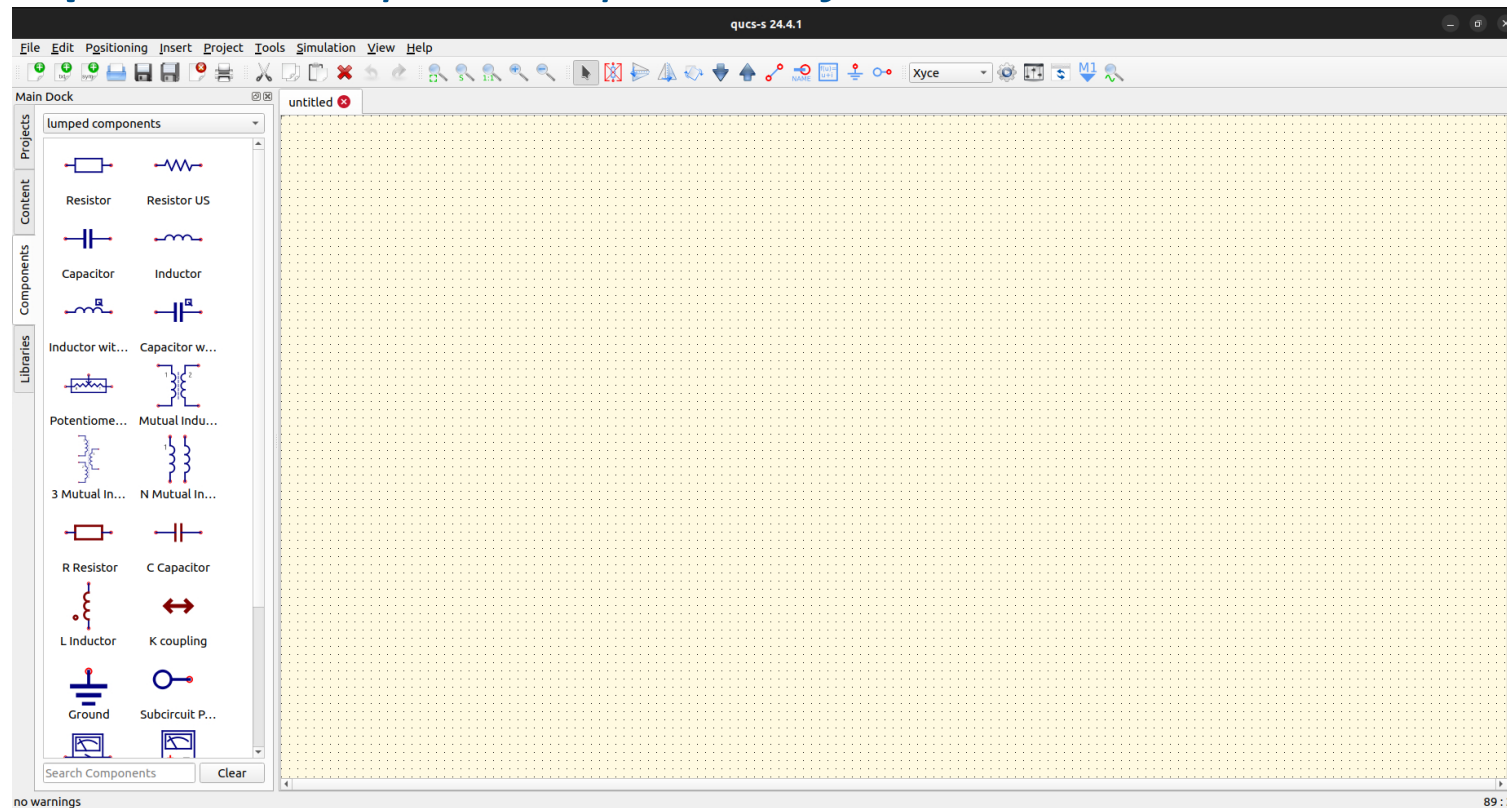
Press “Ctrl+H” to show hidden folders



Simple Navigation in QUCS-S



- 0 To launch without specific file reference: `qucs-s`
- 0 Opening a specific file: `qucs-s -i path/to/file.sch`

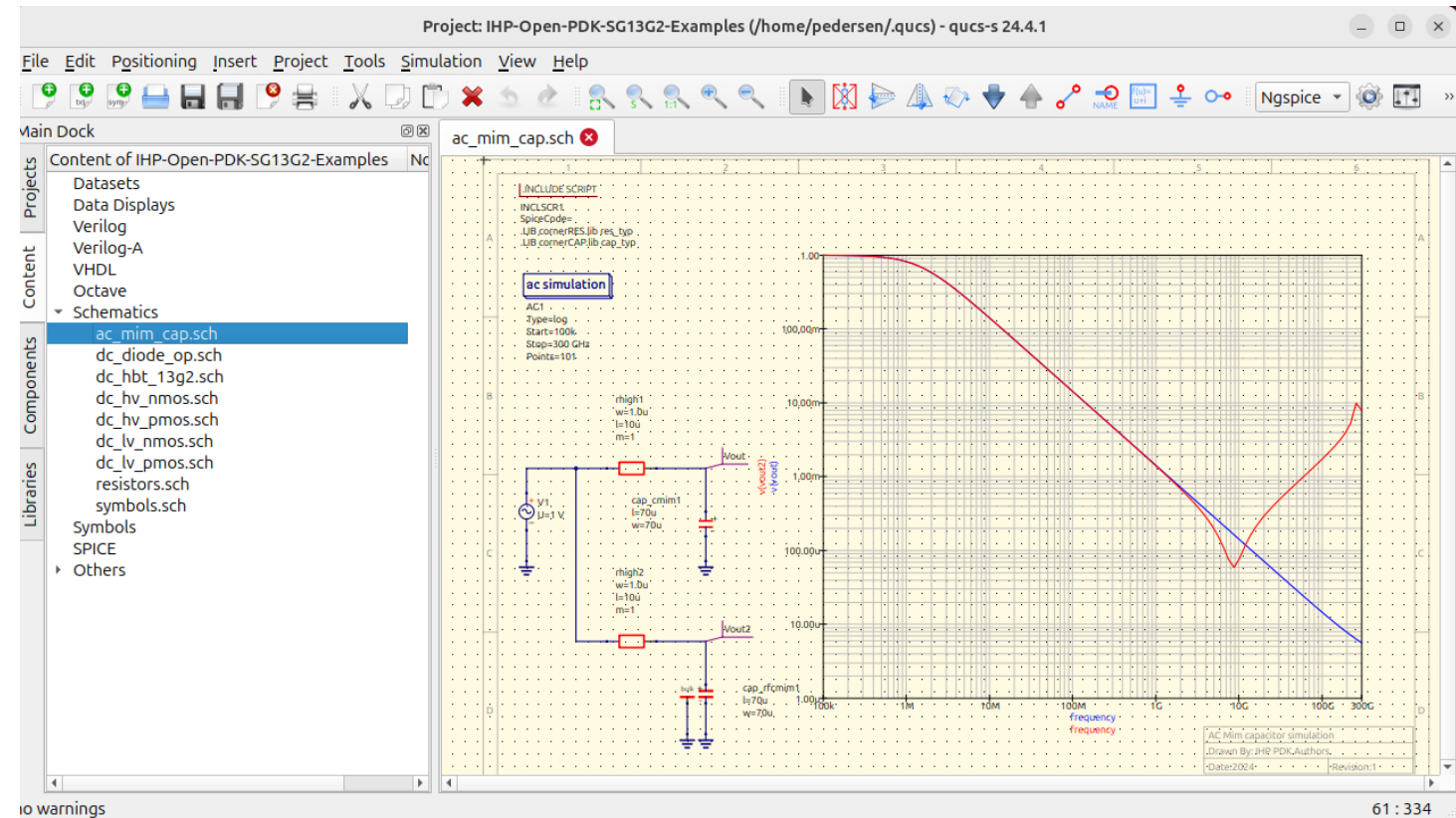
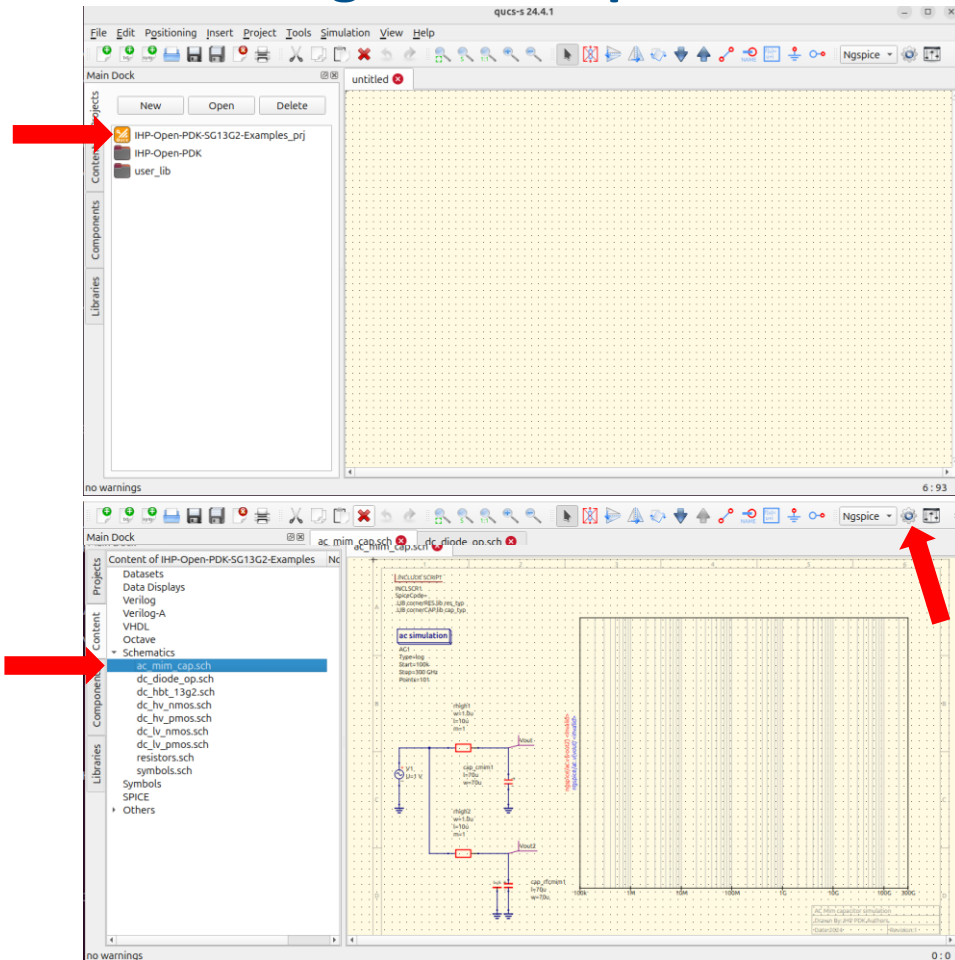


Qucs Shortcuts: <https://qucs-help.readthedocs.io/en/0.0.18/short.html>

Simple Navigation in QUCS-S



Running An Example

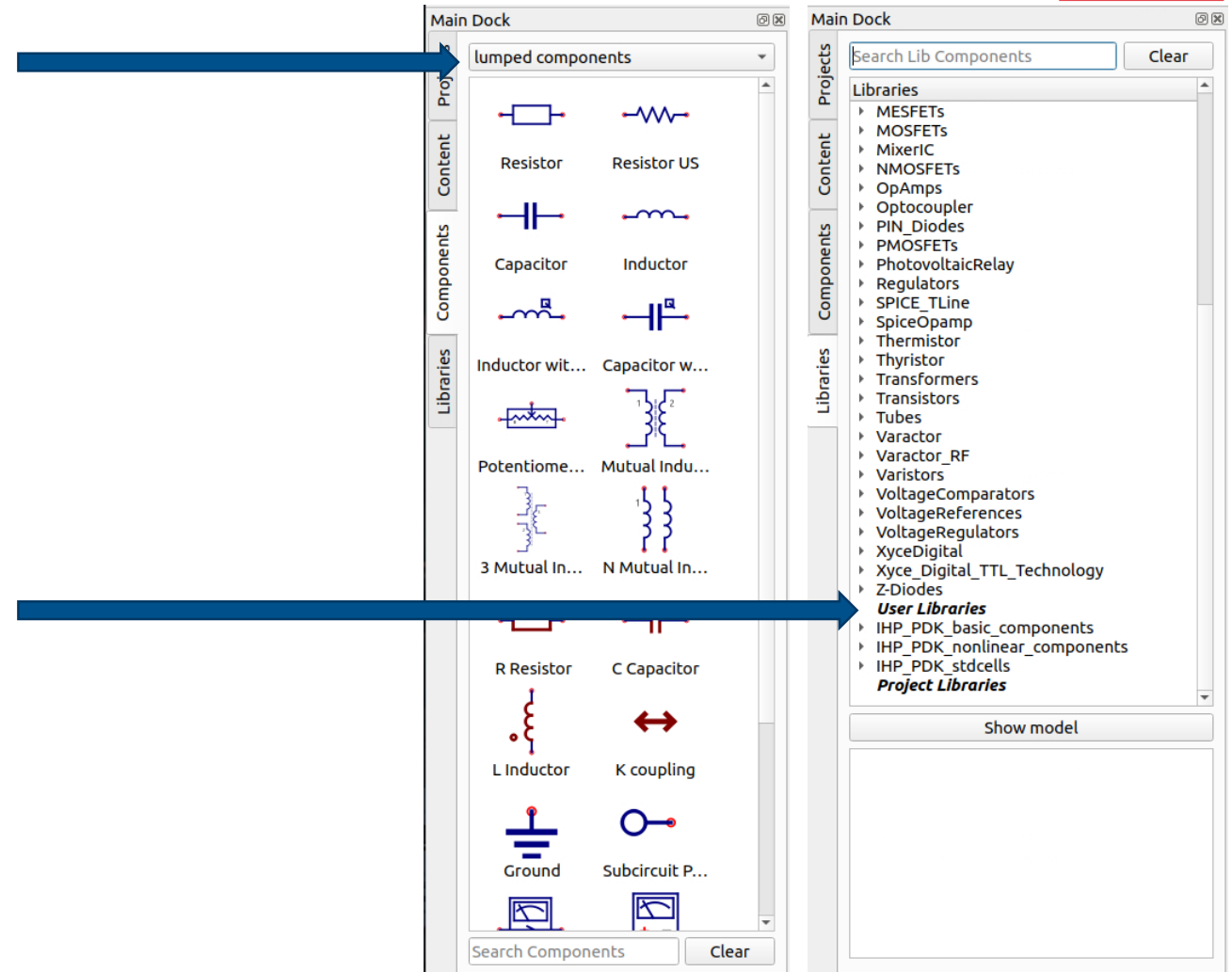


Simple Navigation in QUCS-S

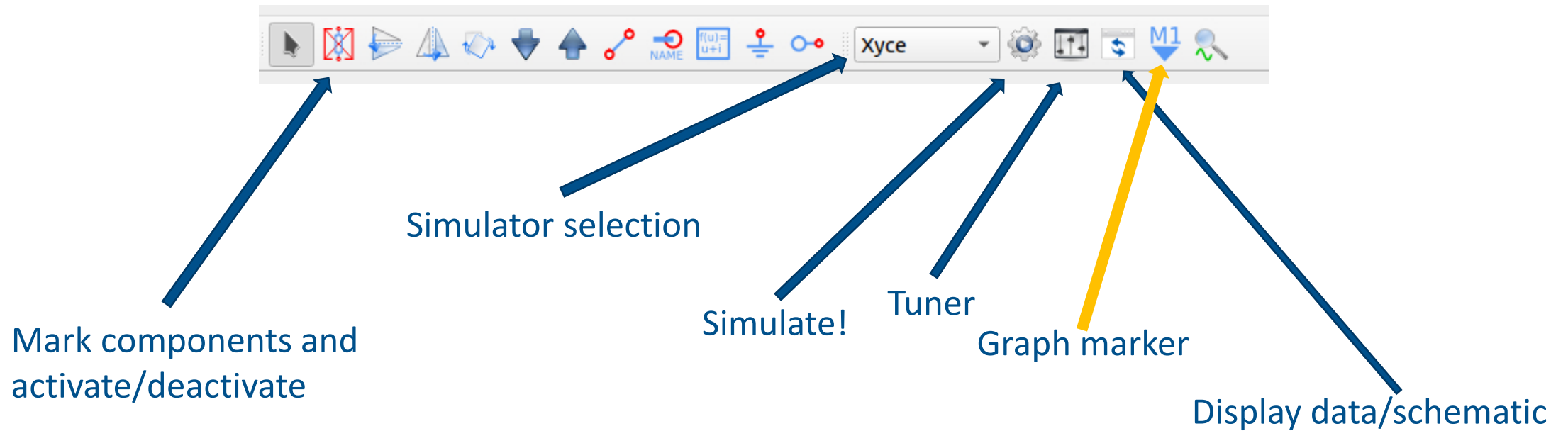


Main Dock

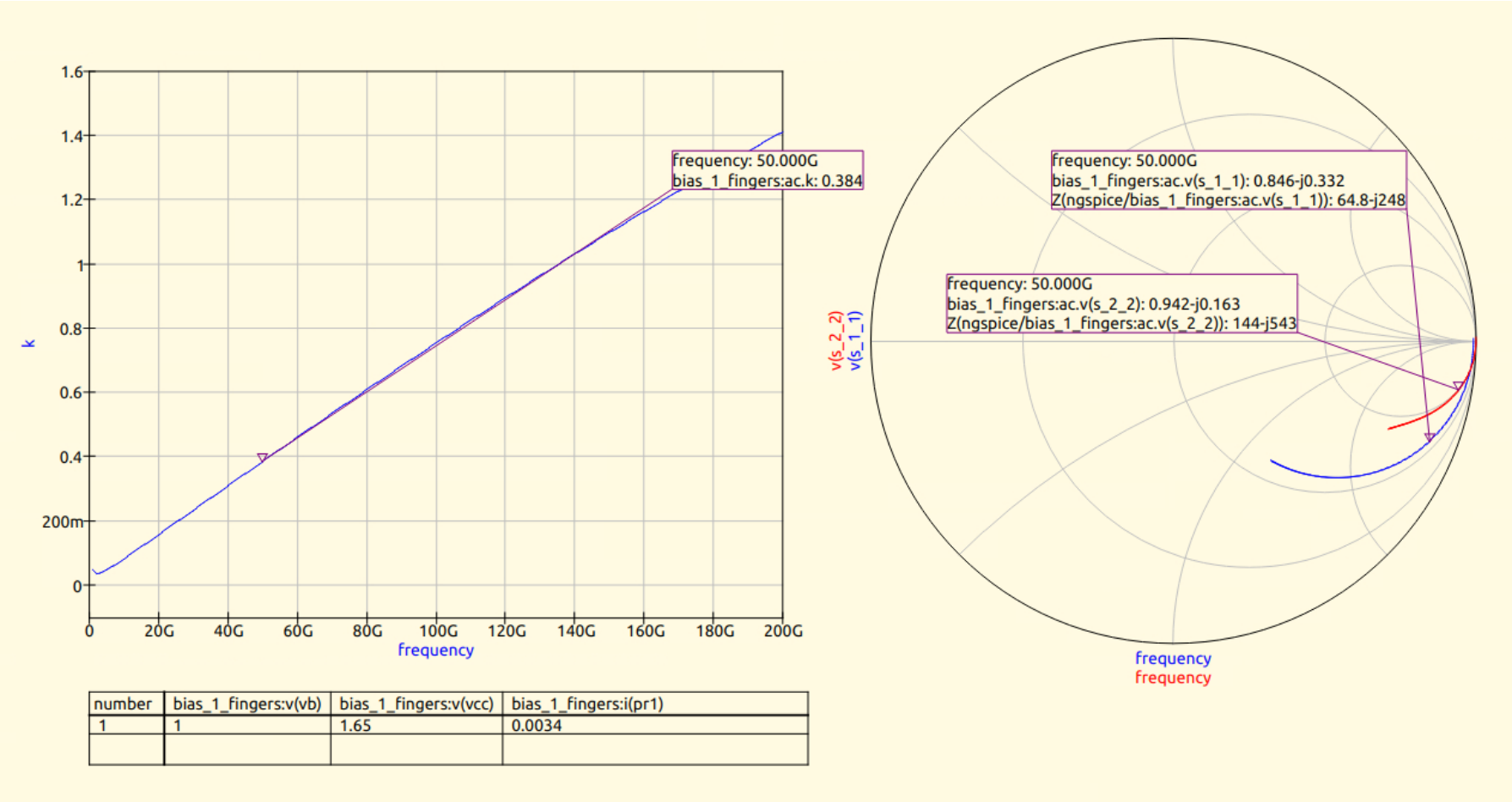
- Components: Stores standard libs and simulation blocks etc..
- The library panes stores the Open PDK components with the symbol passives (simple drag and drop)



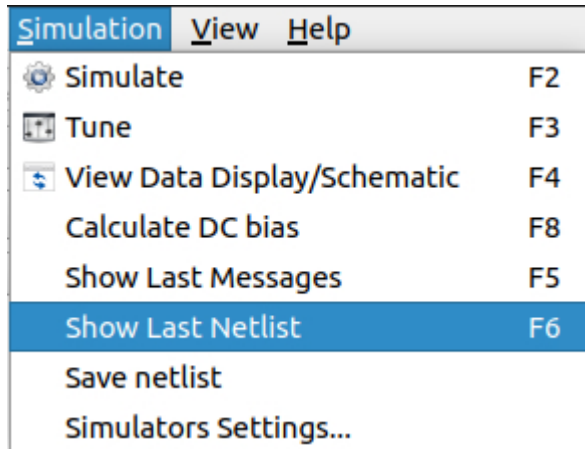
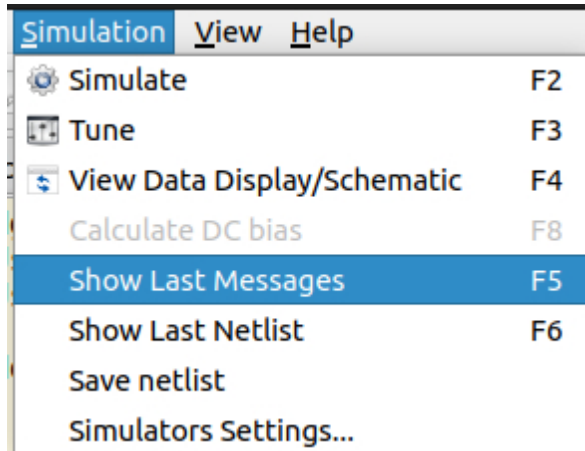
Simple Navigation in QUCS-S



Simple Navigation in QUCS-S



Simple Navigation in QUCS-S



```
Ngspice started...
Using SPARSE 1.3 as Direct Linear Solver
Using SPARSE 1.3 as Direct Linear Solver

Note: No compatibility mode selected!

Circuit: * qucs 24.4.1 /home/pedersen/projects/ihp-analogacademy/modules/module_2_50ghz_mpa/part_1_biasing/schematic/bias_1_fingers/bias_1_fingers.sch
Doing analysis at TEMP = 27.000000 and TNOM = 27.000000

No. of Data Rows : 200
binary raw file "spice4qucs.sp1.plot"
Reset re-loads circuit * qucs 24.4.1 /home/pedersen/projects/ihp-analogacademy/modules/module_2_50ghz_mpa/part_1_biasing/schematic/bias_1_fingers/bias_1_fingers.sch
Circuit: * qucs 24.4.1 /home/pedersen/projects/ihp-analogacademy/modules/module_2_50ghz_mpa/part_1_biasing/schematic/bias_1_fingers/bias_1_fingers.sch
Doing analysis at TEMP = 27.000000 and TNOM = 27.000000

No. of Data Rows : 1
Reset re-loads circuit * qucs 24.4.1 /home/pedersen/projects/ihp-analogacademy/modules/module_2_50ghz_mpa/part_1_biasing/schematic/bias_1_fingers/bias_1_fingers.sch
Circuit: * qucs 24.4.1 /home/pedersen/projects/ihp-analogacademy/modules/module_2_50ghz_mpa/part_1_biasing/schematic/bias_1_fingers/bias_1_fingers.sch

ngspice-43 done
* Qucs 24.4.1 /home/pedersen/projects/IHP-AnalogAcademy/modules/module_2_50GHz_MPA/part_1_biasing/schematic/bias_1_fingers/bias_1_fingers.sch
.INCLUDE "/usr/local/share/qucs-s/xspice_cmlib/include/ngspice_mathfunc.inc"

.SUBCKT IHP_PDK_nonlinear_components_npn13G2 gnd c b e bn Nx=1
X1 c b e bn npn13G2 Nx=(Nx)
.ENDS

.LIB cornerHBT.lib hbt_typ

VP3 _net0 0 dc 0 ac 0.632456 SIN(0 0.632456 1MEG) portnum 1 z0 50
C11 _net0 _net1 1u
L5 _net2 _net1 1u
L8 _net3 Vcc 1u
VPr1 _net3 Collector_voltage DC 0
V3 _net2 0 DC 0.97
C12 _net3 _net4 1u
VP4 _net4 0 dc 0 ac 0.632456 SIN(0 0.632456 1MEG) portnum 2 z0 50
V1 Vcc 0 DC 1.65

Xnnp13G3 0 Collector_voltage _net1 0 gnd IHP_PDK_nonlinear_components_npn13G2 Nx=10

.control
SP LIN 200 1G 200G
let k = (1 - abs(s_1_1)^2 - abs(s_2_2)^2 + abs(s_1_1 * s_2_2 - s_1_2 * s_2_1)^2) / (2 * abs(s_1_2 * s_2_1))
write spice4qucs.sp1.plot S_1_1 Y_1_1 Z_1_1 S_1_2 Y_1_2 Z_1_2 S_2_1 Y_2_1 Z_2_1 S_2_2 Y_2_2 Z_2_2 k
destroy all
reset

op
print v(Collector_voltage) i(VPr1) v(Vcc) > spice4qucs.dcl.ngspice.dc.print
destroy all
reset

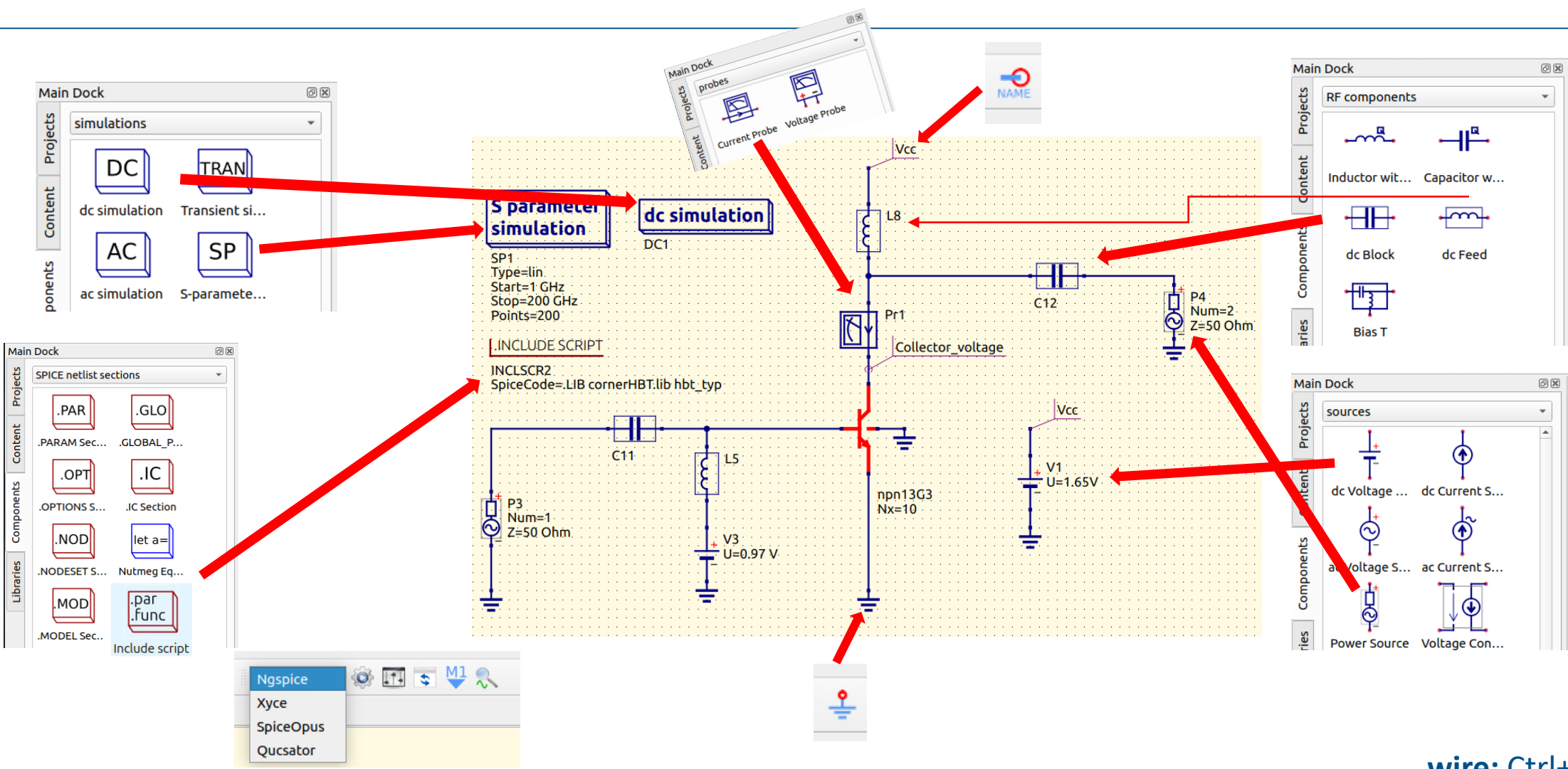
exit
.endc
.END
```

Part 1

Biasing

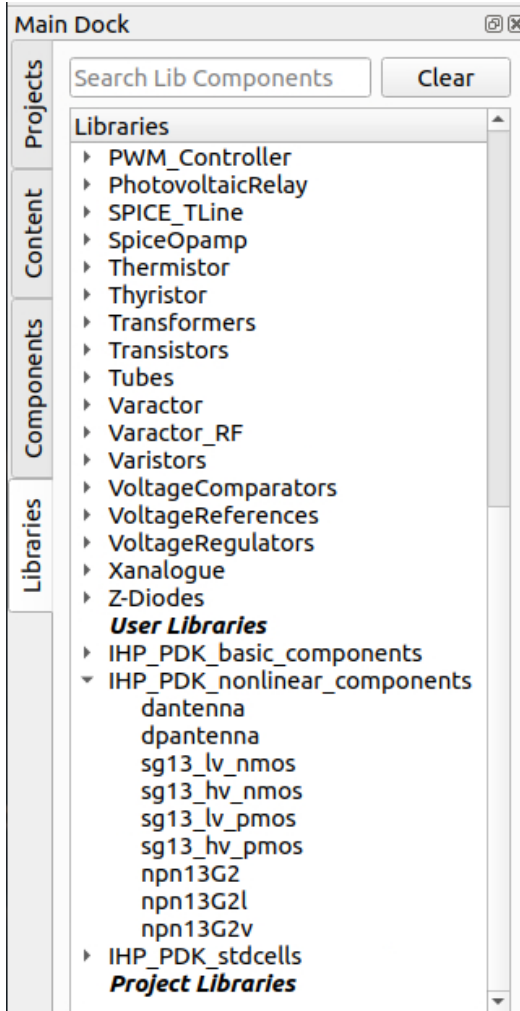
Reference: [modules/module_2_50GHz_MPA/part_1_biasing](#)

Building The Schematic

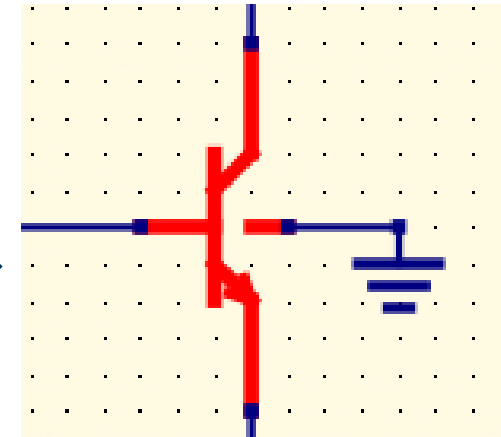


wire: Ctrl+W

Building The Schematic



- User Libraries**
- IHP_PDK_basic_components
- IHP_PDK_nonlinear_components
 - dantenna
 - dpantenna
 - sg13_lv_nmos
 - sg13_hv_nmos
 - sg13_lv_pmos
 - sg13_hv_pmos
 - npn13G2
 - npn13G2l
 - npn13G2v
- IHP_PDK_stdcells
- Project Libraries**

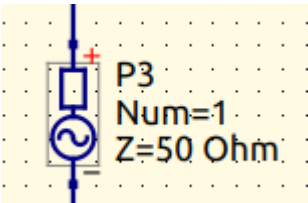


Check Component Parameters



You have to set different parameters that aren't necessarily shown!

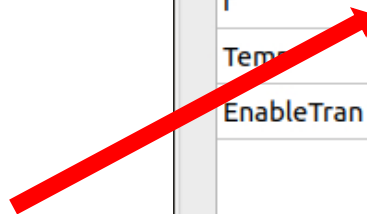
Example:



Double Click



!!!



Edit Component Properties

ac power source

Name: ☒ display in schematic

Properties

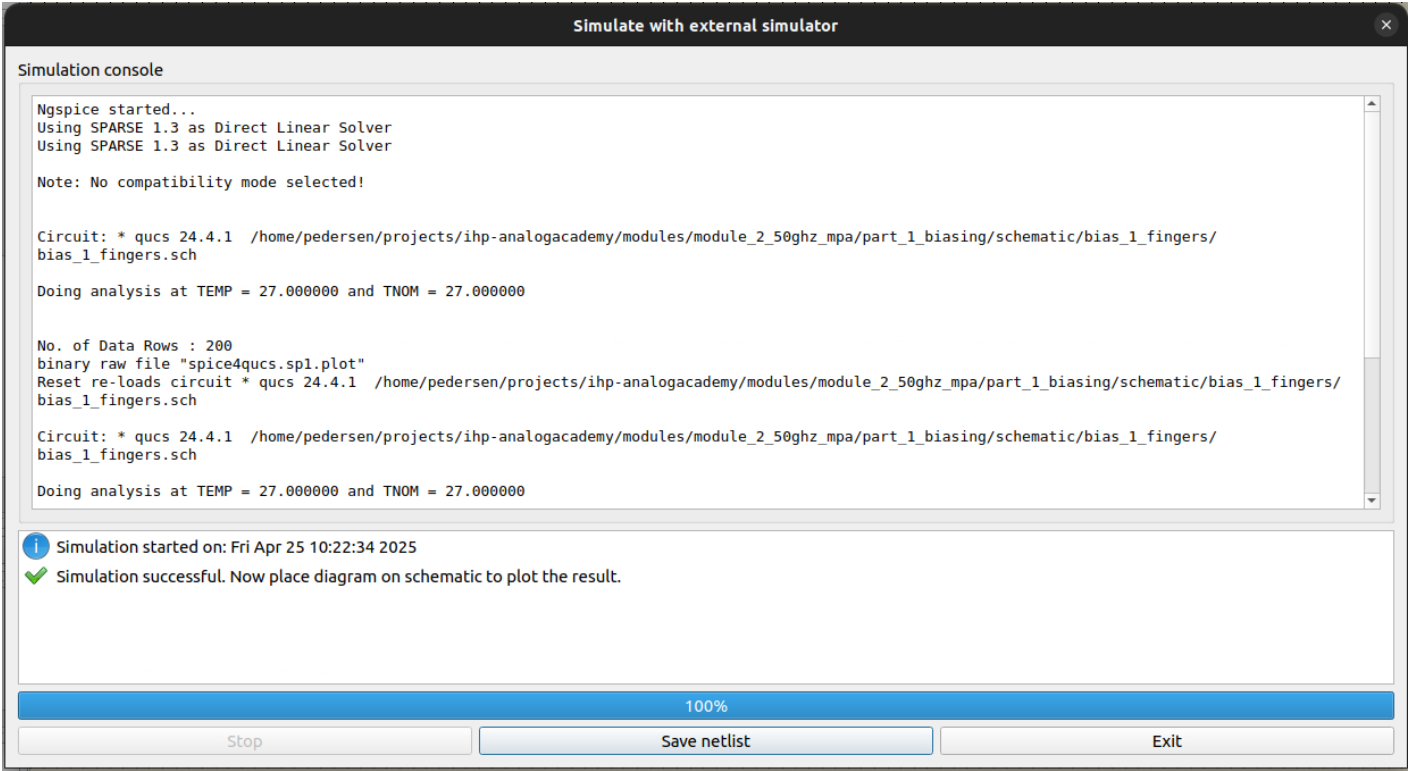
Name	Value	display	Description
Num	1	yes	number of the port
Z	50 Ohm	yes	port impedance
P	0 dBm	no	(available) ac power in Watts
f	50 GHz	no	frequency in Hertz
Temp	26.85	no	simulation temperature in degree Celsius
EnableTran	true	no	enable transient model as sine source [true,false]

Num
number of the port

☒ display in schematic

Same for Port 2

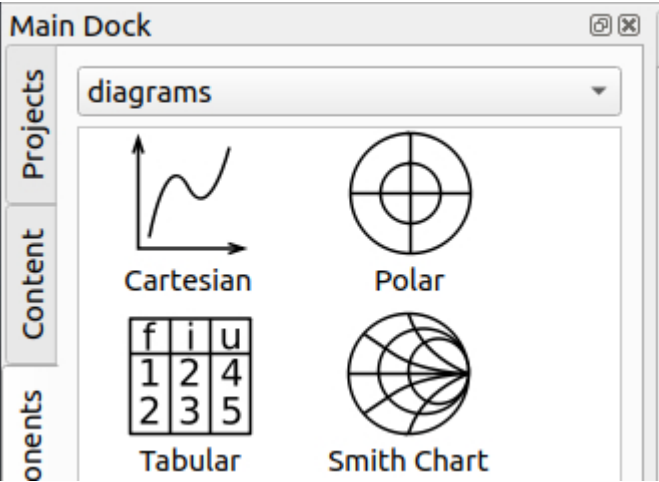
Running The Simulation



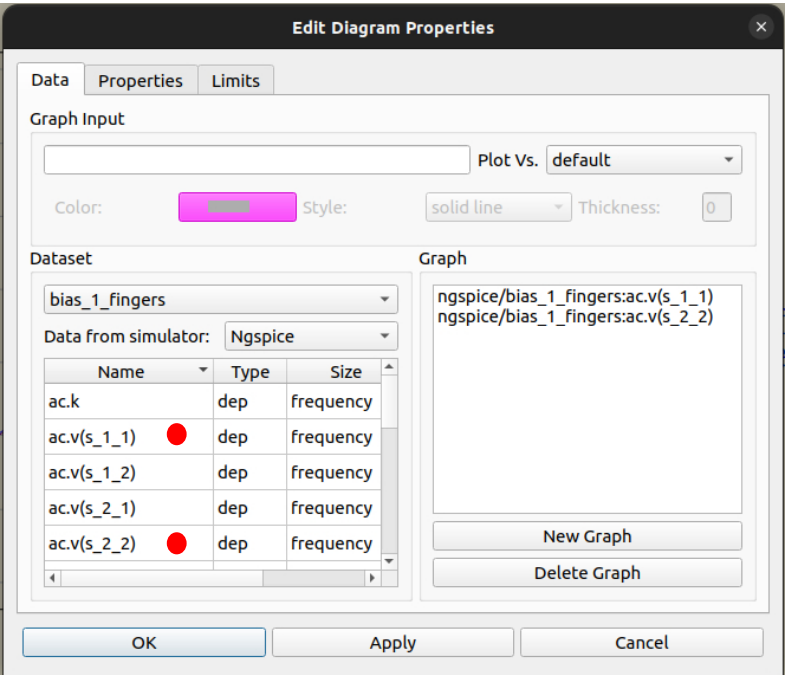
Viewing The First Results



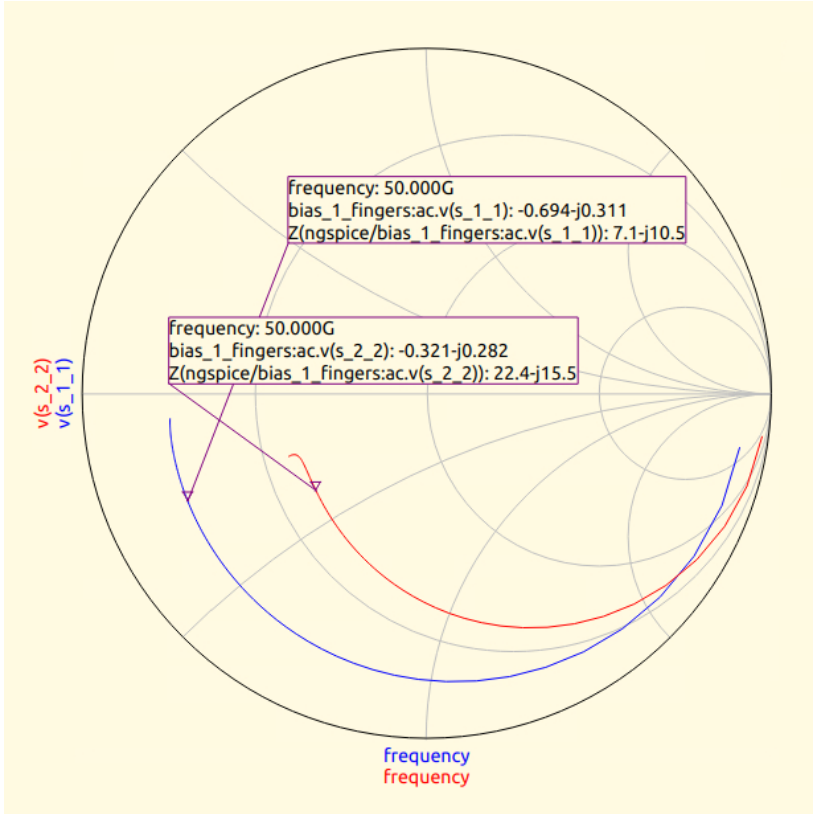
Start by switching to data display 



Select the Smith Chart

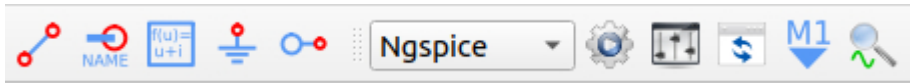


Double click the S11 and S22



View the data and insert markers with 
Use arrow keys to move the marker

Inserting Equation



Nutmeg
NutmegEq2
Simulation=ALL
y=1



Name: NutmegEq1 ☒ display in schematic

Properties

Name	Value	display	Description
Simulation	SP1	yes	Simulation name
k	$(1 - \text{abs}(s_{1_1})^2 - \text{abs}(s_{2_2})^2 + \text{abs}(s_{1_1} * s_{2_2} - s_{1_2} * s_{2_1})^2) / (2 * \text{abs}(s_{1_2} * s_{2_1}))$	yes	

Simulation
Simulation name
SP1

☒ display in schematic



Nutmeg
NutmegEq1
Simulation=SP1
 $k = (1 - \text{abs}(s_{1_1})^2 - \text{abs}(s_{2_2})^2 + \text{abs}(s_{1_1} * s_{2_2} - s_{1_2} * s_{2_1})^2) / (2 * \text{abs}(s_{1_2} * s_{2_1}))$

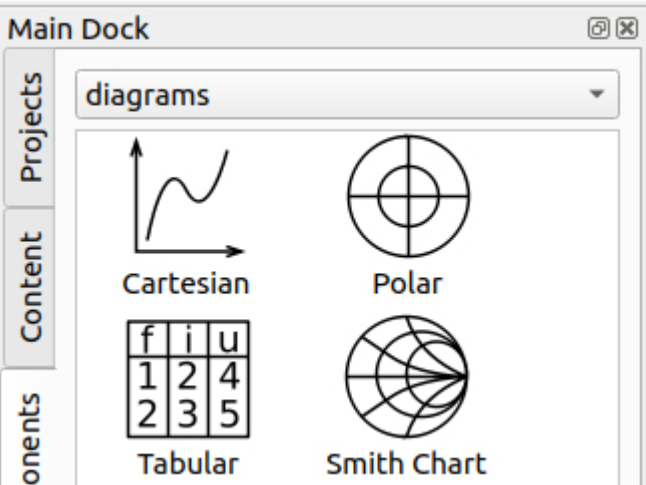
Stability / K-Factor



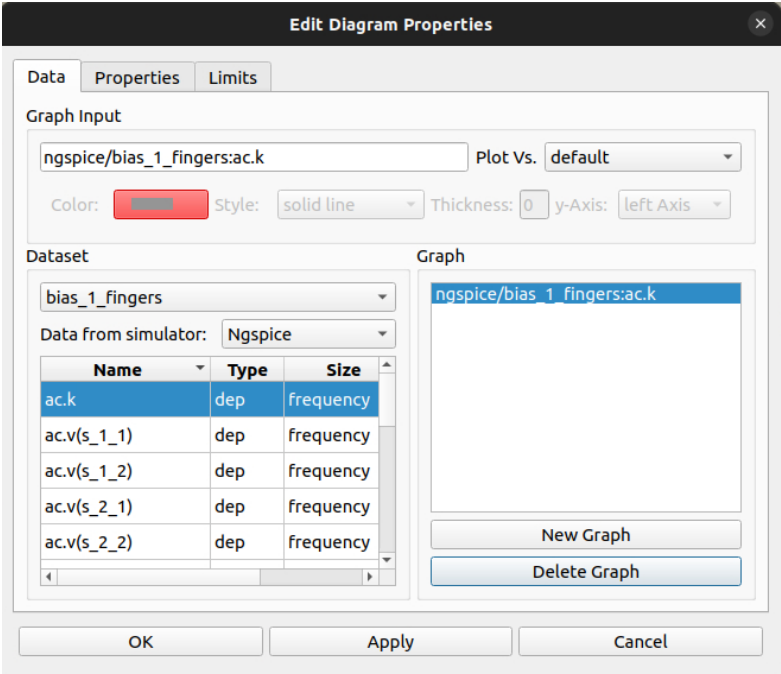
$$k = (1 - \text{abs}(s_{1_1})^2 - \text{abs}(s_{2_2})^2 + \text{abs}(s_{1_1} * s_{2_2} - s_{1_2} * s_{2_1})^2) / (2 * \text{abs}(s_{1_2} * s_{2_1}))$$



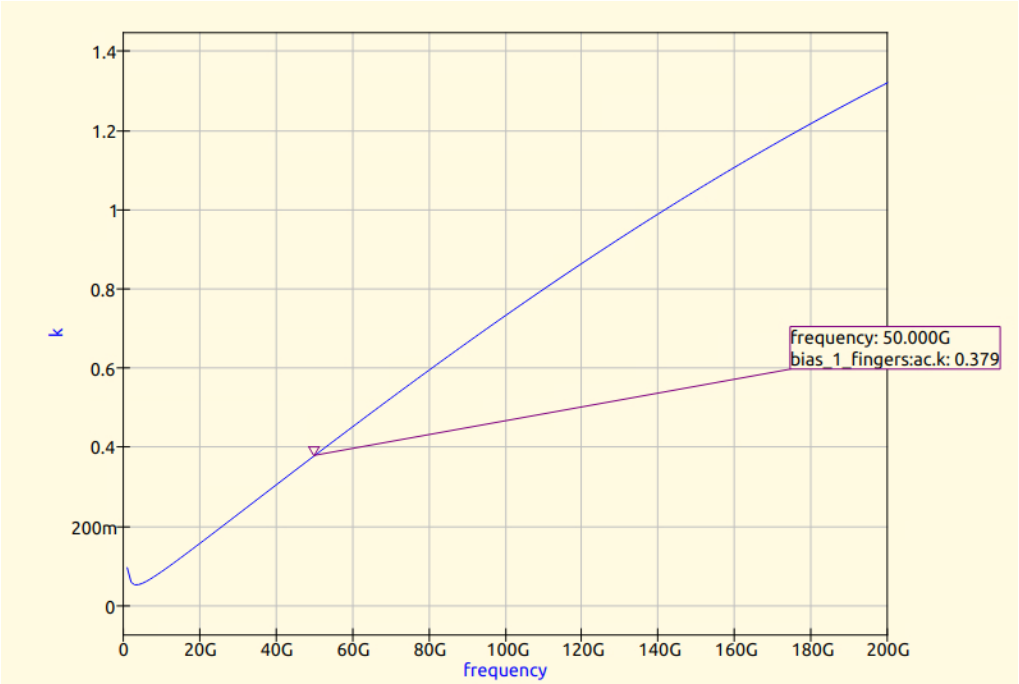
Viewing The K-Factor



Select the Cartesian plot

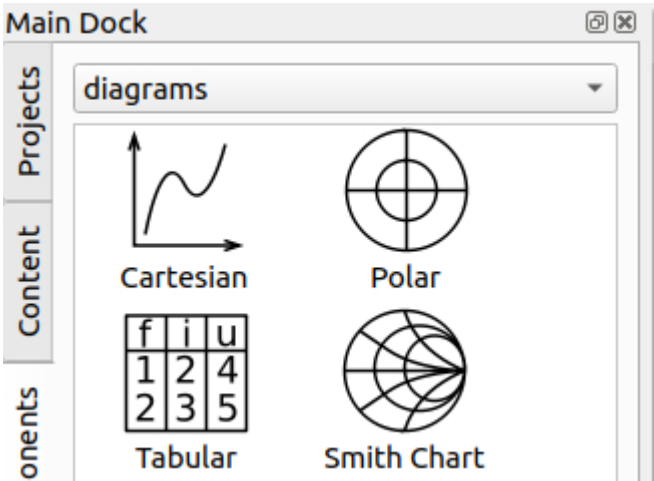


Select the K-factor

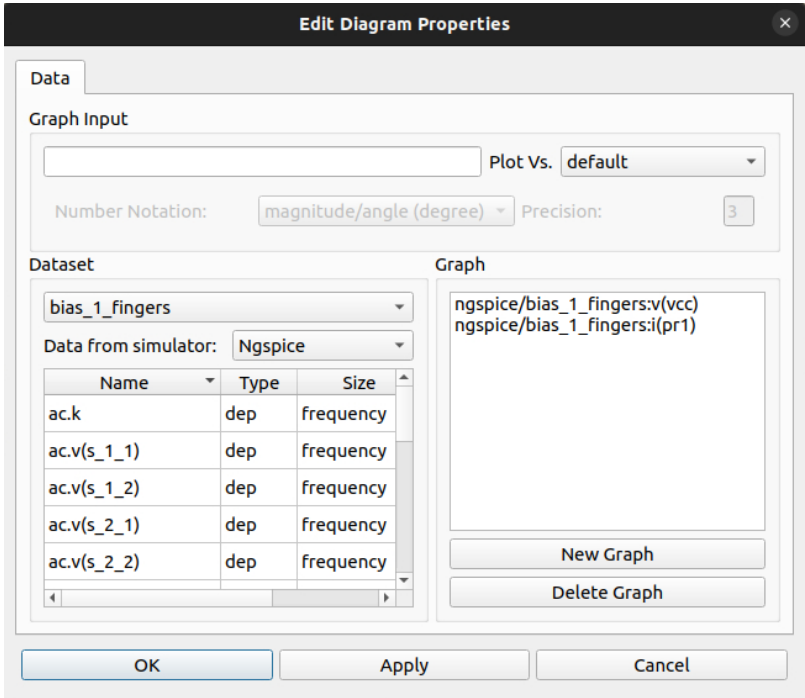


Clearly not stable!

Viewing Data From Table



Select tabular



Select the collector voltage and current

number	bias_1_fingers:v(vcc)	bias_1_fingers:i(pr1)
1	1.65	0.0269

Checking some bias points

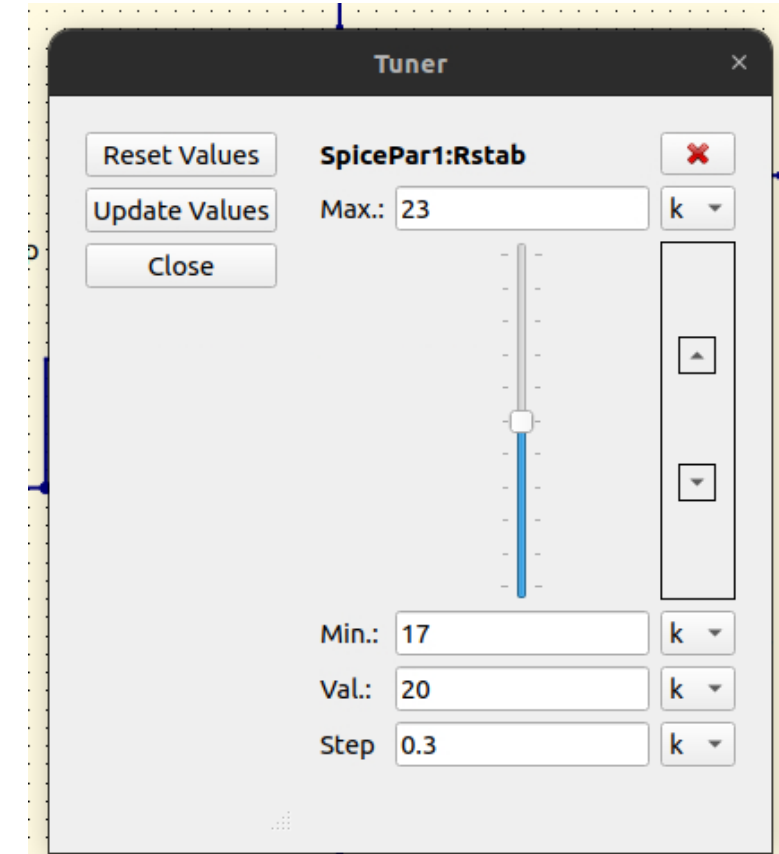
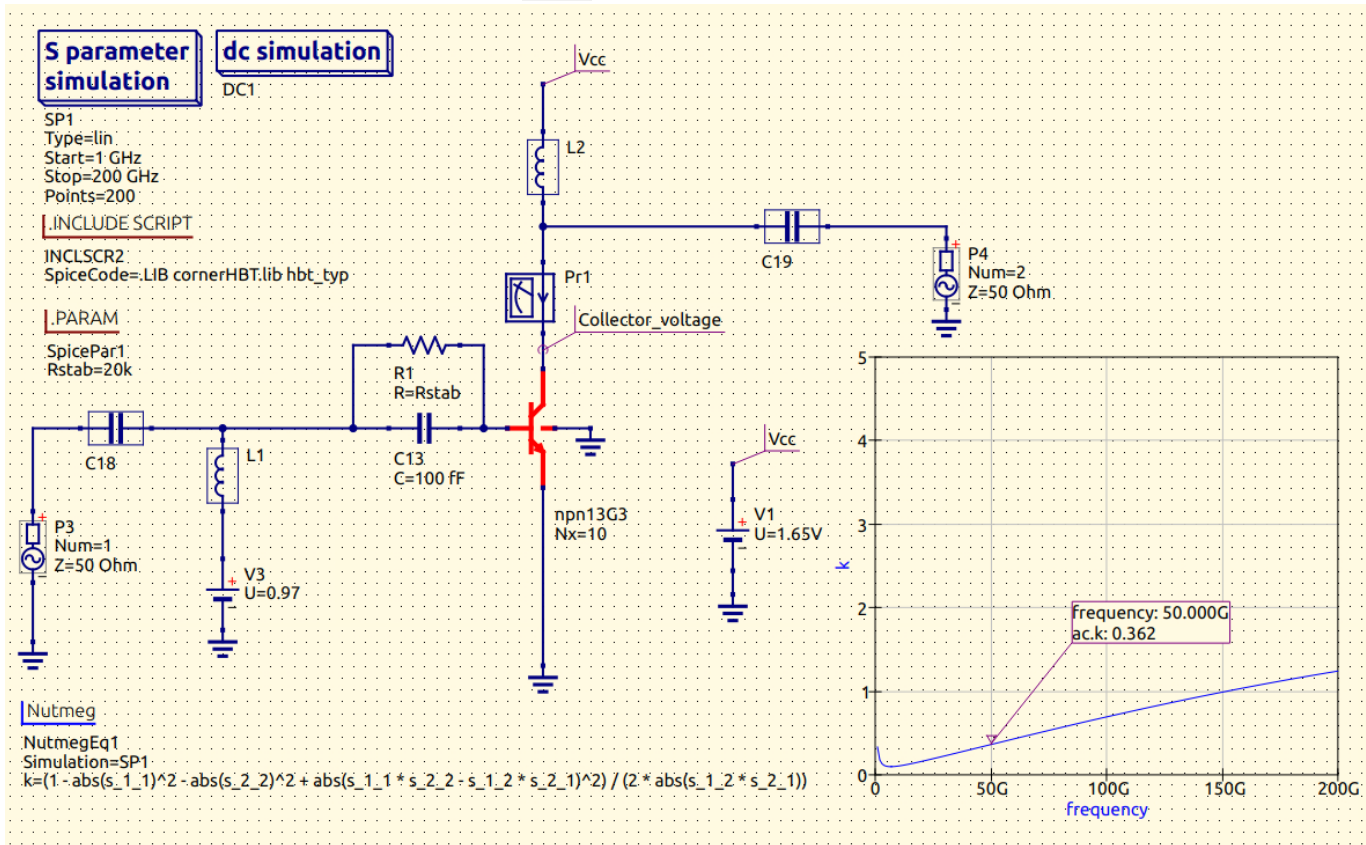
- We want to tune the resistance of the base resistor!
- We introduce a .Param block:



Tuning Components



Instantiate the Cartesian plot inside the schematic and view the K-factor From here open the tuning function 



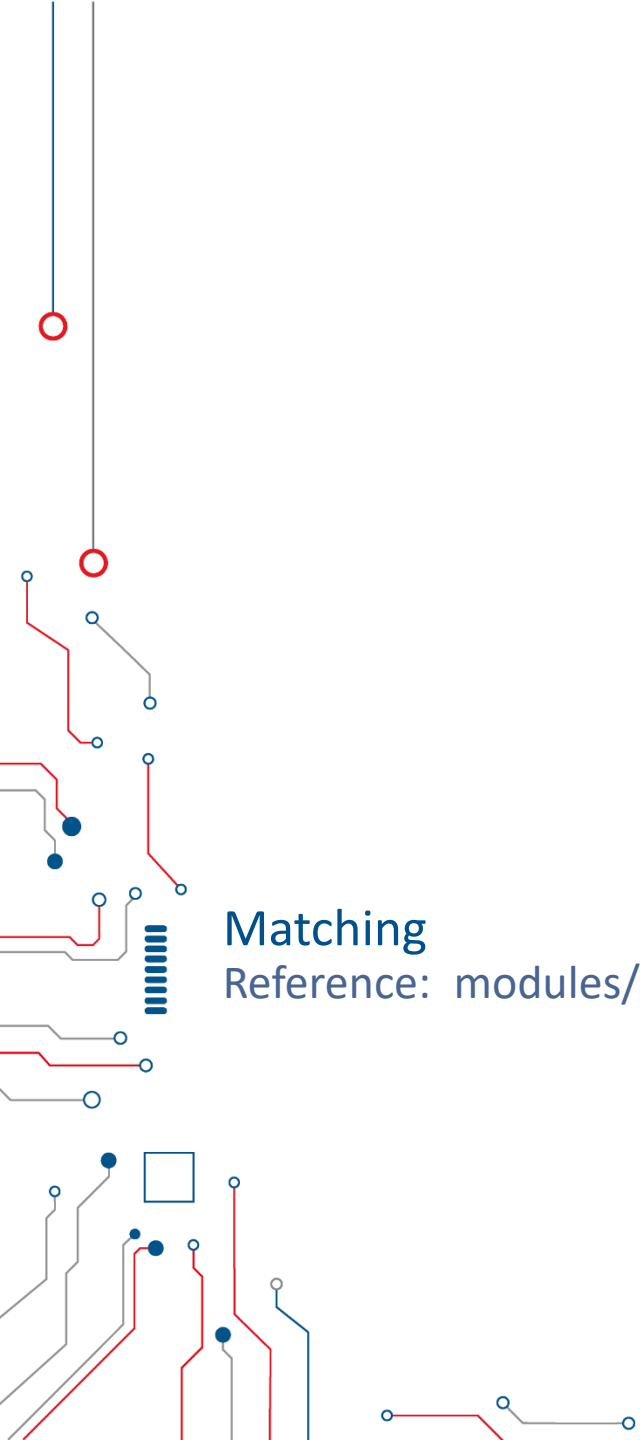
Select the Rstab parameter in the .param block and start tuning

Stabilize The Amplifier 😊



- 0 Try as best as you can to stabilize the amplifier
- 0 You can employ any method you like (emitter degeneration, biasing etc...)
- 0 Its not critical to stabilize the amplifier since we only want to create a appropriate flow 😊
- 0 If unsure, stabilize the amp by adjusting components for $K > 1$ at 50 GHz.

Part 2

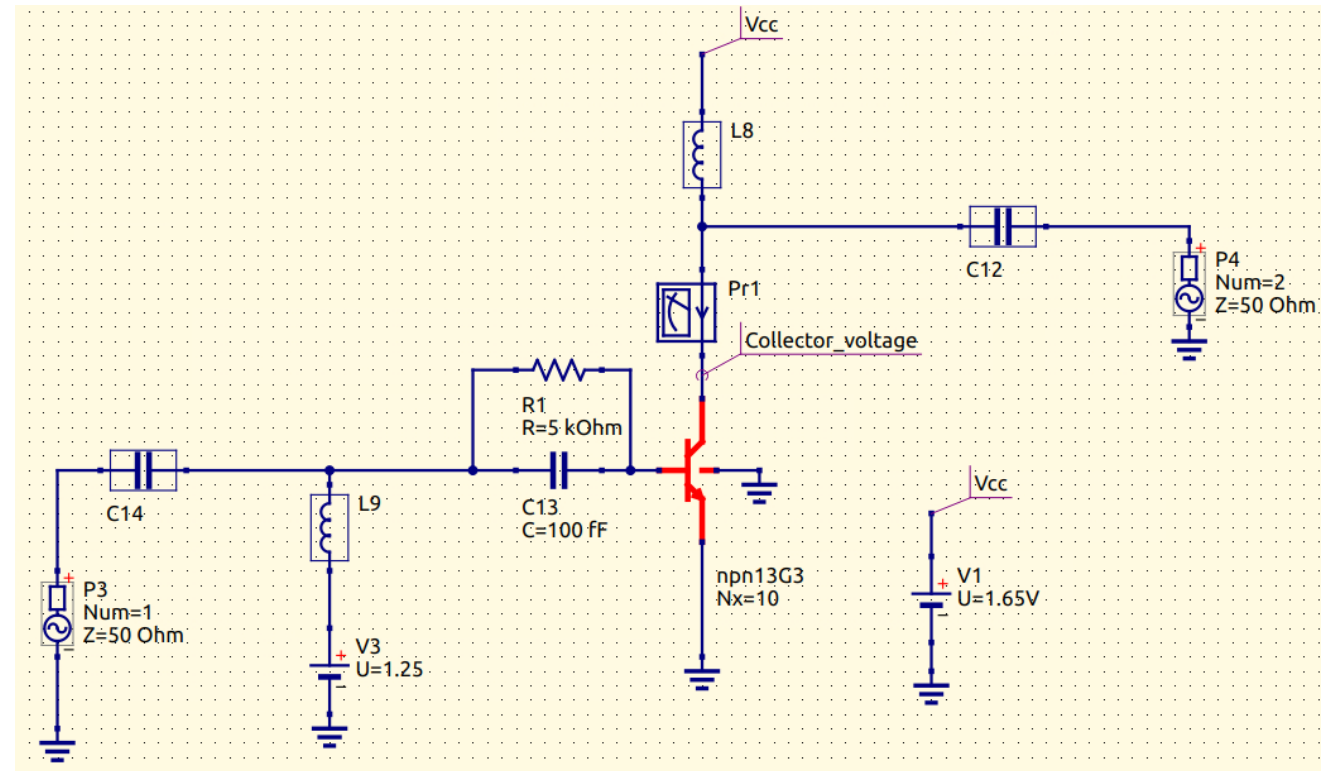
A decorative circuit pattern is located on the left side of the slide. It features a vertical line of red circles at the top, followed by a series of red and blue lines that branch out and connect to various points. There are also several blue dots and a small blue square. The pattern is intricate and resembles a microchip layout.

Matching

Reference: [modules/module_2_50GHz_MPA/part_2_matching_ideal](#)

Matching

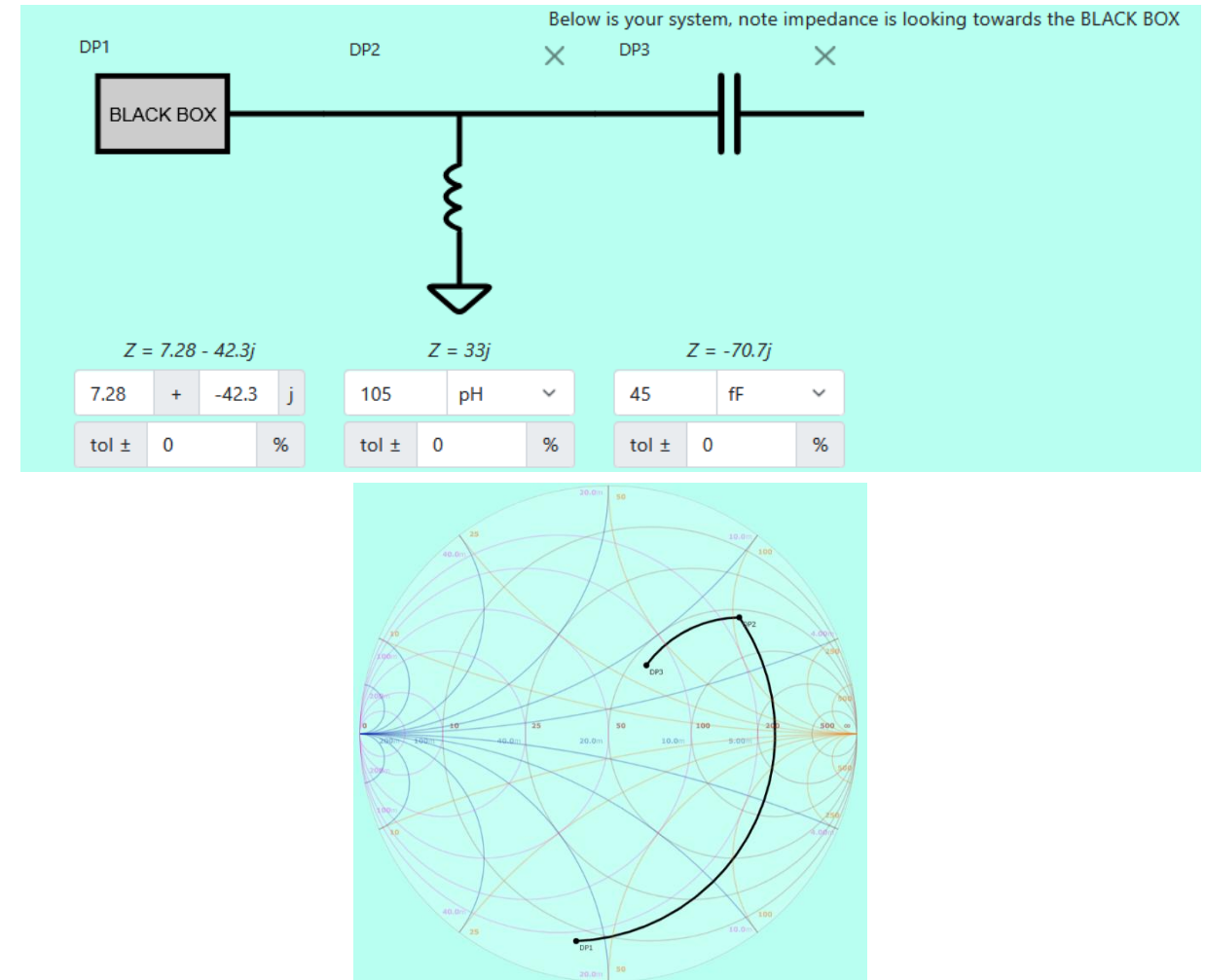
- 0 To move forward we will use the schematic seen to the right
- 0 This is simply done to keep quite consistent with the first iteration of the design earlier
- 0 You may continue with your own schematic to keep better stability factor 😊



Online Matching Tool



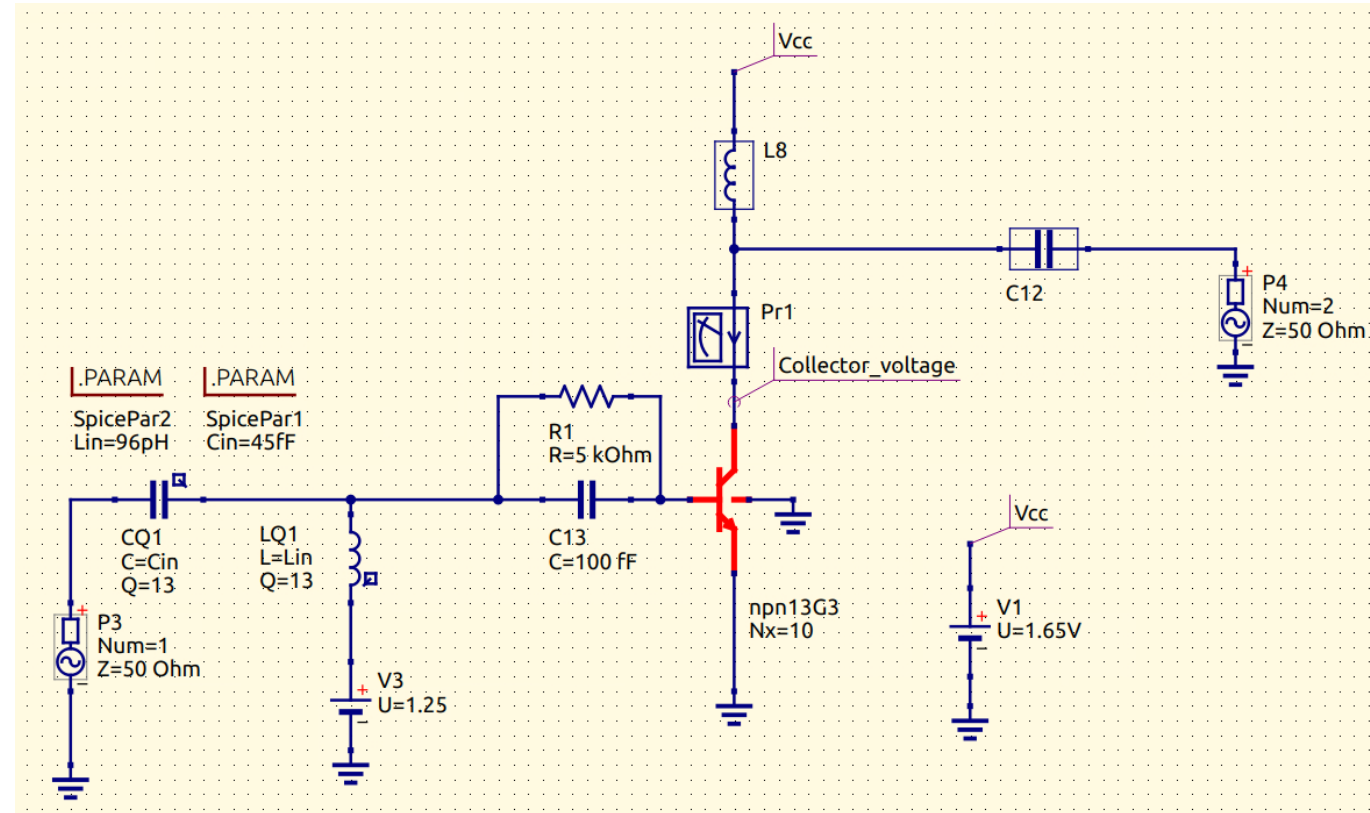
- 0 At this point you can navigate to the following link to match your input section:
https://www.will-kelsey.com/smith_chart/
- 0 You can do the matching in QUCS-S using the matching tool under tools, but not for this kind of section that we wish to create
- 0 When an appropriate matching have been found you can refer to the next slide to insert the components and tune the parameters



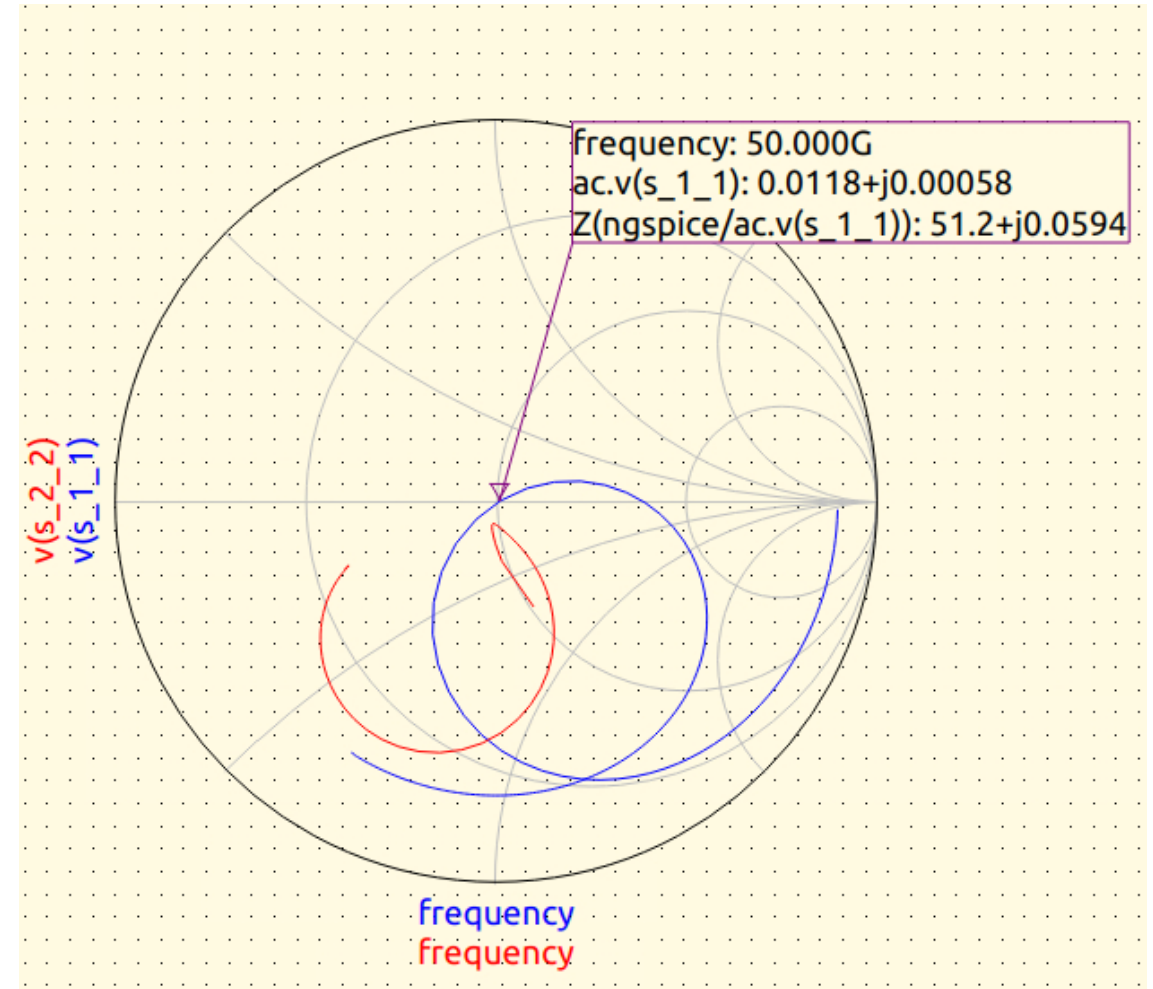
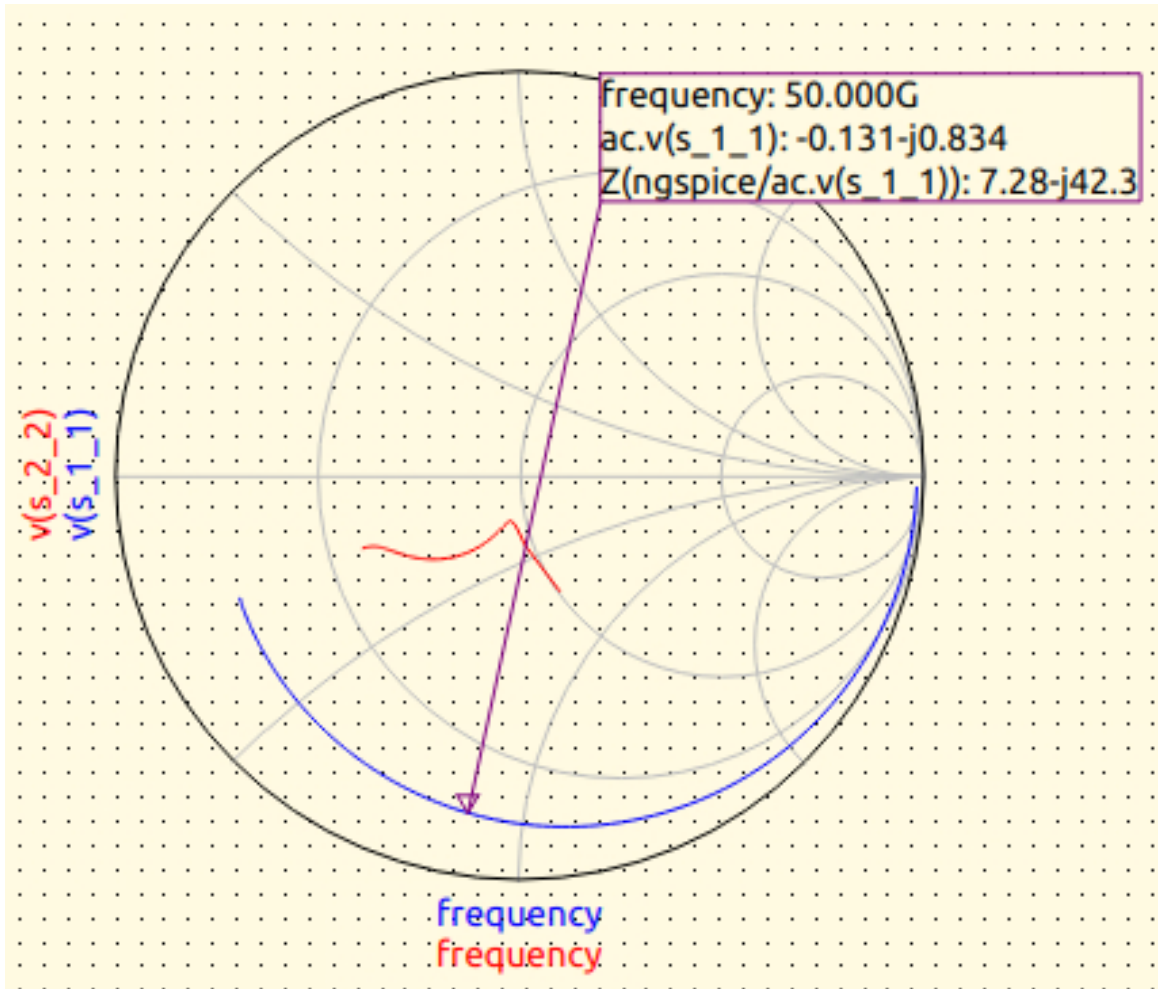
Matching



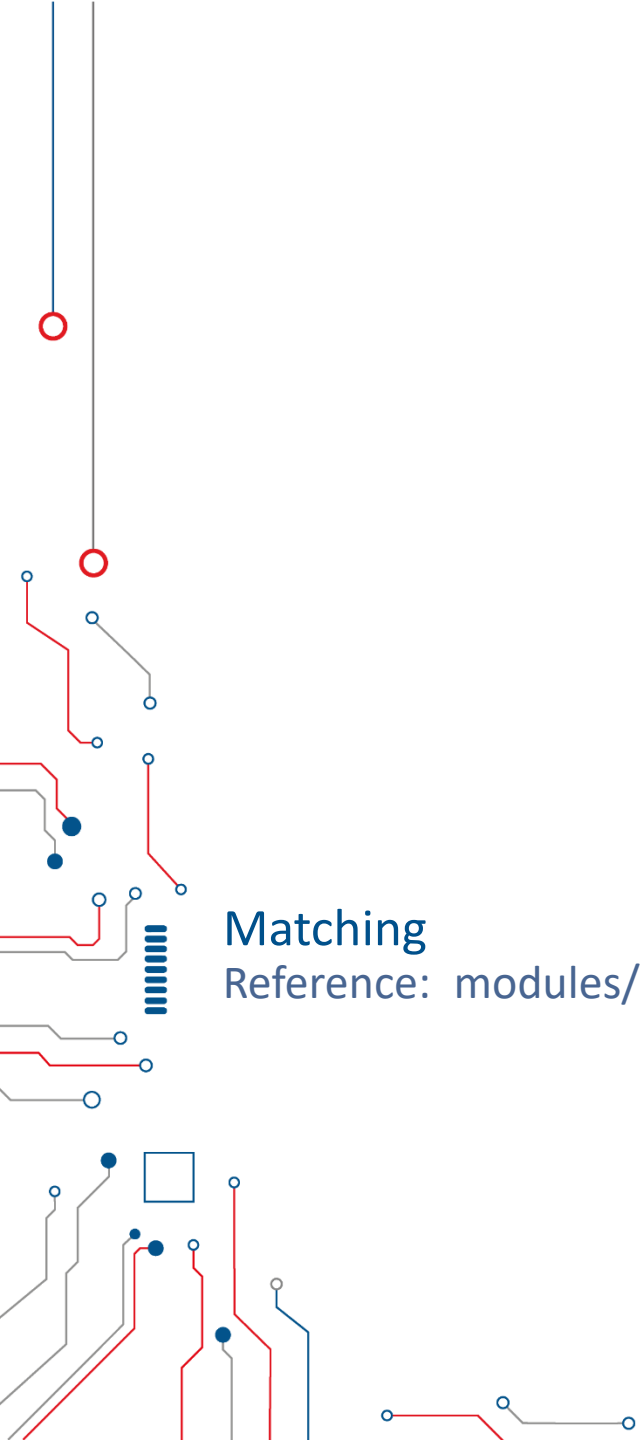
- We will start by inserting “real” capacitor and inductor in the input and (remember to set frequency of Q-factor components)
- Since we define variables for the capacitance and inductance we also need to include **.param** blocks for this
- This will be used to tune the input sizes to get best input matching
- **C** and **L** values are tuned based on parameters from the previous slide—yours may vary 😊



Matching



Part 3

A decorative circuit pattern is located on the left side of the slide. It features a vertical line of red circles at the top, followed by a series of red and blue lines that branch out and connect to various points. There are also several blue dots and a small blue square. The pattern is stylized and abstract, resembling a circuit board layout.

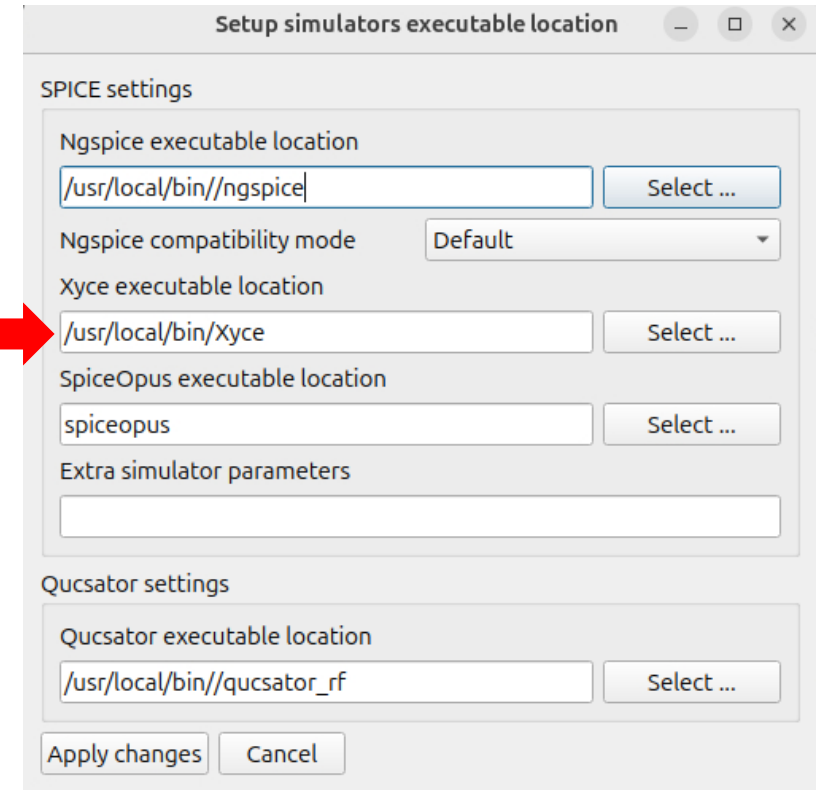
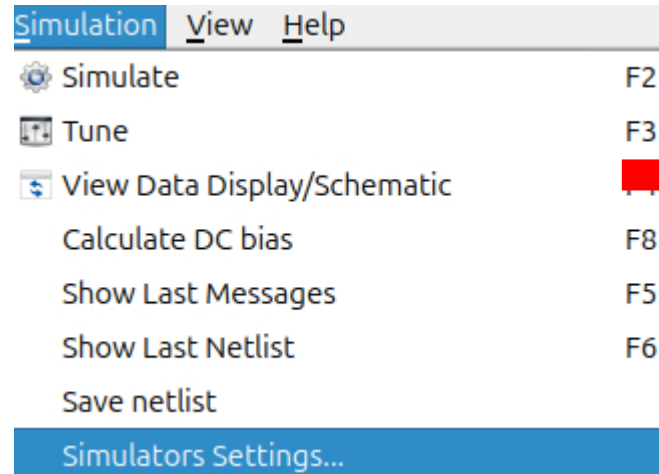
Matching

Reference: [modules/module_2_50GHz_MPA/part_3_nonlinear_analysis](#)

Xyce Setup



-0 Make sure the path to the simulator is set correctly!



Nonlinear Analysis



- 0 We switch simulator to Xyce
- 0 This imposes some issues!!
- 0 Math calculations for plotting is not possible!
- 0 Sweeping is somehow challenging
- 0 Lets look at the schematic setup on the next slide!

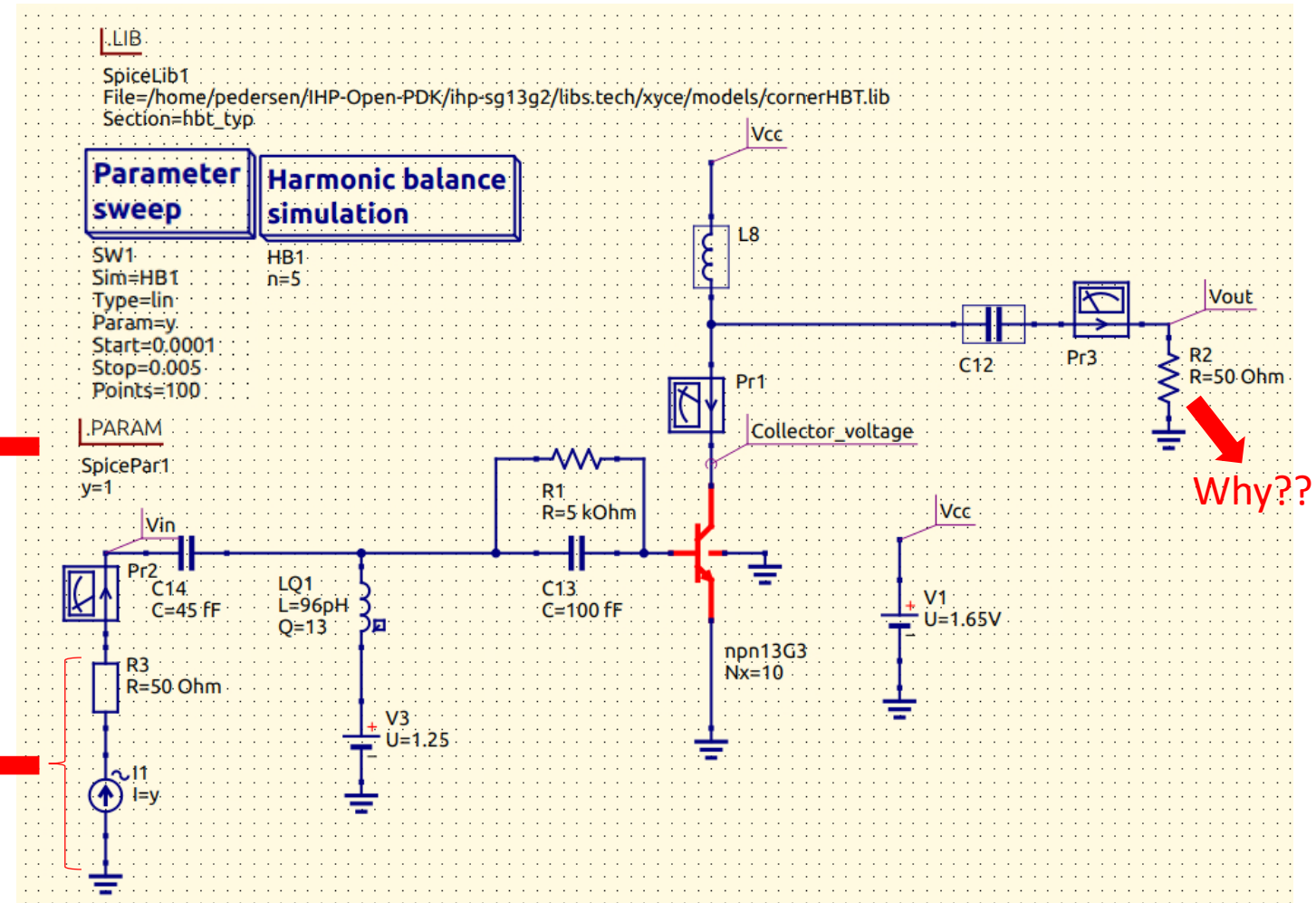
Nonlinear Analysis (Compression Point)



- 0 We note we have to include a parameter sweep which can be found in the main dock under **simulation**
- 0 Also for the model inclusion we have to go under **SPICE netlist section** in the main dock and choose the **.Lib directive**
- 0 Remember to switch the simulator to Xyce!
- 0 Run the Simulation

Why??

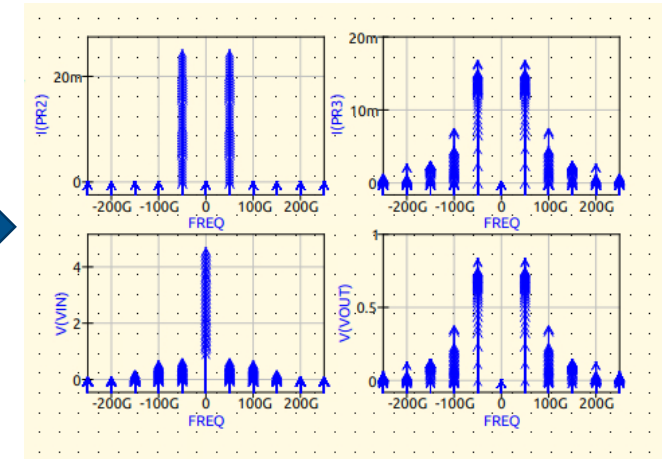
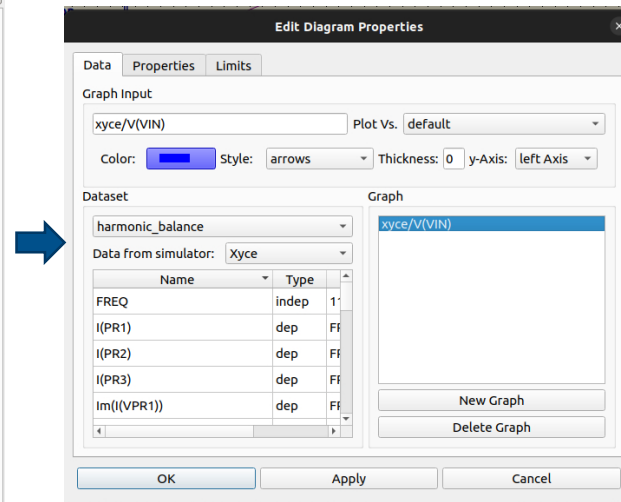
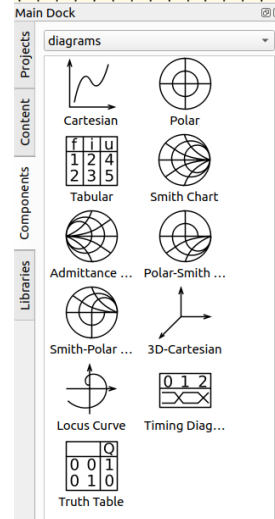
Why??



Visualizing The Results

- After setting up the simulation, we can run it and check the results. Upon completion, the first task is to **instantiate a table** to inspect the output data and ensure proper formatting.
- To extract the data for post-processing we need to plot data, which we will do with a **Cartesian** plot seen on the following slide

Y	FREQ	I(PR2)	V(VIN)
0.0001	-2.5e11	3.86e-21-j4.8e-21	-9.16e-12+j6.97e-12
	-2e11	6.01e-21+j4.45e-21	-2.54e-10-j8.01e-10
	-1.5e11	3.53e-21+j6.83e-21	2.48e-08+j6.65e-08
	-1e11	3.55e-22+j4.65e-21	-6.39e-06+j4.87e-06
	-5e10	-1.72e-20+j5e-05	2.06e-05+j0.00257
	0	-4.88e-22	0.986
	5e10	-1.72e-20+j5e-05	2.06e-05-j0.00257
	1e11	9.55e-22+j4.65e-21	-6.39e-06+j4.87e-06
	1.5e11	5.53e-21-j6.83e-21	2.48e-08-j6.65e-08
	2e11	6.01e-21-j4.45e-21	-2.54e-10+j8.01e-10
	2.5e11	3.86e-21+j4.8e-21	-9.16e-12-j6.97e-12
0.0011	-2.5e11	4.81e-20-j1.12e-20	-1.16e-06+j9.99e-07
	-2e11	5.79e-20+j1.57e-20	3.66e-06-j1.12e-05
	-1.5e11	3.62e-20+j7.82e-20	3.18e-05+j8.59e-05
	-1e11	2.65e-20-j1.44e-20	-0.000756-j0.000585
	-5e10	-1.93e-19-j0.000549	0.000302-j0.0284
	0	3.05e-21	1.51
	5e10	-1.93e-19-j0.000549	0.000302-j0.0284
	1e11	2.65e-20-j1.44e-20	-0.000756-j0.000585
	1.5e11	3.62e-20+j7.82e-20	3.18e-05-j8.59e-05
	2e11	5.79e-20-j1.57e-20	3.66e-06-j1.12e-05
	2.5e11	4.81e-20-j1.12e-20	-1.16e-06-j9.99e-07
0.0021	-2.5e11	9.24e-20-j7.63e-20	-2.39e-05+j2.22e-05
	-2e11	3.05e-20-j2.27e-20	4.95e-05-j0.000143
	-1.5e11	9.35e-20+j2.33e-19	0.000212-j0.000593
	-1e11	6.38e-20+j1.01e-20	-0.00272-j0.00217
	-5e10	-3.15e-19-j0.00105	0.0007-j0.0544



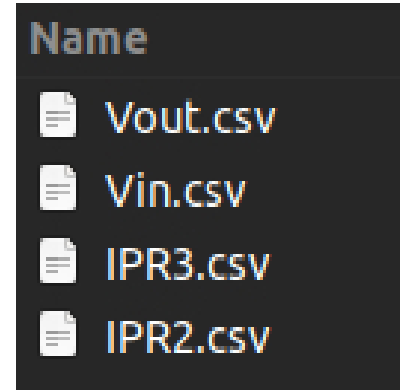
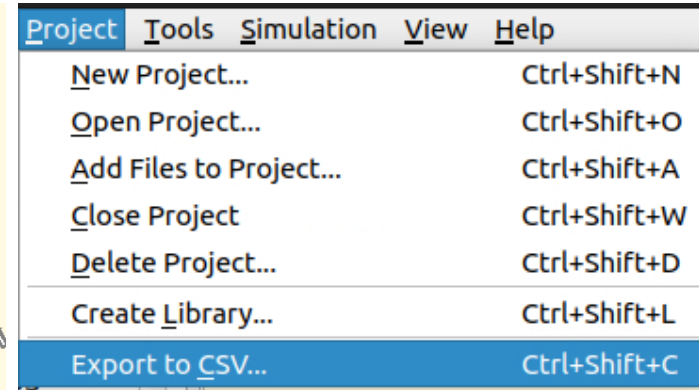
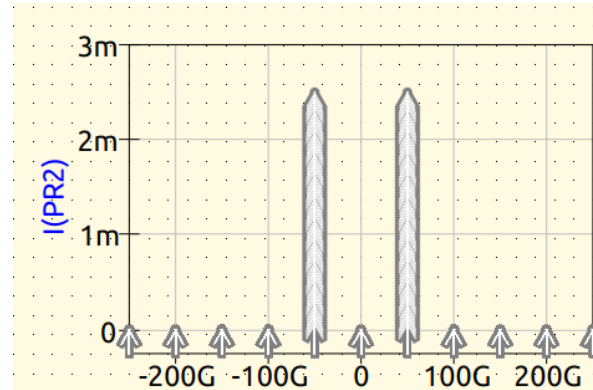
Visualizing The Results



- 0 Mark the data on the graphs and navigate to the following

- 0 Once the all the data for the parameters have been saved, the Jupyter-lab file at the following path can be opened: *module_2_50GHz_MPA/part_3_nonlinear_analysis/python/jupyter lab post_processing.ipynb*

- 0 Remember to enter `venv`



Visualizing The Results



- 0 Demonstrates how to plot extracted data using Python
- 0 Ensure parameter names match those specified for successful extraction
- 0 If schematic labels differ from the slides, open the CSV files to check and adjust the parameter names

```
[1]: # Import libraries
import pandas as pd
pd.set_option('display.float_format', lambda x: '%.22f' % x)
import numpy as np
import matplotlib.pyplot as plt
import os
import scienceplots
plt.style.use(['science', 'ieee'])
# creates figs directory
output_dir = 'figs'
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

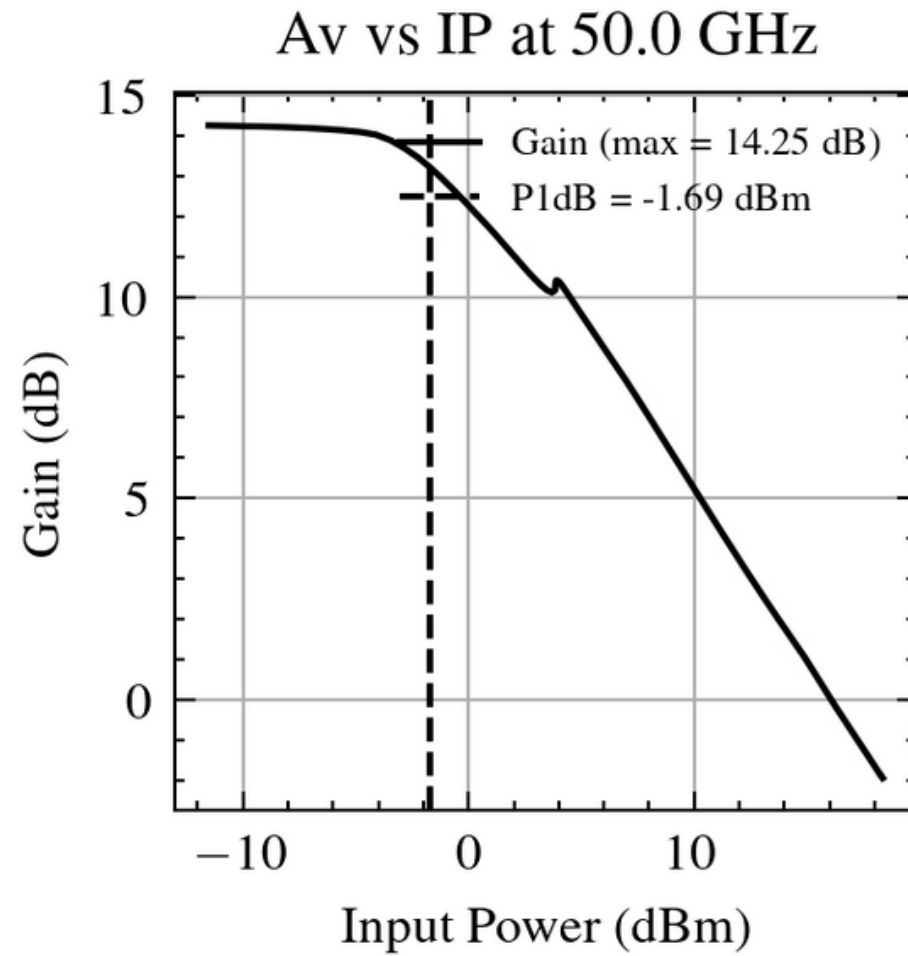
[2]: # Read CSV data (change paths accordingly)
Iin_csv = pd.read_csv('../schematic/compression_1/csv/IPR2.csv', delimiter=';', comment='#')
Iout_csv = pd.read_csv('../schematic/compression_1/csv/IPR3.csv', delimiter=';', comment='#')
Vin_csv = pd.read_csv('../schematic/compression_1/csv/Vin.csv', delimiter=';', comment='#')
Vout_csv = pd.read_csv('../schematic/compression_1/csv/Vout.csv', delimiter=';', comment='#')

# Initialize an empty DataFrame with the correct number of rows based on Iin_csv
power_df = pd.DataFrame({
    'Freq': Iin_csv['FREQ'],
    'P_in': np.zeros(len(Iin_csv)), # Preallocate with zeros
    'P_out': np.zeros(len(Iin_csv)) # Preallocate with zeros
})

# Create complex voltage and current arrays
vin_complex = []
vout_complex = []
Iout_complex = []
Iin_complex = []

# Populate the complex lists
for i in range(len(Vin_csv['r xyce/V(VIN)'))):
    vin_complex.append(complex(Vin_csv['r xyce/V(VIN)'][i], Vin_csv['i xyce/V(VIN)'][i]))
    vout_complex.append(complex(Vout_csv['r xyce/V(VOUT)'][i], Vout_csv['i xyce/V(VOUT)'][i]))
    Iout_complex.append(complex(Iout_csv['r xyce/I(PR3)'][i], Iout_csv['i xyce/I(PR3)'][i]))
    Iin_complex.append(complex(Iin_csv['r xyce/I(PR2)'][i], Iin_csv['i xyce/I(PR2)'][i]))
```

Visualizing The Results





Leibniz Institute
for high
performance
microelectronics

Expert Talk from Volker Muehlhaus



Leibniz Institute
for high
performance
microelectronics

Open Source EM Simulation for SG13G2 using OpenEMS and AWS Palace

Volker Muehlhaus

26 February 2026

Overview

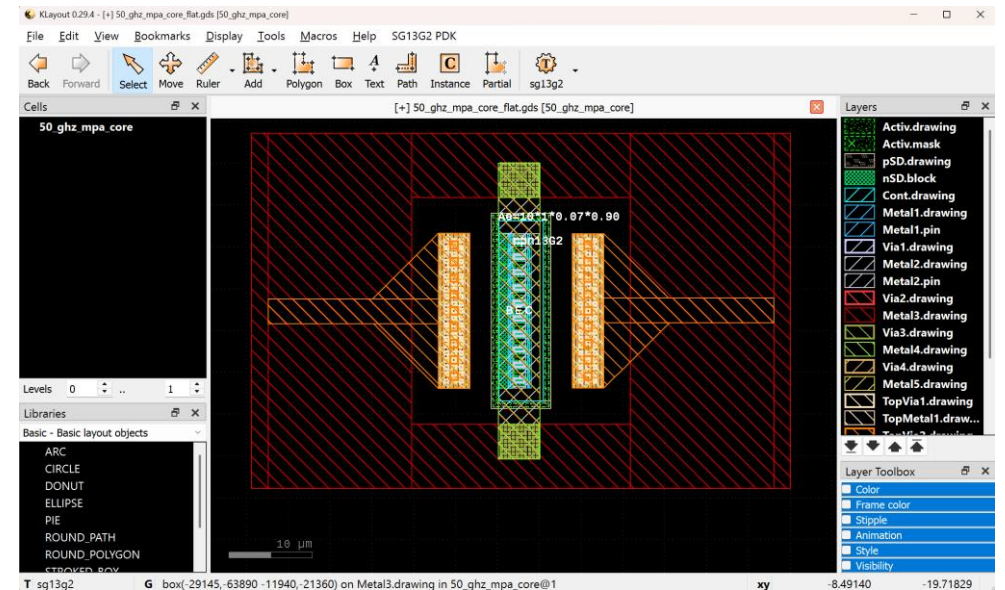
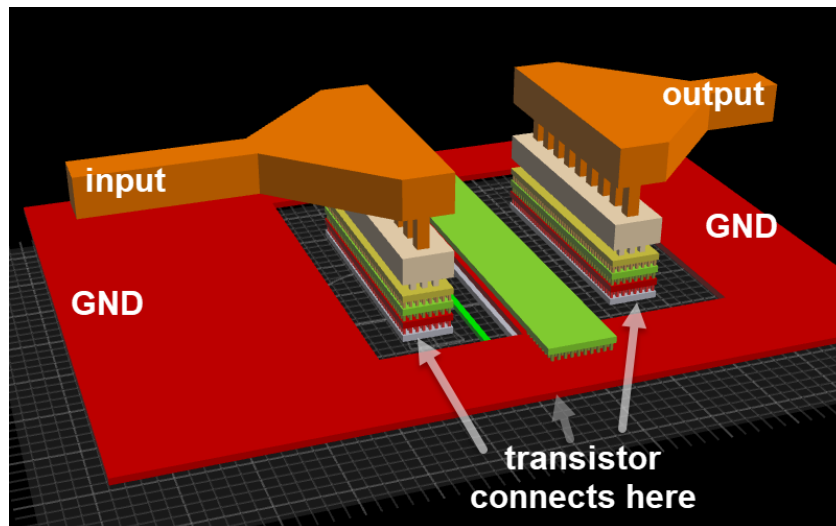


1. Why use EM simulation?
2. openEMS (FDTD) and Palace (FEM) overview
3. Python model workflow overview & example
4. MPA model setup in openEMS, port configuration, mesh and simulation time
5. MPA model in Palace (FEM simulation)
6. Graphical user interface to create & run models
7. Using S-Parameter results, optional lumped model extraction

Why use Electromagnetic (EM) simulation?



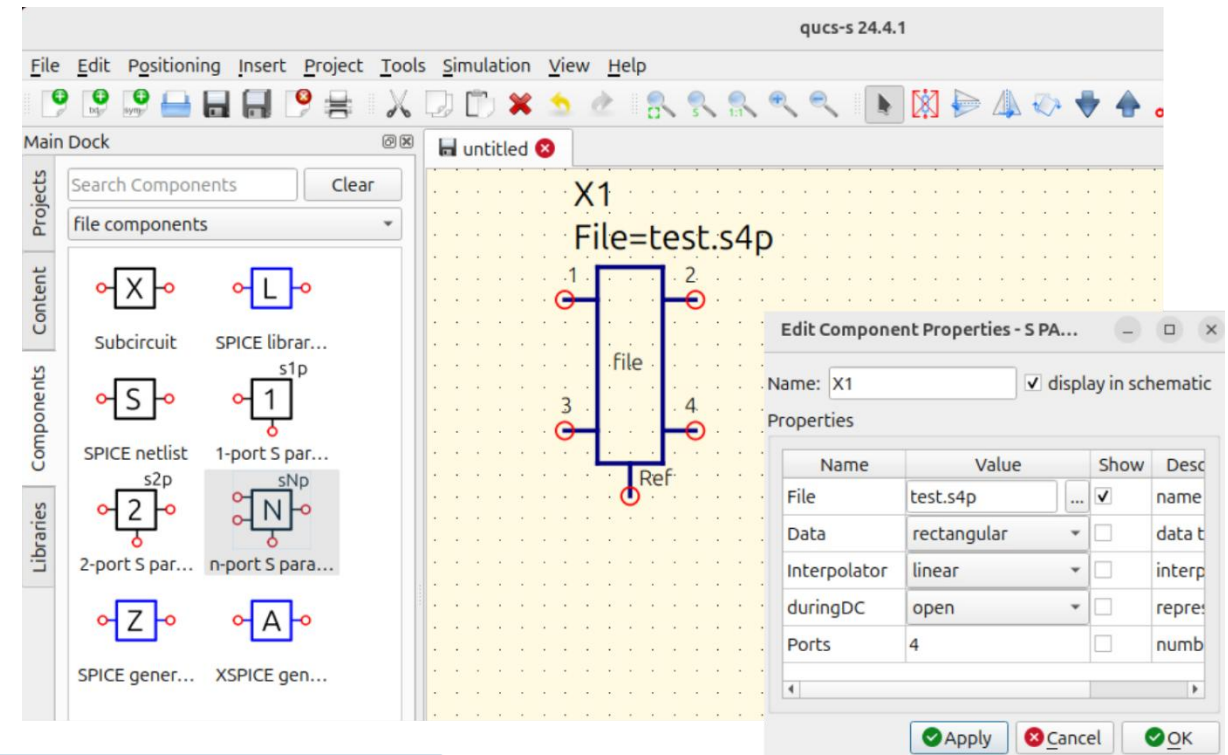
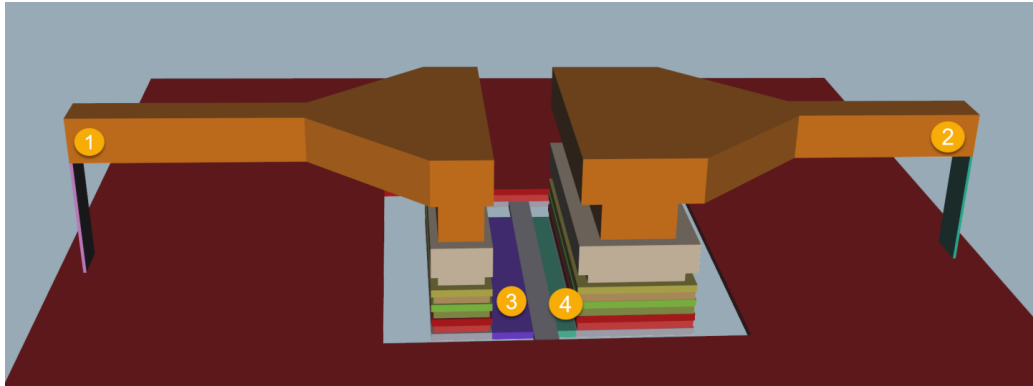
- Interconnect layout adds series R,L and shunt C
- No electrical model of layout/interconnect in the PDK
- We can use EM simulation for this part of design
- FDTD method in openEMS valid up to highest frequencies



EM simulation in the overall workflow



- Need to separate layout/routing from device models, EM model includes layout only
- Create “ports” at the interfaces where EM model connects to circuit models
- Run in EM simulator, result is black box data over frequency
- Connect EM Black box data with device models in circuit simulation

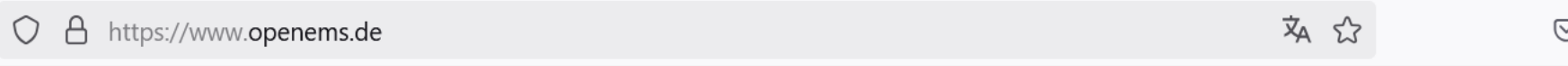


Overview



1. Why use EM simulation?
2. openEMS (FDTD) and Palace (FEM) overview
3. Python model workflow overview & example
4. MPA model setup in openEMS, port configuration, mesh and simulation time
5. MPA model in Palace (FEM simulation)
6. Graphical user interface to create & run models
7. Using S-Parameter results, optional lumped model extraction

Solver using **FDTD** method: openEMS by Thorsten Liebig



openEMS is a free and open electromagnetic field solver using the FDTD method.

- [Documentation](#)
- [Legacy Wiki](#)
- [Github Discussions](#)
- [Imprint](#)

Welcome! openEMS is a free and open electromagnetic field solver using the FDTD method. Matlab or Octave and Python are used as an easy and flexible scripting interface.

Install

[Latest Windows Build](#)

[Linux Build instructions](#)

News and Anouncements

01.01.2023: Since the forum is still down, I have activated a github discussion to ask your question about the usage of openEMS or just give some feedback. [Github Discussions](#)

07.12.2022: New temporary openEMS.de website since the [University DUE](#) hosting is currently offline!

Solver using **FEM** method: Palace by AWS Center for Quantum Computing





Search docs (Ctrl + /)

Home

- Key features
- Contents
- Coming soon

Quick Start

Installation

Running Palace

User Guide

- Overview
- Problem Types
- Simulation Models
- Boundary Conditions
- Postprocessing and Visualization
- Parallelism and GPU Support

Configuration File

- Overview

Home

[GitHub](#) [✎](#) [⚙️](#) [^](#)

Palace: 3D Finite Element Solver for Computational Electromagnetics

Palace, for **PA**rallel **LA**rge-scale Computational Electromagnetics, is an open-source, parallel finite element code for full-wave 3D electromagnetic simulations in the frequency or time domain, using the [MFEM finite element discretization library](#) and [libCEED library](#) for efficient exascale discretizations.

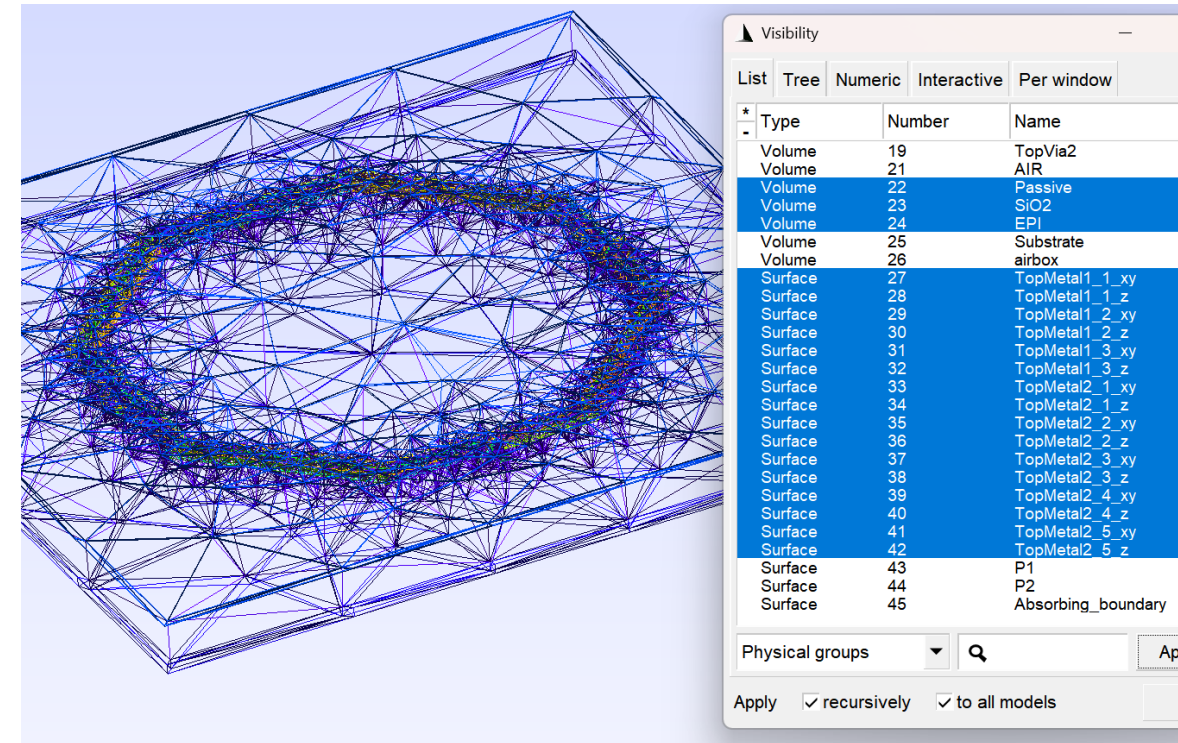
Key features

- [Eigenmode calculations](#) with optional material or radiative loss including lumped impedance boundaries. Automatic postprocessing of energy-participation ratios (EPRs) for [circuit quantization](#) and interface or bulk participation ratios for predicting dielectric loss.
- [Frequency domain driven simulations](#) with surface current excitation and lumped or numeric wave port boundaries. Wideband frequency response calculation using uniform frequency space sampling or an adaptive fast frequency sweep algorithm.
- Explicit or fully-implicit [time domain solver](#) for transient electromagnetic analysis.
- Lumped capacitance and inductance matrix extraction via [electrostatic](#) and [magnetostatic](#) problem formulations.
- Support for a wide range of [mesh file formats](#) for structured and unstructured meshes, with built-in uniform or region-based parallel [mesh refinement](#).
- Solution-based [Adaptive Mesh Refinement \(AMR\)](#) for all simulation types aside from transient. Nonconformal refinement is supported for all mesh types, and conformal refinement for simplex meshes.
- Arbitrary high-order finite element spaces and curvilinear mesh support thanks to the [MFEM library](#).
- Scalable algorithms for the [solution of linear systems of equations](#), including matrix-free p -multigrid utilizing [high-order operator partial assembly](#), parallel sparse direct solvers, and algebraic multigrid (AMG) preconditioners, for fast performance on platforms ranging from laptops to HPC systems.
- Support for [hardware acceleration using NVIDIA or AMD GPUs](#), including multi-GPU parallelism, using pure CUDA and HIP code as well as [MAGMA](#) and other libraries.

FEM Method used in AWS Palace



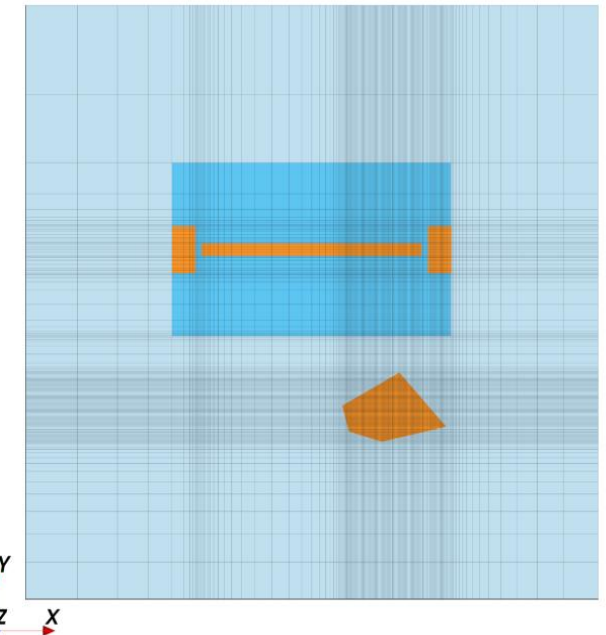
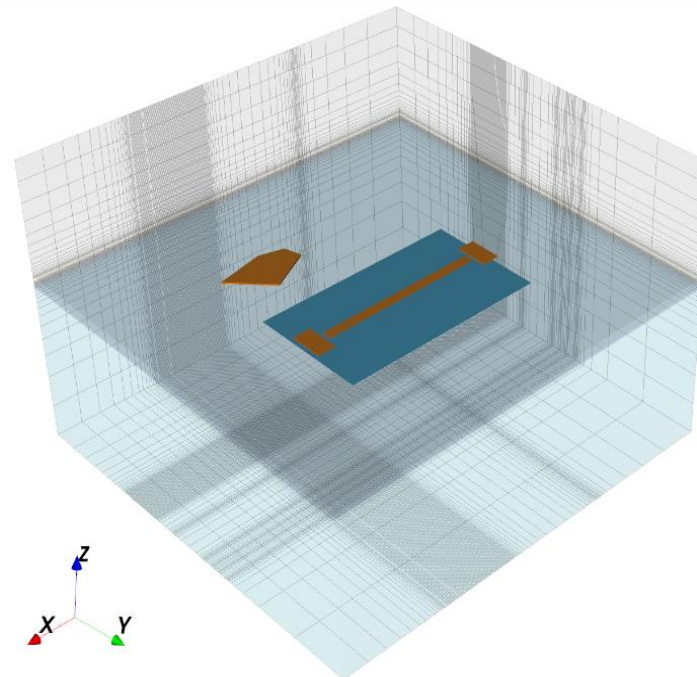
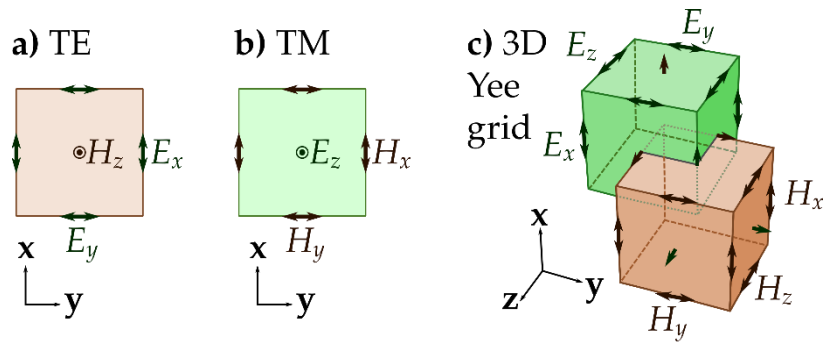
- Simulation method is finite elements in frequency domain
- Solves one frequency after another, adaptive frequency sweep for interpolation
- Simulation volume is meshed into tetrahedrons
- Local mesh refinement, small mesh cells do **not** propagate across the entire simulation model
- Optional adaptive mesh refinement during simulation, based on resulting field distribution
- Disadvantage: for N mesh cells, RAM use grows $O(N^{1.5})$



FDTD Method used in openEMS: Mesh



- Simulation method is **F**inite **D**ifference in **T**ime **D**omain
- Simulation domain is divided into many small boxes
- Calculates all E, H value for each box at one timestep, then next timestep, next timestep...
- Each mesh line cuts through entire simulation domain, no local mesh refinement
- For N mesh cells, RAM use grows $O(N)$

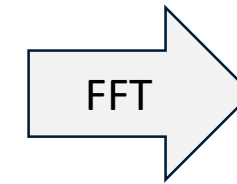
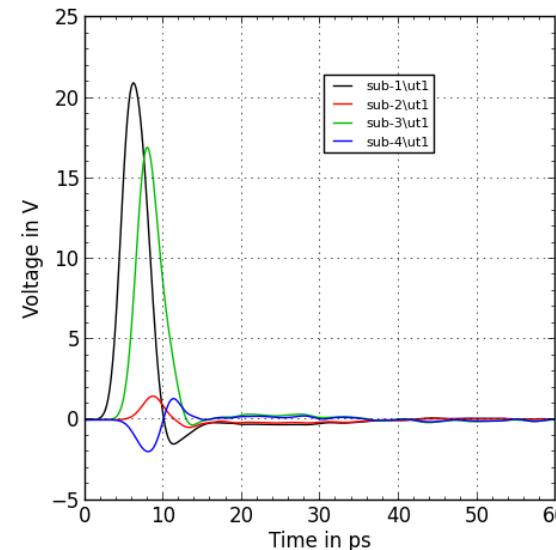
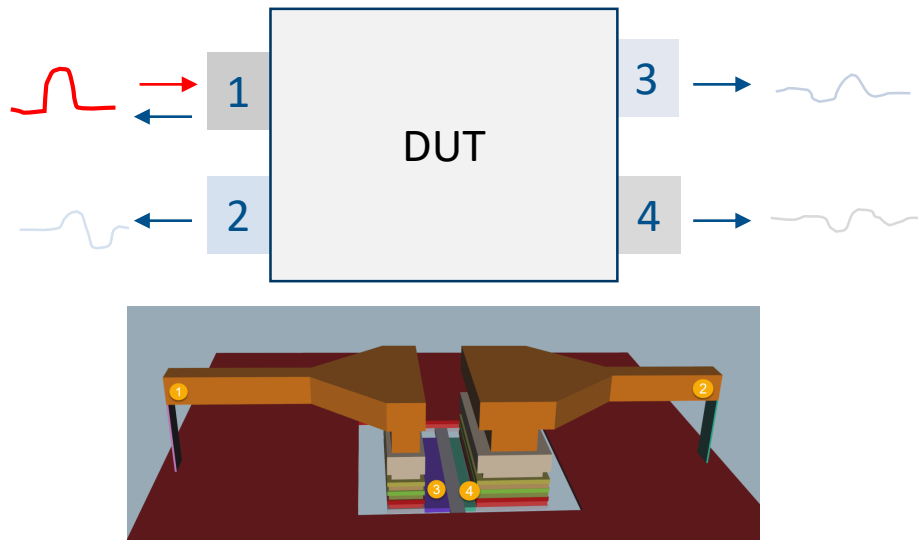


Von FDominec - Eigenes Werk, CC BY-SA 4.0,
<https://commons.wikimedia.org/w/index.php?curid=37637085>

FDTD Method used in openEMS: Time Domain



- Excite one port with gaussian pulse => wideband continuous spectrum
- FDTD calculates E and H in time domain, step by step, until energy in model is (near) zero
- Wideband S-Parameter obtained by Fourier transform of time signals at port(s)
- We get one column of S-matrix (wideband) per port excitation
- Repeat for all ports to get full [S] matrix



$$\begin{bmatrix} S_{11} & S_{12} & S_{13} & S_{14} \\ S_{21} & S_{22} & S_{21} & S_{24} \\ S_{31} & S_{32} & S_{33} & S_{24} \\ S_{41} & S_{42} & S_{43} & S_{44} \end{bmatrix}$$

FDTD (openEMS) vs. FEM (Palace)



- Very similar IHP workflows, based on Python model script
- Model scripts reads GDSII layout and XML stackup file, result is *.snp Touchstone data

openEMS

- Excitation of **one port** after another, wideband sweep results per port excitation
- Simulation time increases linear with port count
- Slows down when very small geometry detail requires small mesh cell (small time step!)
- Required memory scales $O(N)$, needs less RAM for large models than FEM method
- openEMS uses up to ~ 4 CPU cores efficiently
- openEMS runs native on Windows and Linux (Linux version is faster)

Palace

- FEM simulates **one frequency** after another (multiple ports at low extra cost)
- Adaptive frequency sweep (interpolation)
- Small geometry detail does not cost much
- Required memory scales $O(N^{1.5})$, needs more RAM for large models than FDTD
- FEM with Palace scales well up to hundreds of CPU cores, uses *MPI* for load sharing, designed for high performance computing
- Linux only (does not run on Windows)

Overview



1. Why use EM simulation?
2. openEMS (FDTD) and Palace (FEM) overview
3. Python model workflow overview & example
4. MPA model setup in openEMS, port configuration, mesh and simulation time
5. MPA model in Palace (FEM simulation)
6. Graphical user interface to create & run models
7. Using S-Parameter results, optional lumped model extraction

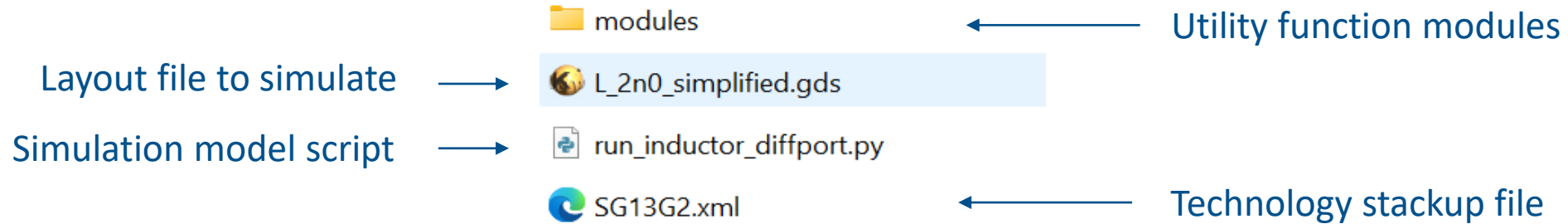
IHP workflow for FDTD using openEMS



- <https://github.com/VolkerMuehlhaus/openems> ihp_sg13g2
- Add **additional software on top of openEMS** that specifically targets RFIC EM
- Modular approach, keep the main model simple & clean, easy to re-use
- Read geometries directly from GDSII layout files
- Read all technology stackup & material data from XML file
- Geometry pre-processing options, e.g. via array merging
- Automatic meshing based on GDSII data, requires only one value set by user

- Examples for different use cases in repository, including output processing & plot
- For new model, it is easiest to start from existing examples and change only a few code lines

OpenEMS workflow minimum example

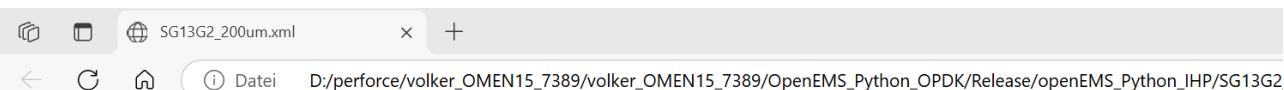


New since Dec 2025:

Instead of a local copy of „modules“ folder, you can also install module gds2openEMS (pip install gds2openEMS) and include that module in your model code, see this example:

https://github.com/VolkerMuehlhaus/openems_ihp_sg13g2/tree/main/more_examples/model_syntax_gds2openEMS

Geometry input file, stackup input file



This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<Stackup schemaVersion="2.0">
  <Materials>
    <Material Name="Activ" Type="Conductor" Permittivity="1" DielectricLossTangent="0" Conductivity="357141.0" Color="00ff00"/>
    <Material Name="Metal1" Type="Conductor" Permittivity="1" DielectricLossTangent="0" Conductivity="21640000.0" Color="39bfff"/>
    <Material Name="Metal2" Type="Conductor" Permittivity="1" DielectricLossTangent="0" Conductivity="23190000.0" Color="ccccd9"/>
    <Material Name="Metal3" Type="Conductor" Permittivity="1" DielectricLossTangent="0" Conductivity="23190000.0" Color="d80000"/>
    <Material Name="Metal4" Type="Conductor" Permittivity="1" DielectricLossTangent="0" Conductivity="23190000.0" Color="93e837"/>
    <Material Name="Metal5" Type="Conductor" Permittivity="1" DielectricLossTangent="0" Conductivity="23190000.0" Color="dcd146"/>
    <Material Name="TopMetal1" Type="Conductor" Permittivity="1" DielectricLossTangent="0" Conductivity="27800000.0" Color="ffe6bf"/>
    <Material Name="TopMetal2" Type="Conductor" Permittivity="1" DielectricLossTangent="0" Conductivity="30300000.0" Color="ff8000"/>
    <Material Name="TopVia2" Type="Conductor" Permittivity="1" DielectricLossTangent="0" Conductivity="3143000.0" Color="ff8000"/>
    <Material Name="TopVia1" Type="Conductor" Permittivity="1" DielectricLossTangent="0" Conductivity="2191000.0" Color="ffe6bf"/>
    <Material Name="Via4" Type="Conductor" Permittivity="1" DielectricLossTangent="0" Conductivity="1660000.0" Color="deac5e"/>
    <Material Name="Via3" Type="Conductor" Permittivity="1" DielectricLossTangent="0" Conductivity="1660000.0" Color="9ba940"/>
    <Material Name="Via2" Type="Conductor" Permittivity="1" DielectricLossTangent="0" Conductivity="1660000.0" Color="ff3736"/>
    <Material Name="Via1" Type="Conductor" Permittivity="1" DielectricLossTangent="0" Conductivity="1660000.0" Color="ccccff"/>
    <Material Name="Cont" Type="Conductor" Permittivity="1" DielectricLossTangent="0" Conductivity="2390000.0" Color="00ffff"/>
    <Material Name="Passive" Type="Dielectric" Permittivity="6.6" DielectricLossTangent="0.0" Conductivity="0" Color="a0a0f0"/>
    <Material Name="SiO2" Type="Dielectric" Permittivity="4.1" DielectricLossTangent="0.0" Conductivity="0" Color="fffcad"/>
    <Material Name="Substrate" Type="Semiconductor" Permittivity="11.9" DielectricLossTangent="0" Conductivity="2.0" Color="01e0ff"/>
    <Material Name="EPI" Type="Semiconductor" Permittivity="11.9" DielectricLossTangent="0" Conductivity="5.0" Color="294fff"/>
    <Material Name="AIR" Type="Dielectric" Permittivity="1.0" DielectricLossTangent="0.0" Conductivity="0" Color="d0d0d0"/>
    <Material Name="LOWLOSS" Type="Conductor" Permittivity="1" DielectricLossTangent="0" Conductivity="1E10" Color="ff0000"/>
    <Material Name="MIM_equiv" Type="Dielectric" Permittivity="16.87" DielectricLossTangent="0.0" Conductivity="0" Color="ff0000"/>
  </Materials>
  <ELayers LengthUnit="um">
    <Dielectrics>
      <Dielectric Name="AIR" Material="AIR" Thickness="300.0000"/>
      <Dielectric Name="Passive" Material="Passive" Thickness="0.4000"/>
      <Dielectric Name="SiO2" Material="SiO2" Thickness="15.7303"/>
      <Dielectric Name="EPI" Material="EPI" Thickness="3.7500"/>
      <Dielectric Name="Substrate" Material="Substrate" Thickness="180.0000"/>
    </Dielectrics>
    <Layers>
      <Substrate Offset="183.75"/>
      <Layer Name="Activ" Type="conductor" Zmin="0.0000" Zmax="0.4000" Material="Activ" Layer="1"/>
      <Layer Name="Metal1" Type="conductor" Zmin="1.0400" Zmax="1.4600" Material="Metal1" Layer="8"/>
      <Layer Name="Metal2" Type="conductor" Zmin="2.0000" Zmax="2.4900" Material="Metal2" Layer="10"/>
      <Layer Name="Metal3" Type="conductor" Zmin="3.0300" Zmax="3.5200" Material="Metal3" Layer="30"/>
      <Layer Name="Metal4" Type="conductor" Zmin="4.0600" Zmax="4.5500" Material="Metal4" Layer="50"/>
      <Layer Name="Metal5" Type="conductor" Zmin="5.0900" Zmax="5.5800" Material="Metal5" Layer="67"/>
      <Layer Name="TopMetal1" Type="conductor" Zmin="6.4303" Zmax="8.4303" Material="TopMetal1" Layer="126"/>
      <Layer Name="TopMetal2" Type="conductor" Zmin="11.2303" Zmax="14.2303" Material="TopMetal2" Layer="134"/>
      <Layer Name="TopVia2" Type="via" Zmin="8.4303" Zmax="11.2303" Material="TopVia2" Layer="133"/>
    </Layers>
  </ELayers>
</Stackup>
```

```
# ===== input files and path settings =====
```

```
gds_filename = "L_2n0_simplified.gds" # geometries
XML_filename = "SG13G2.xml"           # stackup
```

```
# preprocess GDSII for safe handling of cutouts/holes?
preprocess_gds = False
```

```
# get path for this simulation file
script_path = utilities.get_script_path(__file__)
```

```
# use script filename as model basename
model_basename = utilities.get_basename(__file__)
```

```
# set and create directory for simulation output
sim_path = utilities.create_sim_path (script_path,model_basename)
print('Simulation data directory: ', sim_path)
```

Simulation control

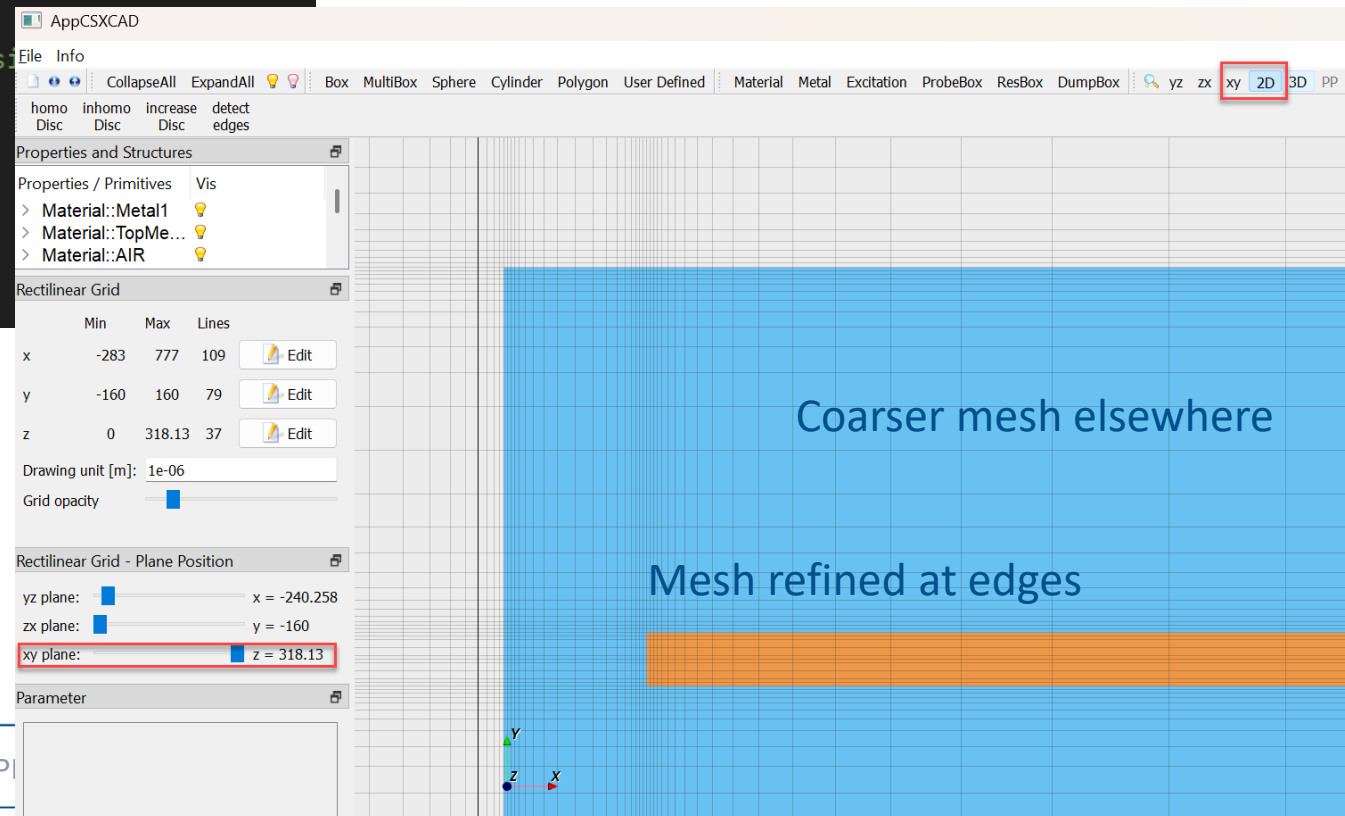


- Mesh is built automatically based on geometry
- Only one parameter to set required minimum mesh size (required detail resolution)

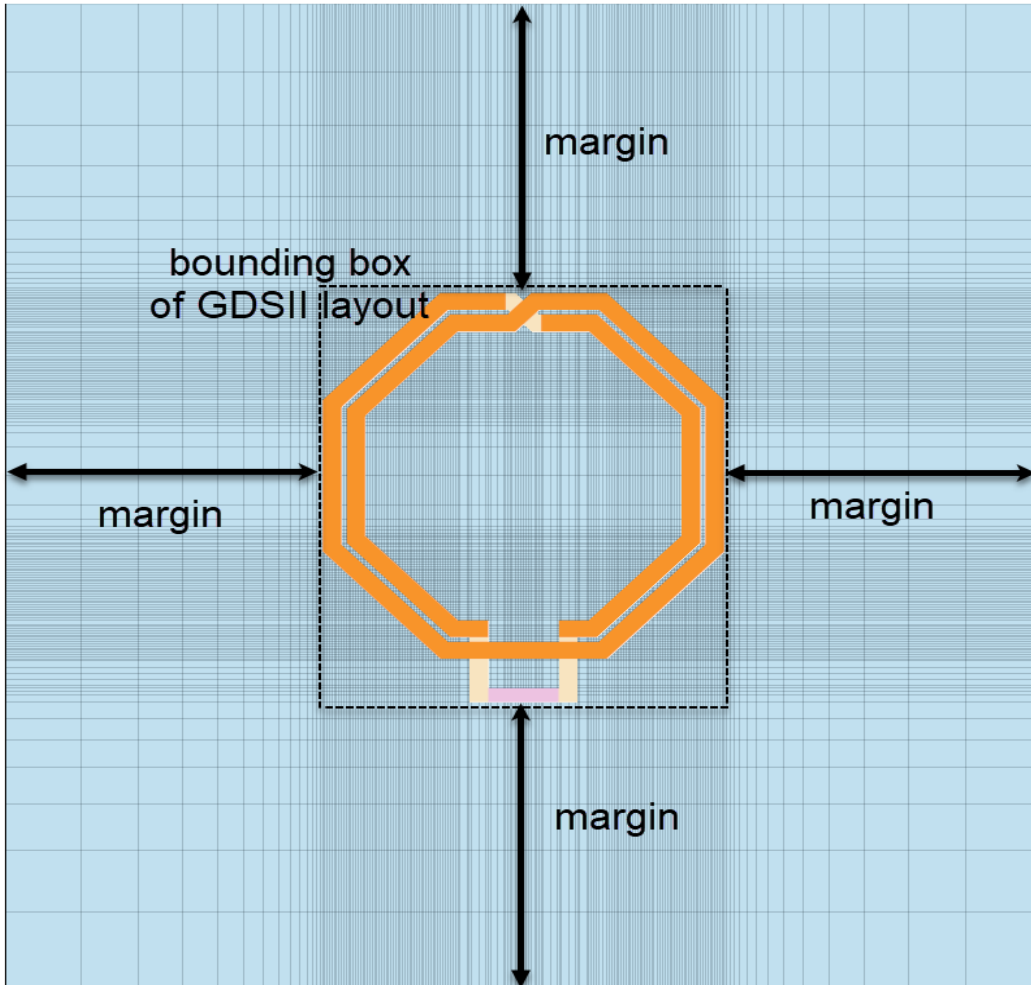
```
unit    = 1e-6 # geometry is in microns
margin  = 50    # distance in microns from GDSII geometry boundary to s

fstart  = 0e9
fstop   = 110e9
numfreq = 401

refined_cellsize = 1 # mesh cell size in conductor region
```



Margins to simulation boundary

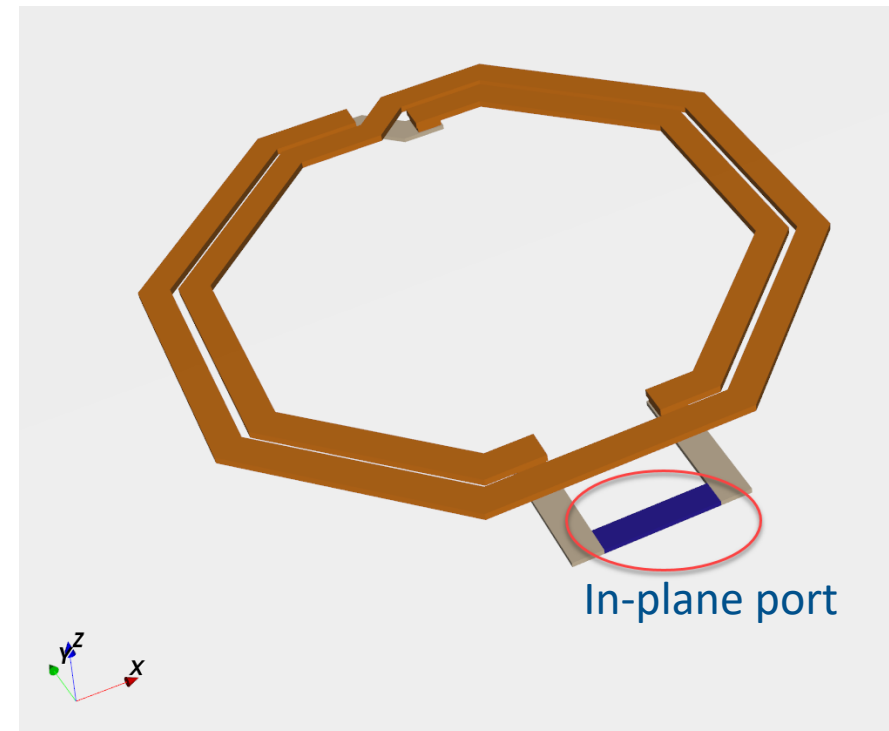
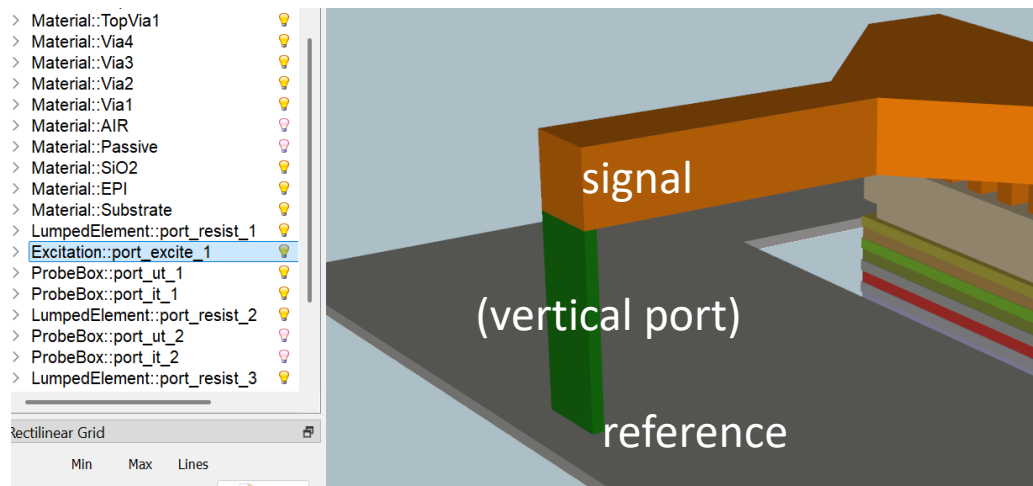


```
# ===== simulation settings =====  
  
unit    = 1e-6    # geometry is in microns  
margin  = 200     # distance in microns from GDSII geometry boundary to simulation boundary  
  
fstart  = 0  
fstop   = 30e9  
numfreq = 401  
  
refined_cellsize = 1.0 # mesh cell size in conductor region  
  
# choices for boundary:  
# 'PEC' : perfect electric conductor (default)  
# 'PMC' : perfect magnetic conductor, useful for symmetries  
# 'MUR' : simple MUR absorbing boundary conditions  
# 'PML_8' : PML absorbing boundary conditions  
Boundaries = ['PEC', 'PEC', 'PEC', 'PEC', 'PEC', 'PEC']  
  
cells_per_wavelength = 20 # how many mesh cells per wavelength, must be 10 or more  
energy_limit = -50        # end criteria for residual energy (dB)
```

Ports: in-plane or vertical between layers



- In-plane ports between metals on same layer
- Vertical ports between metals of different layers



-

58

Overview

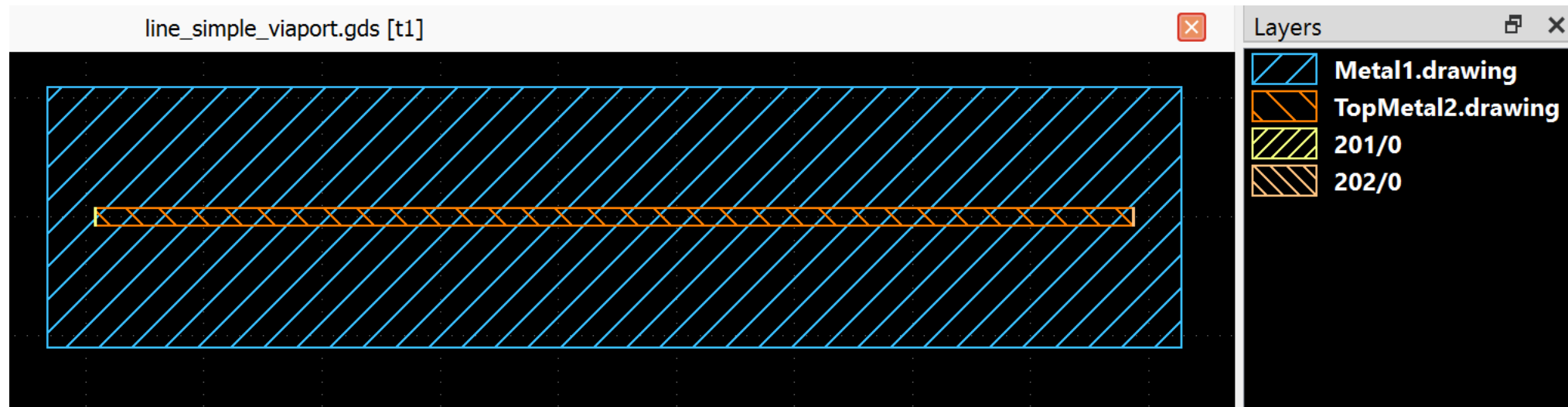


1. Why use EM simulation?
2. openEMS and FDTD method basics
3. What is different between standard openEMS and new Python workflow?
4. Typical simulation run
5. Simulated vs. measured
6. Summary

Simple model: Transmission line



- Geometry: TopMetal2 transmission line with Metal1 ground plane
- Metal1 ground plane must be drawn to include it in model
- Input port 1 drawn as small box on layer 201
- Output port 2 drawn as small box on layer 202



```
unit = 1e-6 # geometry is in microns
margin = 50 # distance in microns from GDSII geometry boundary to simulation boundary

fstart = 0e9
fstop = 110e9
numfreq = 401

refined_cellsize = 1 # mesh cell size in conductor region

# choices for boundary:
# 'PEC' : perfect electric conductor (default)
# 'PMC' : perfect magnetic conductor, useful for symmetries
# 'MUR' : simple MUR absorbing boundary conditions
# 'PML_8' : PML absorbing boundary conditions
Boundaries = ['PEC', 'PEC', 'PEC', 'PEC', 'PEC', 'PEC']

cells_per_wavelength = 20 # how many mesh cells per wavelength, must be 10 or more
energy_limit = -40 # end criteria for residual energy (dB)
```

16.02

Run model

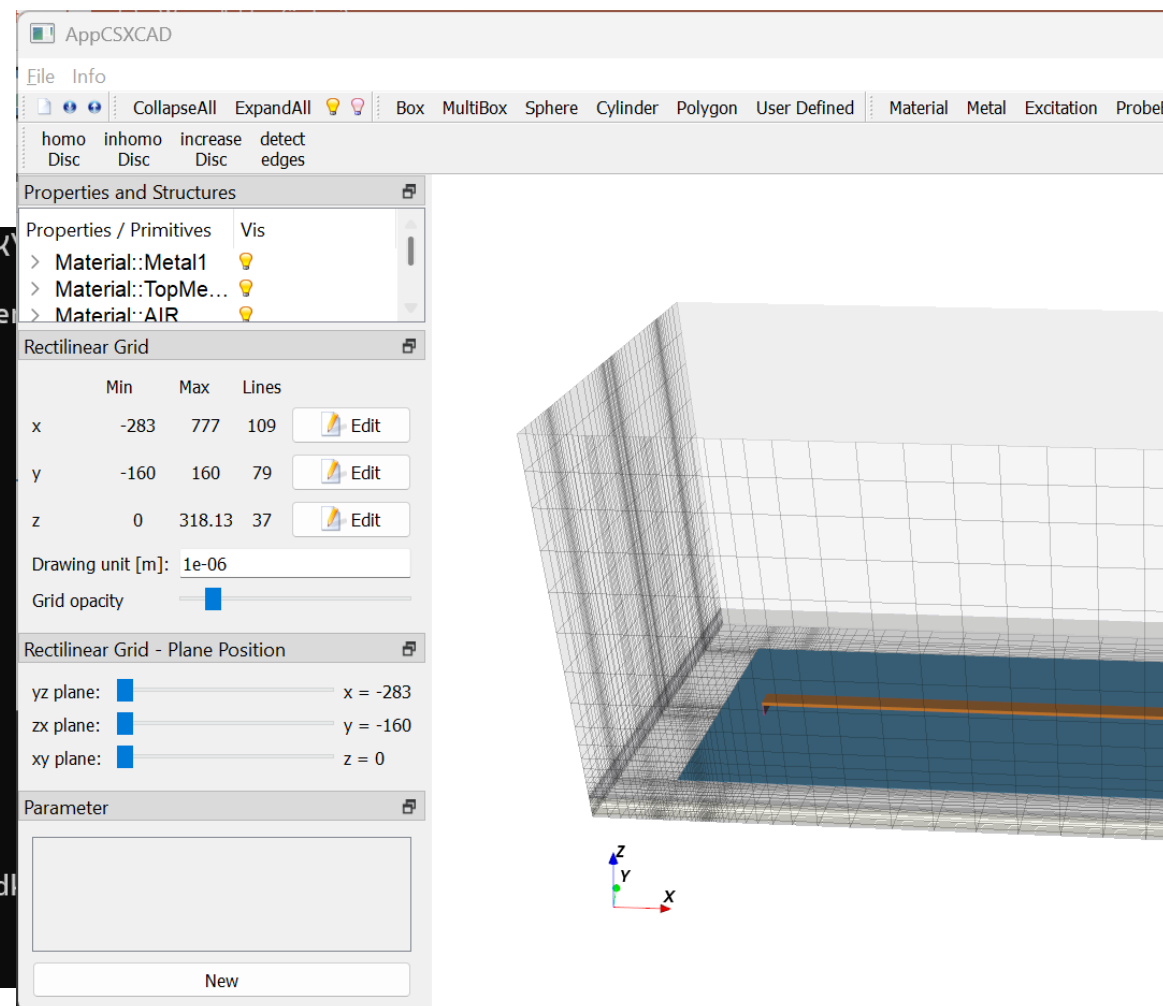


- Change to model directory
- From command line: `python3 run_line_viaport.py`

```
D:\perforce\volker_OMEN15_7389\volker_OMEN15_7389\OpenEMS_Python_OPDK
t.py
Simulation data directory: d:\perforce\volker_omen15_7389\volker_omen15_7389\openems_python_opdk
_ihp\output\run_line_viaport_data
Reading XML stackup file: SG13G2_nosub.xml
Reading GDSII input file: line_simple_viaport.gds

-----
Mesh cells by axis (total 327 kcells):
x = 109
y = 79
z = 38
Smallest cell size:
dx = 1.0000
dy = 1.0000
dz = 0.4000
-----

Starting AppCSXCAD 3D viewer with file:
d:\perforce\volker_omen15_7389\volker_omen15_7389\openems_python_opdk
_data\sub-1\run_line_viaport.xml
Close AppCSXCAD to continue or press <Ctrl>-C to abort
```



FDTD Simulation (finished)



- Simulation of this port excitation is finished when residual energy is below limit defined in code

```
C:\Windows\System32\cmd.e  X  +  v

Create FDTD operator (compressed SSE + multi-threading)
FDTD simulation size: 109x79x37 --> 318607 FDTD cells
FDTD timestep is: 1.70744e-15 s; Nyquist rate: 2662 timesteps @1.10006e+11 Hz
openEMS::SetupFDTD: Warning, the timestep seems to be very small --> long simulation
Excitation signal length is: 30506 timesteps (5.20873e-11s)
Max. number of timesteps: 1000000000 ( --> 32780.4 * Excitation signal length)
Create FDTD engine (compressed SSE + multi-threading)
Running FDTD engine... this may take a while... grab a cup of coffee!?!
[ @ 4s] Timestep: 665 || Speed: 51.0 MC/s (6.243e-03 s/TS) || Energy: ~2.25e-18 (-0.00dB)
[ @ 10s] Timestep: 1995 || Speed: 62.3 MC/s (5.111e-03 s/TS) || Energy: ~4.62e-19 (-6.84dB)
[ @ 15s] Timestep: 3325 || Speed: 92.0 MC/s (3.461e-03 s/TS) || Energy: ~2.69e-19 (-9.19dB)
[ @ 20s] Timestep: 5320 || Speed: 120.1 MC/s (2.653e-03 s/TS) || Energy: ~4.15e-21 (-27.30dB)
[ @ 25s] Timestep: 7315 || Speed: 139.5 MC/s (2.285e-03 s/TS) || Energy: ~1.04e-21 (-33.33dB)
[ @ 30s] Timestep: 9975 || Speed: 161.0 MC/s (1.979e-03 s/TS) || Energy: ~3.70e-22 (-37.81dB)
[ @ 35s] Timestep: 12635 || Speed: 177.5 MC/s (1.795e-03 s/TS) || Energy: ~2.73e-22 (-39.12dB)
[ @ 39s] Timestep: 15253 || Speed: 187.7 MC/s (1.697e-03 s/TS) || Energy: ~1.94e-22 (-40.61dB)
[ @ 44s] Timestep: 17955 || Speed: 190.9 MC/s (1.669e-03 s/TS) || Energy: ~1.94e-22 (-40.61dB)
[ @ 48s] Timestep: 20615 || Speed: 202.4 MC/s (1.574e-03 s/TS) || Energy: ~1.94e-22 (-40.61dB)
[ @ 52s] Timestep: 23275 || Speed: 204.6 MC/s (1.557e-03 s/TS) || Energy: ~1.94e-22 (-40.61dB)
[ @ 57s] Timestep: 25935 || Speed: 169.4 MC/s (1.880e-03 s/TS) || Energy: ~1.94e-22 (-40.61dB)
Multithreaded Engine: Best performance found using 11 threads.
[ @ 1m01s] Timestep: 28595 || Speed: 198.4 MC/s (1.606e-03 s/TS) || Energy: ~1.94e-22 (-40.61dB)
[ @ 1m06s] Timestep: 31255 || Speed: 199.8 MC/s (1.595e-03 s/TS) || Energy: ~1.94e-22 (-40.61dB)
[ @ 1m10s] Timestep: 33915 || Speed: 208.5 MC/s (1.528e-03 s/TS) || Energy: ~1.94e-22 (-40.61dB)
[ @ 1m14s] Timestep: 37240 || Speed: 232.0 MC/s (1.373e-03 s/TS) || Energy: ~1.94e-22 (-40.61dB)
Time for 37240 iterations with 318607.00 cells : 74.87 sec
Speed: 158.46 MCells/s
FDTD simulation completed successfully for excitation [1]
Creating S-parameter file

fstart = 0e9
fstop = 110e9
numfreq = 401

refined_cellsize = 1 # mesh cell size in conductor region

# choices for boundary:
# 'PEC' : perfect electric conductor (default)
# 'PMC' : perfect magnetic conductor, useful for symmetries
# 'MUR' : simple MUR absorbing boundary conditions
# 'PML_8' : PML absorbing boundary conditions
Boundaries = ['PEC', 'PEC', 'PEC', 'PEC', 'PEC', 'PEC']

cells_per_wavelength = 20 # how many mesh cells per wavelength, must be 10 or more
energy_limit = -40 # end criteria for residual energy (dB)
```

Data plot, create data file

```
sub1_data_path = simulation_setup.runSimulation (excite_ports, FDTD, sim_path, model_basename)

##### evaluation of results with composite GSG ports #####

if preview_only==False:

    # define dB function for S-parameters
    def dB(value):
        return 20.0*np.log10(np.abs(value))

    # define phase function for S-parameters
    def phase(value):
        return np.angle(value, deg=True)

    f = np.linspace(fstart,fstop,numfreq)

    # get results, CSX port definition is read from simulation ports object
    s11 = utilities.calculate_Sij (1, 1, f, sim_path, simulation_ports)
    s21 = utilities.calculate_Sij (2, 1, f, sim_path, simulation_ports)

    # S12, S22 is NOT available because we have NOT simulated port2 excitation
    # fake it by assuming symmetry
    s22 = s11
    s12 = s21

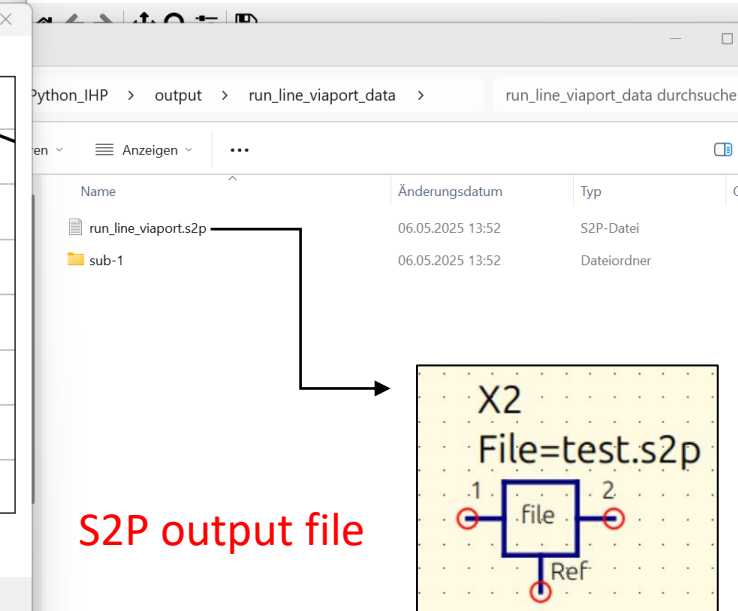
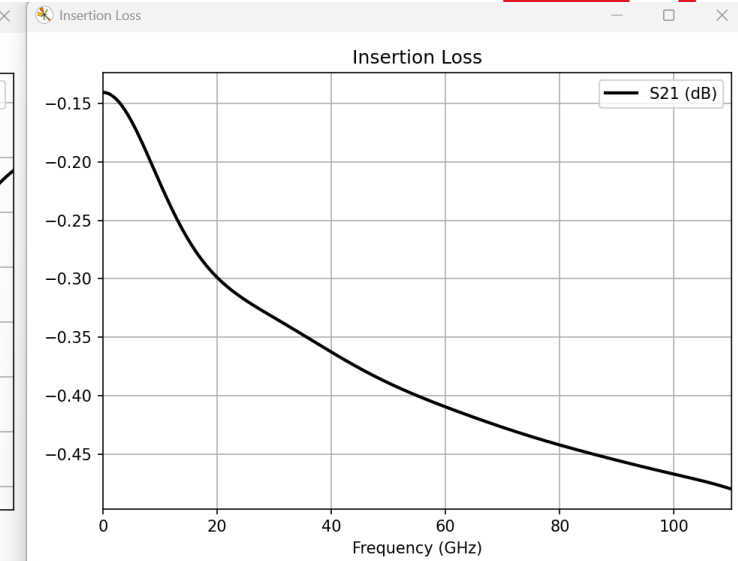
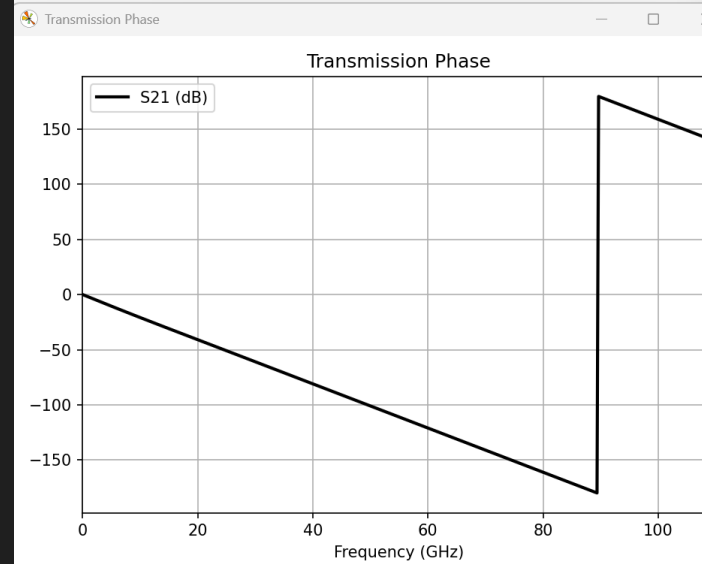
    # write Touchstone S2P file
    s2p_name = os.path.join(sim_path, model_basename + '.s2p')
    utilities.write_snp (np.array([[s11, s21],[s12,s22]]),f, s2p_name)

    fig, axis = plt.subplots(num='Return Loss', tight_layout=True)
    axis.plot(f/1e9, dB(s11), 'k-', linewidth=2, label='S11 (dB)')
    axis.grid()
    axis.set_xmargin(0)
    axis.set_xlabel('Frequency (GHz)')
    axis.set_title('Return Loss')
    axis.legend()
```

fake symmetry

write S2P file

S11 plot



OpenEMS internal data structure in file system



- Each excitation creates a separate directory to store data
- Combined S-parameter file stored one level above

sub-1	11.03.2025 16:18	Dateiordner
sub-2	11.03.2025 16:45	Dateiordner
sub-3	11.03.2025 17:16	Dateiordner
sub-4	11.03.2025 20:29	Dateiordner
run_core_50ghz_mpa.s4p	11.03.2025 20:58	S4P-Datei



output > run_core_50ghz_mpa_data > sub-1		sub-1
Anzeigen		
Name	Änderungsdatum	
et	11.03.2025 18:56	
ht	11.03.2025 18:56	
port_it_1	11.03.2025 19:26	
port_it_2	11.03.2025 19:26	
port_it_3	11.03.2025 19:26	
port_it_4	11.03.2025 19:26	
port_ut_1	11.03.2025 19:26	
port_ut_2	11.03.2025 19:26	
port_ut_3	11.03.2025 19:26	
port_ut_4	11.03.2025 19:26	
run_core_50ghz_mpa.xml	11.03.2025 18:55	

```
# get results, CSX port definition is read from simulation ports object
# all S-parameter data is available because we have simulated all port excitations
s11 = utilities.calculate_Sij (1, 1, f, sim_path, simulation_ports)
s21 = utilities.calculate_Sij (2, 1, f, sim_path, simulation_ports)
s31 = utilities.calculate_Sij (3, 1, f, sim_path, simulation_ports)
s41 = utilities.calculate_Sij (4, 1, f, sim_path, simulation_ports)
s12 = utilities.calculate_Sij (1, 2, f, sim_path, simulation_ports)
s22 = utilities.calculate_Sij (2, 2, f, sim_path, simulation_ports)
s32 = utilities.calculate_Sij (3, 2, f, sim_path, simulation_ports)
s42 = utilities.calculate_Sij (4, 2, f, sim_path, simulation_ports)
s13 = utilities.calculate_Sij (1, 3, f, sim_path, simulation_ports)
s23 = utilities.calculate_Sij (2, 3, f, sim_path, simulation_ports)
s33 = utilities.calculate_Sij (3, 3, f, sim_path, simulation_ports)
s43 = utilities.calculate_Sij (4, 3, f, sim_path, simulation_ports)
s14 = utilities.calculate_Sij (1, 4, f, sim_path, simulation_ports)
s24 = utilities.calculate_Sij (2, 4, f, sim_path, simulation_ports)
s34 = utilities.calculate_Sij (3, 4, f, sim_path, simulation_ports)
s44 = utilities.calculate_Sij (4, 4, f, sim_path, simulation_ports)
```

```
s4p_name = os.path.join(sim_path, model_basename + '.s4p')
utilities.write_snp (np.array([[s11,s21,s31,s41], [s12,s22,s32,s42], [s13,s23,s33,s43], [s14,s24,s34,s44]]),f, s4p_name)
```

Overview

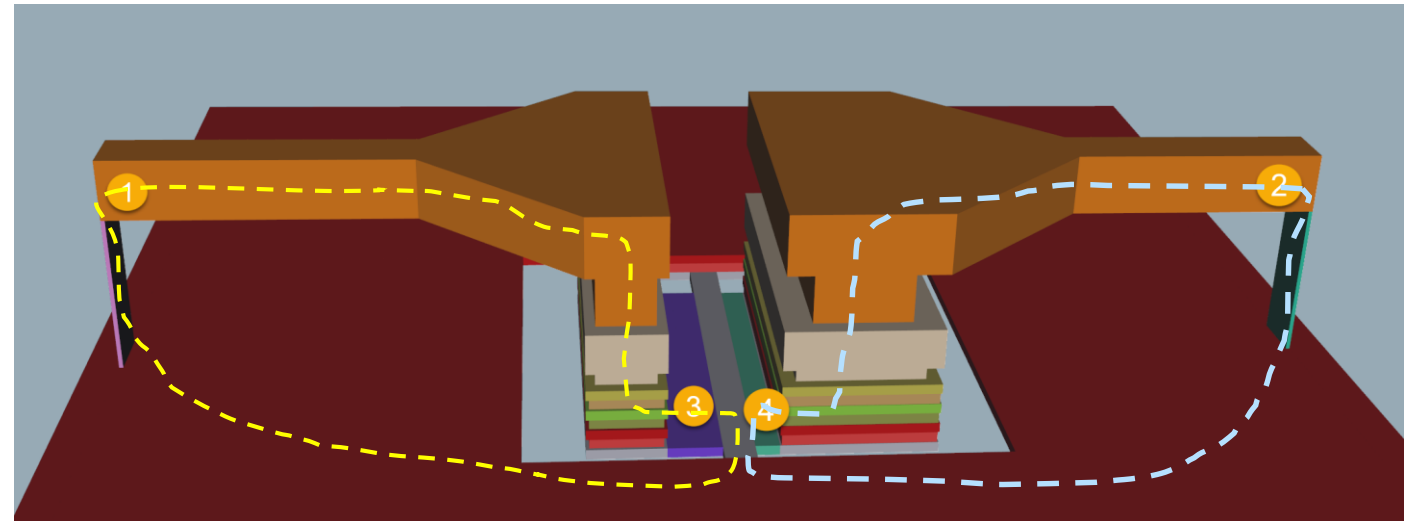
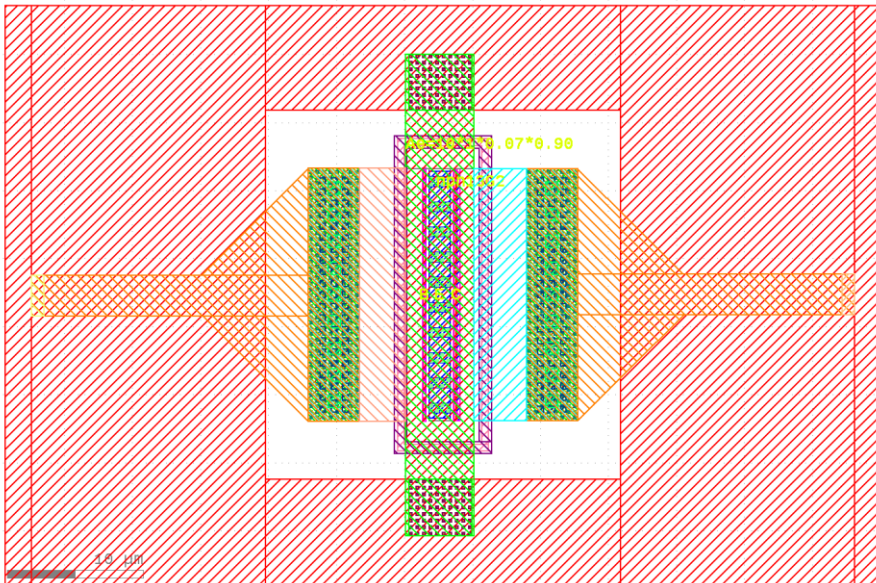


1. Why use EM simulation?
2. openEMS (FDTD) and Palace (FEM) overview
3. Python model workflow overview & example
4. MPA model setup in openEMS, port configuration, mesh and simulation time
5. MPA model in Palace (FEM simulation)
6. Graphical user interface to create & run models
7. Using S-Parameter results, optional lumped model extraction

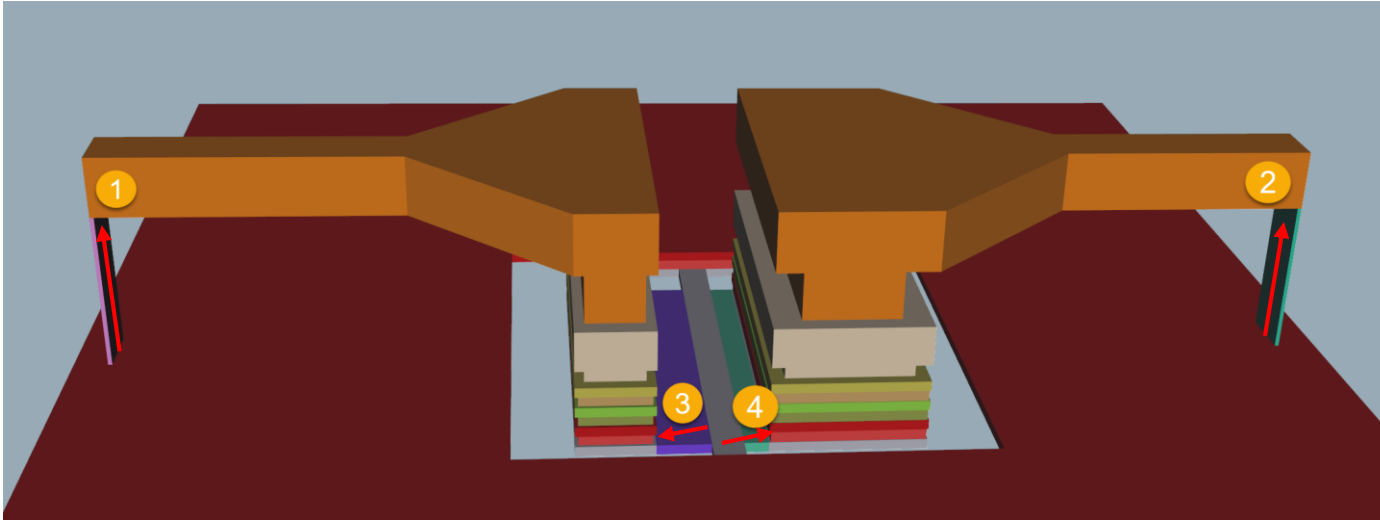
Example model PA core



- PA core is layout with transistor feed + via stack, transistor itself is modelled by ports
- Transistor has center „strip“ as common terminal for Emitter
- Port 3 (Base) and Port 4 (Collector) use that shared ground reference
- Note the closed loops for currents



PA core port definitions

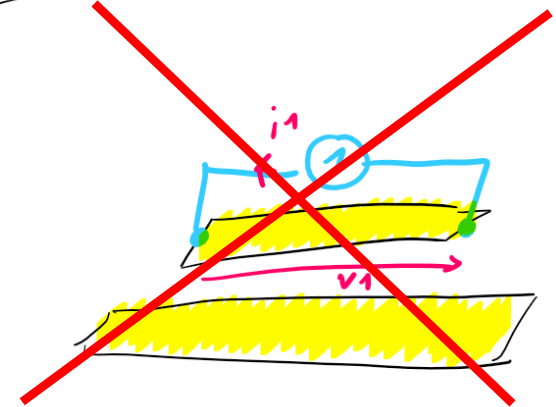
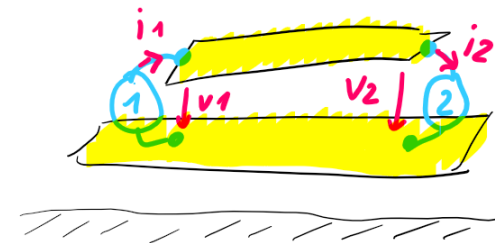
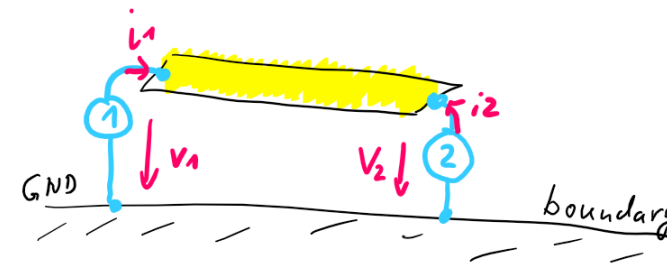
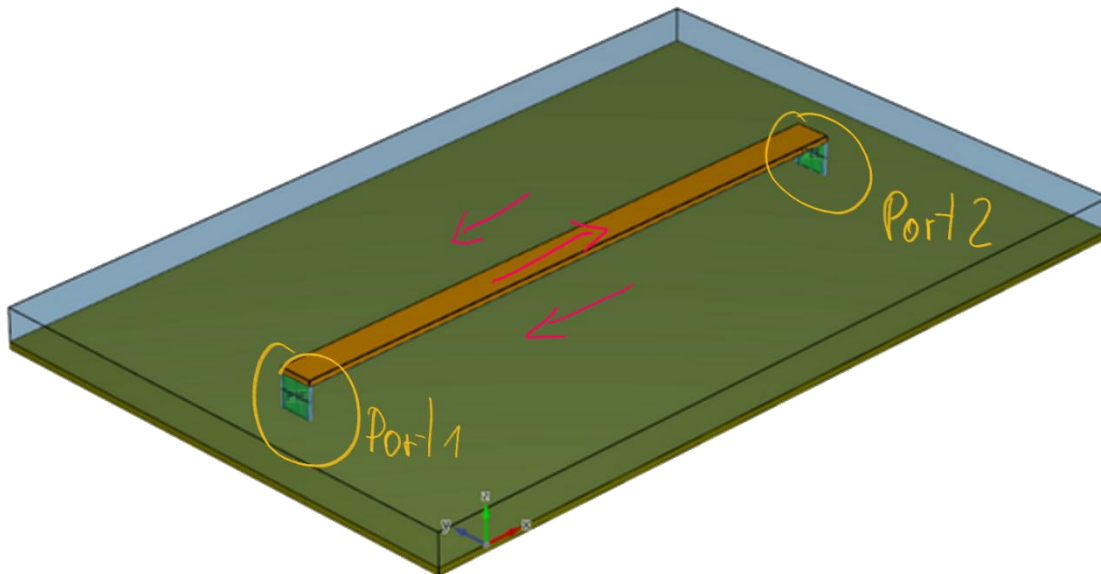


```
simulation_ports = simulation_setup.all_simulation_ports()
# instead of in-plane port specified with target_layername, we here use via port specified with from_layername and to_layername.
simulation_ports.add_port(simulation_setup.simulation_port(portnumber=1, voltage=1, port_Z0=50, source_layernum=201,
    from_layername='Metal3', to_layername='TopMetal2', direction='z'))
simulation_ports.add_port(simulation_setup.simulation_port(portnumber=2, voltage=1, port_Z0=50, source_layernum=202,
    from_layername='Metal3', to_layername='TopMetal2', direction='z'))
simulation_ports.add_port(simulation_setup.simulation_port(portnumber=3, voltage=1, port_Z0=50, source_layernum=203,
    target_layername='Metal2', direction='-x'))
simulation_ports.add_port(simulation_setup.simulation_port(portnumber=4, voltage=1, port_Z0=50, source_layernum=204,
    target_layername='Metal2', direction='x'))
```

Ports in openEMS and Palace FEM



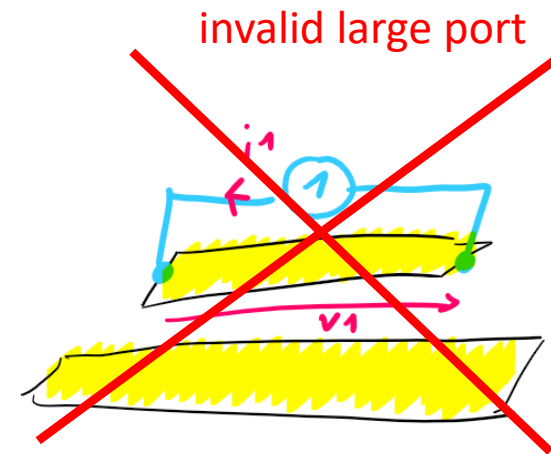
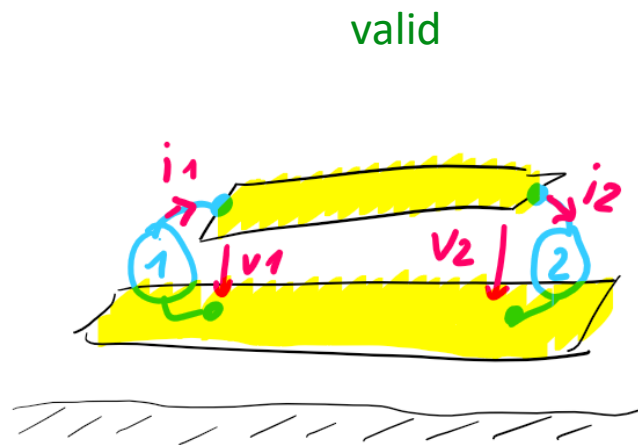
- Ports define the **input/output interfaces to simulated layout**
- **Port $\neq$ Pin**
- Attention ADS Momentum users: ports require signal and reference terminal
- Rule: current only flows in closed loops!
- The port resistor physically closes the loop



Why do we discuss ports in such detail?



- The EM simulator will always give you a result, even if your model setup is not what you intended to model
- Ports are one of the major error sources in EM simulation
- For accurate results, proper port setup is most important
- In openEMS and other 3D EM, keep ports small (compared to wavelength and other geometry)



Simulation Mesh in openEMS

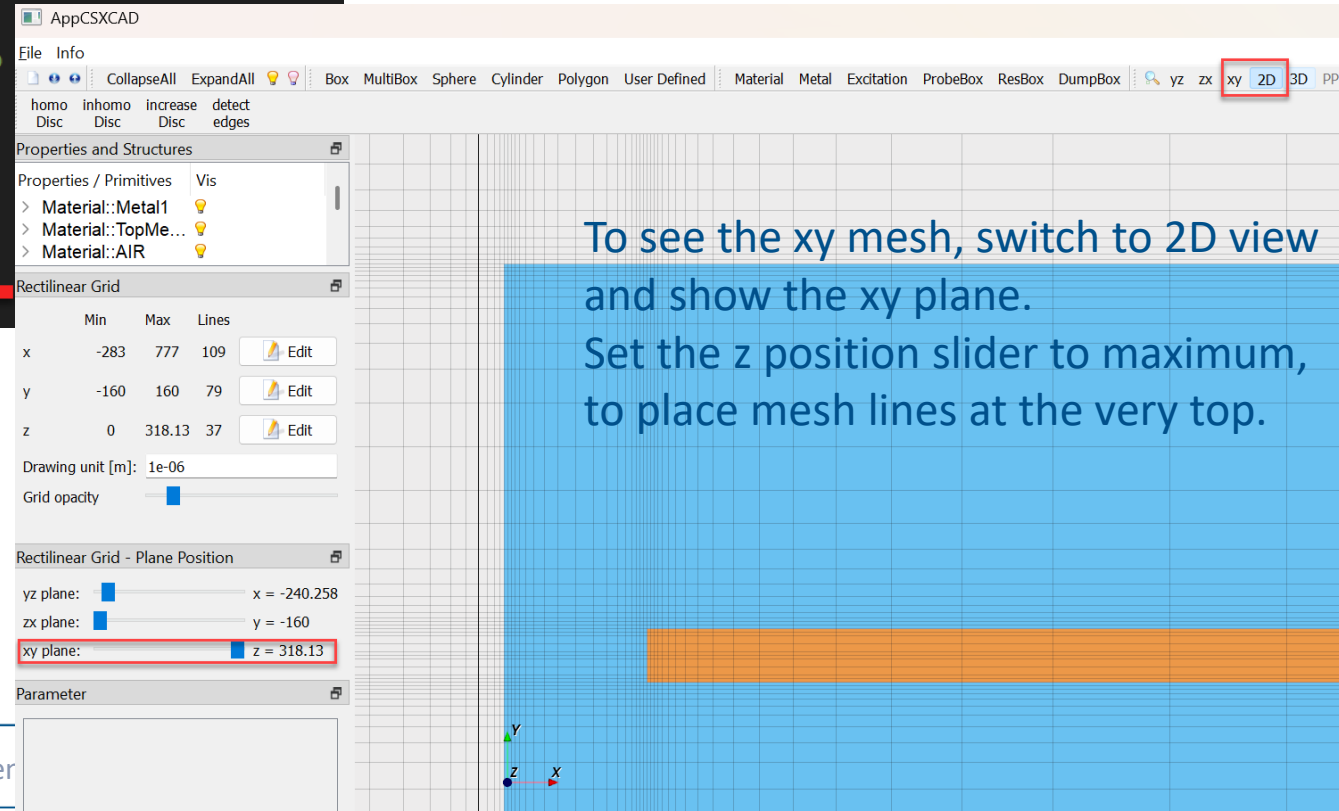


- Creating the simulation mesh is a major difficulty in standard openEMS
- In this new workflow for IHP SG13G2, mesh is built automatically based on geometry
- One user input to set required minimum mesh size (required detail resolution)

```
unit    = 1e-6 # geometry is in microns
margin  = 50   # distance in microns from GDSII geometry boundary to

fstart  = 0e9
fstopt  = 110e9
numfreq = 401

refined_cellsize = 1 # mesh cell size in conductor region
```



Why not always use very fine mesh?



- Using small cell size has two effects:
 - Minimum mesh size gets smaller => simulation time step gets smaller => more time steps needed
 - More mesh cells required for simulation volume
 - These two effects combined: solve many more cells for many more timesteps => longer simul. time

The screenshot displays a simulation environment with several components:

- Code Editor:** Shows a Python script with parameters: `fstart = 0e9`, `fstop = 110e9`, `numfreq = 401`, and `refined_cellsize = 1` (commented as "# mesh cell size in cond"). Below the code, a terminal window shows the execution of `run_line_viaport.py` and the output of mesh cell counts.
- Material Properties:** A list of materials including Metal1, TopMetal2, AIR, Passive, SiO2, Spacing, LumpedElement, and Excitation.
- Rectilinear Grid Settings:** A table showing the grid dimensions and a list of plane positions.
- 3D Mesh Visualization:** A 3D view of the simulation volume, showing a blue grid and an orange rectangular region.

Mesh cells by axis (total 267 kcells):

Axis	Count
x	109
y	79
z	31

Smallest cell size:

Axis	Size
dx	1.0000
dy	1.0000
dz	0.4000

from Metal1 thickness

run_line_viaport.py

Simulation time with different mesh size (openEMS)



- refined_cellsize = 1: min=0.40, 267 kcells, 81263 timesteps, 20s
- refined_cellsize = 0.5: min=0.40, 421 kcells, 106392 timesteps, 35s
- refined_cellsize = 0.2: min=0.14, 919 kcells, 236468 timesteps, 184s

- When Metal1 with thickness 0.4 μm is smallest mesh dimension, we simply see the difference in total mesh count, the FDTD timestep is unchanged
- When refined_cellsize creates mesh cell smaller than that, we have the combined effect of more mesh cells and smaller mesh cell => smaller time step, so total simulation time is much longer

```
compiled against: 9.1.0  
  
Create FDTD operator (compressed SSE + multi-threading)  
FDTD simulation size: 109x79x31 --> 266941 FDTD cells  
FDTD timestep is: 1.85951e-15 s; Nyquist rate: 2444 timesteps @1.1002e+11 Hz  
openEMS::SetupFDTD: Warning, the timestep seems to be very small --> long simulation. Check your  
Excitation signal length is: 28012 timesteps (5.20885e-11s)
```

Simulation Mesh and Skin Effect (openEMS)



- Conductors modelled in openEMS flow as **solid volumes with conductivity**
- Must **mesh into skin depth** to include skin effect accurately in results
- Skin depth for Aluminium:

0.58μm @ 20 GHz

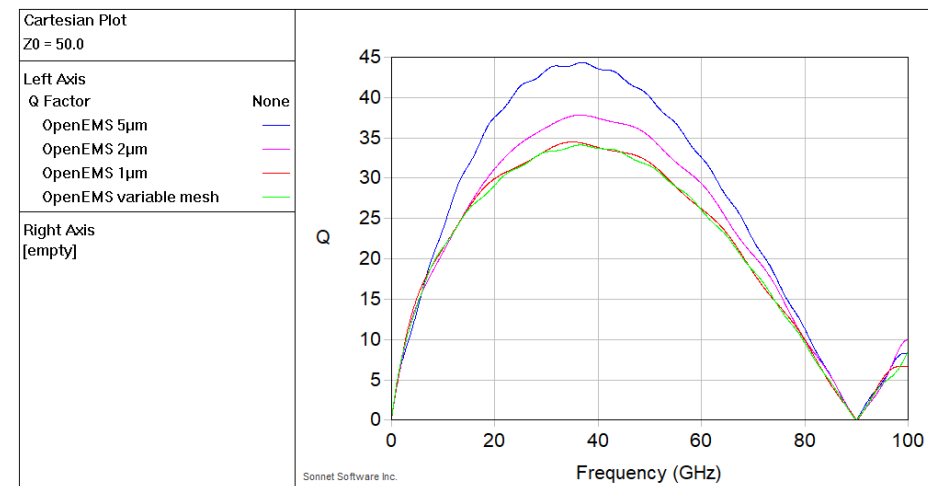
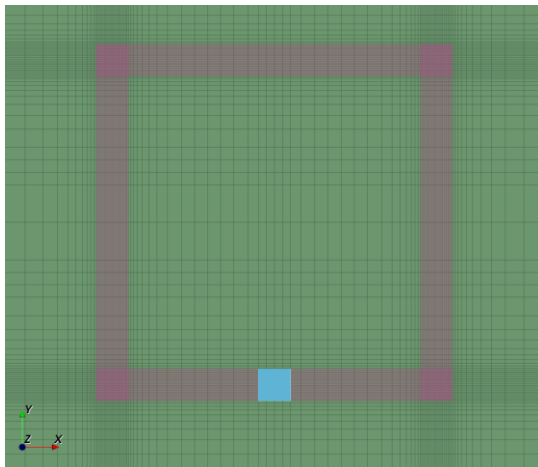
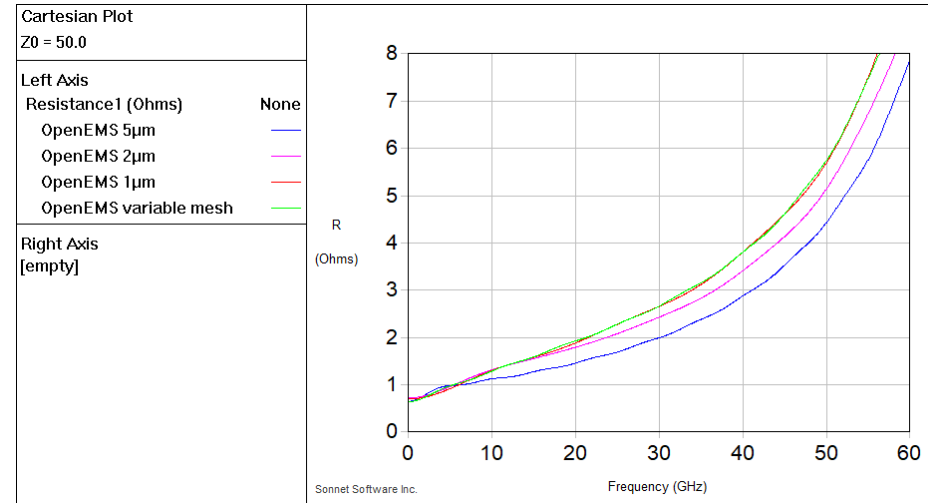
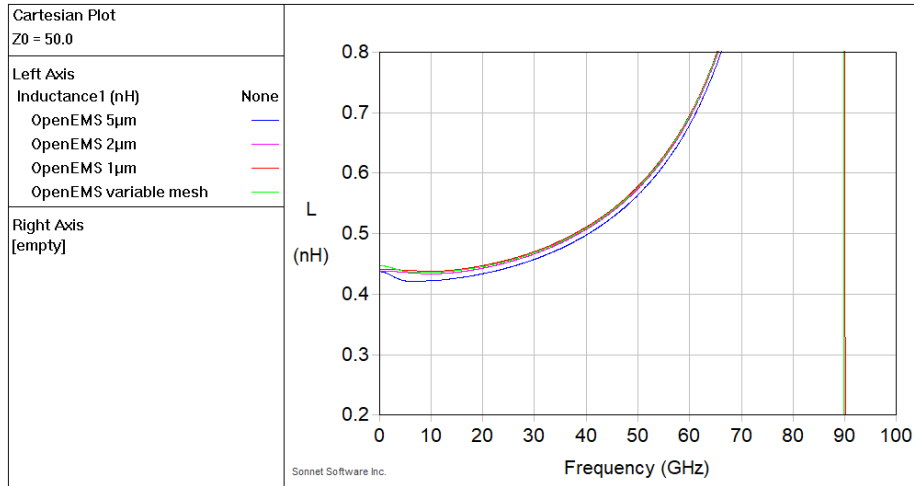
0.37μm @ 50 GHz

0.26 μm @ 100 GHz

$$\delta = \sqrt{\frac{\rho}{\pi f_o \mu_r \mu_o}}$$

- Set refined_cellsize with skin effect in mind, if accurate loss is critical for your model

Simple square loop at different mesh size (openEMS)



OpenEMS mesh resolution summary



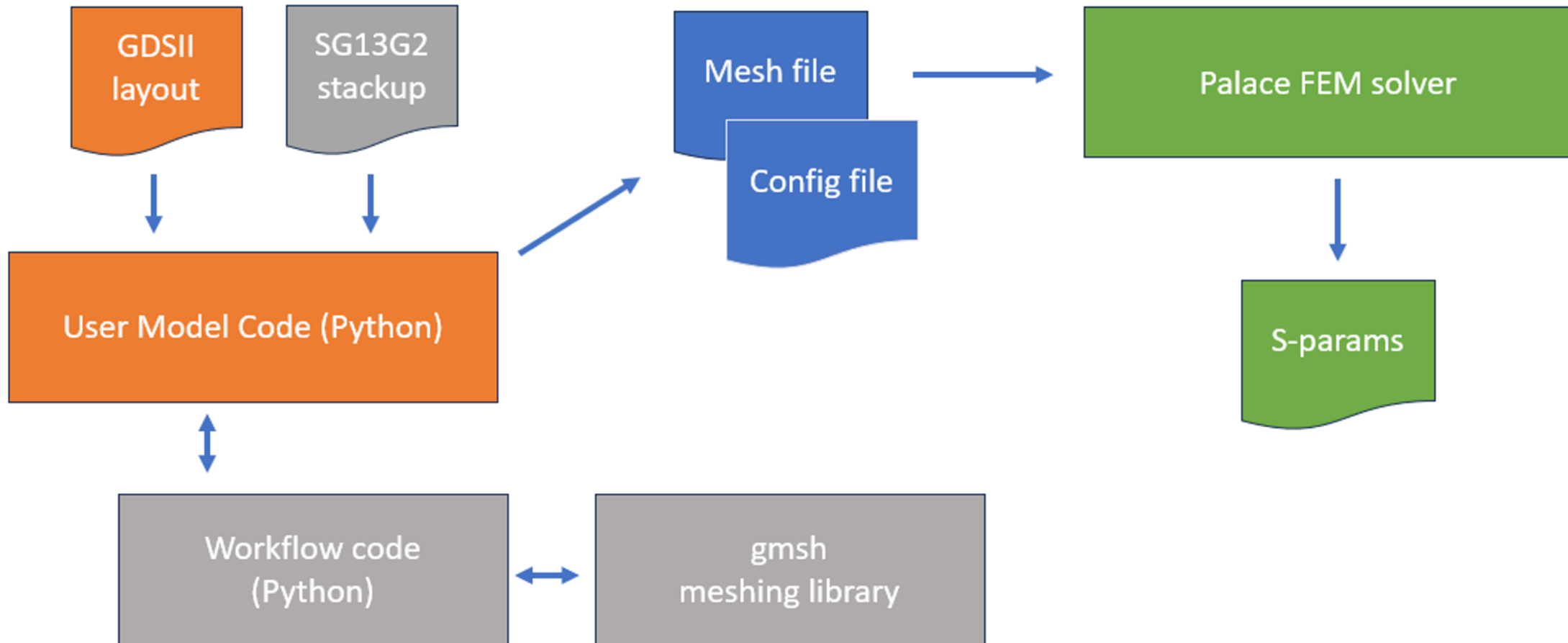
- Small geometry detail is limited by setting `refined_cellsize`, smaller detail is ignored
- Mesh resolution also matters to get correct skin effect loss
- Set mesh resolution as small as necessary, but not smaller ;-)
- Learn from running a few test cases (convergence study)

Overview



1. Why use EM simulation?
2. openEMS (FDTD) and Palace (FEM) overview
3. Python model workflow overview & example
4. MPA model setup in openEMS, port configuration, mesh and simulation time
5. MPA model in Palace (FEM simulation)
6. Graphical user interface to create & run models: EMStudio and setupEM
7. Using S-Parameter results, optional lumped model extraction

Palace flow gds2palace



Palace simulation flow (gds2palace)



- Model is Python script, very similar to openEMS flow
- Palace flow executes these steps:
 - Create simulation input files for Palace solver: mesh file and config file
 - Start Palace simulation (Linux only)
 - Collect Palace results and save in Touchstone *.snp format
- Syntax in model code is slightly different format, but same overall concept

```
settings['unit']    = 1e-6 # geometry is in microns
settings['margin']  = 20   # distance in microns from GDSII geometry boundary to stackup bound
settings['air_around'] = 20 # optional: distance in microns from stackup boundary to simula

settings['fstart']  = 1e9
settings['fstop']   = 350e9
settings['fstep']   = 5e9

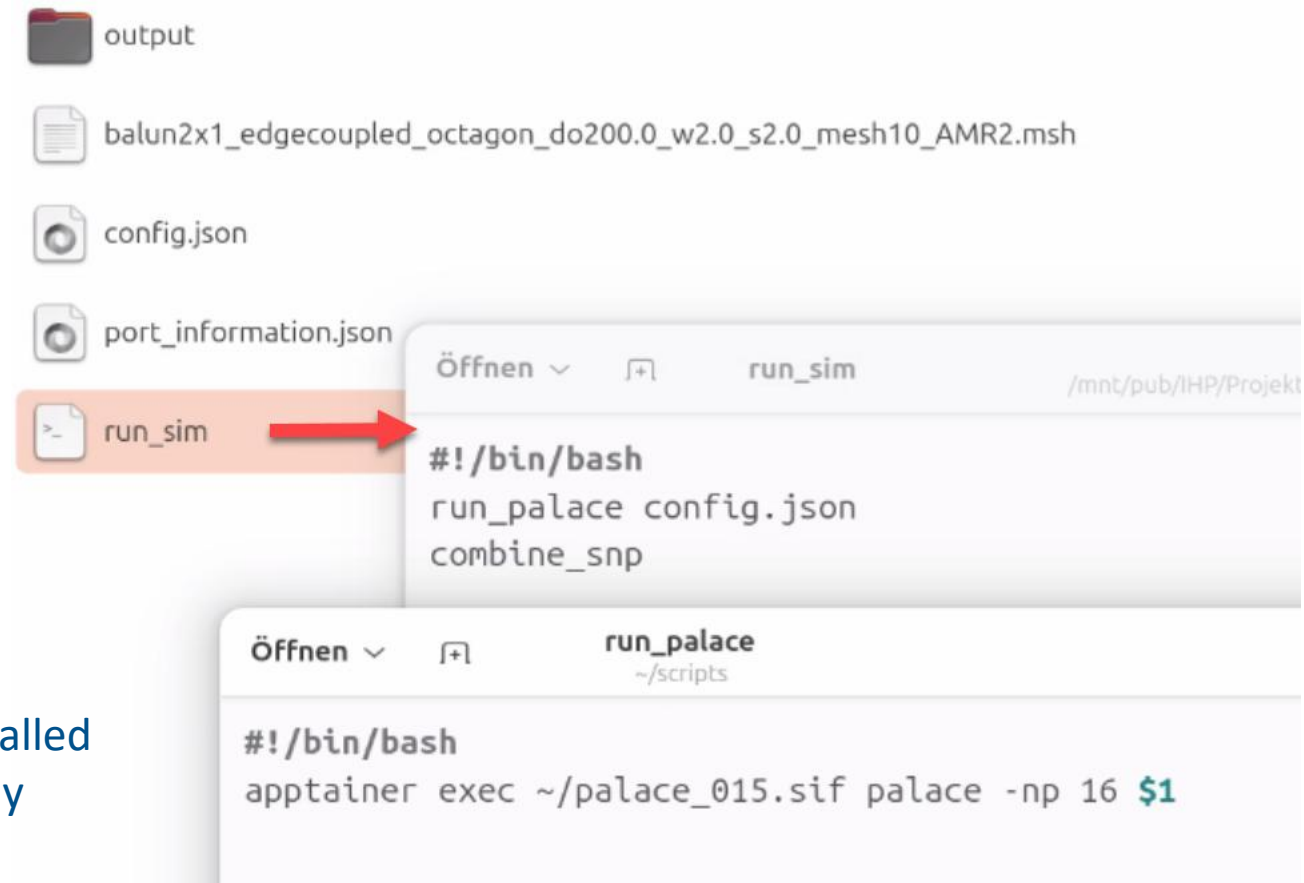
settings['refined_cellsize'] = 5 # mesh cell size in conductor region
settings['cells_per_wavelength'] = 10 # how many mesh cells per wavelength, must be 10 or mo

settings['meshsize_max'] = 30 # microns, override cells_per_wavelength
settings['adaptive_mesh_iterations'] = 0
```

Files created by model script



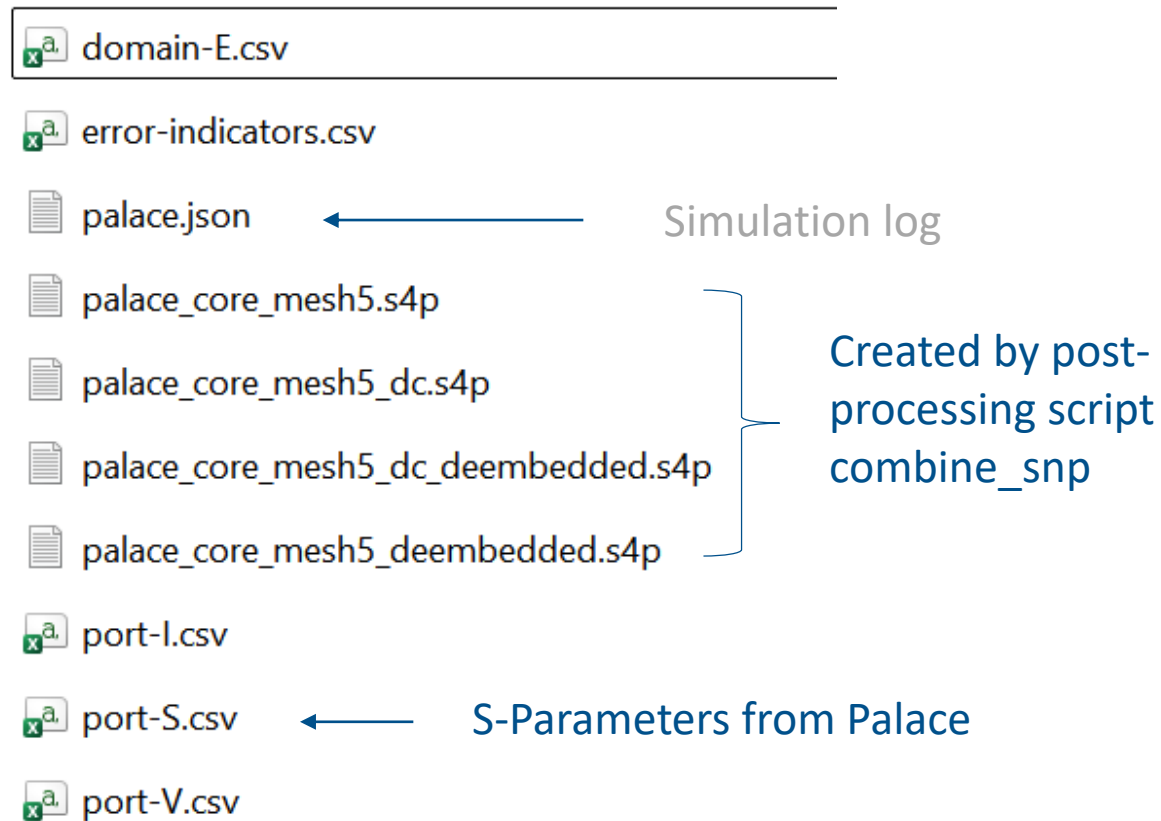
Mesh file
Palace config file
(settings & materials)
Optional metadata
Script to start Palace



run_palace can vary, you might have installed Palace using aptainer or compiled locally or send to remote machine

https://github.com/VolkerMuehlhaus/gds2palace_ihp_sg13g2/tree/main/scripts

Output files created by Palace flow



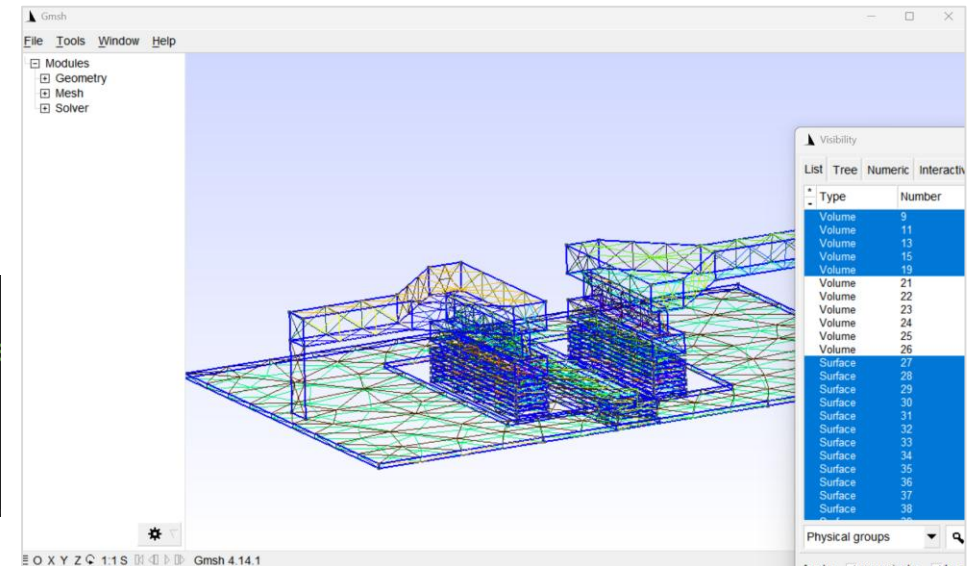
Palace mesh – what is different to openEMS?



- Simulation mesh is created using gmsh mesh engine (typical FEM meshing tool)
- Mesh generation always follows ALL geometry input (does NOT skip/ignore small detail)
- Setting `refined_cellsize` is used to further refine mesh along conductor edges
- Palace uses detailed field modelling within each mesh cell, using 2nd order basis function
- Metals are modelled by surface impedance, no need to mesh into skin effect!
- `refined_cellsize` values typically 2 ... 5 micron
- Adaptive mesh refinement (AMR) is possible
- Fine initial mesh is often good enough without AMR!

```
settings['refined_cellsize'] = 5 # mesh cell size in conductor region
settings['cells_per_wavelength'] = 10 # how many mesh cells per wavelength

settings['meshsize_max'] = 30 # microns, override cells_per_wavelength
settings['adaptive_mesh_iterations'] = 0
```



Overview



1. Why use EM simulation?
2. openEMS (FDTD) and Palace (FEM) overview
3. Python model workflow overview & example
4. MPA model setup in openEMS, port configuration, mesh and simulation time
5. MPA model in Palace (FEM simulation)
6. Graphical user interface to create & run models
7. Using S-Parameter results, optional lumped model extraction

EMStudio (official IHP solution)



- User interface to create & run model scripts for openEMS and Palace flow
- Hybrid solution with editor + user interface that sync both directions

The image displays three screenshots of the EMStudio application interface, illustrating its hybrid design. The first screenshot on the left shows the 'Main' tab with simulation settings like 'cells_per_wavelength' and 'energy_limit'. The middle screenshot shows the 'Ports' tab with a table of port configurations. The third screenshot on the right shows the 'Python' tab with a script editor. Two large blue arrows point from the 'Main' and 'Ports' tabs towards the 'Python' tab, indicating the synchronization between the user interface and the model script.

Num	Voltage	Z0	Source Layer	From Layer	To Layer	Direction
1	1	50	201	Metal1	TopMetal2	z
2	1	50	202	Metal1	TopMetal2	z

```
# change current path to model script path
os.chdir(os.path.dirname(os.path.abspath(__file__)))

# ===== simulation settings =====

settings['unit'] = 1e-06 # geometry is in microns
settings['margin'] = 50 # distance in microns from GDSII geometry boundary to simulation boundary

settings['fstart'] = 0e9
settings['fstop'] = 110000000000
settings['numfreq'] = 401

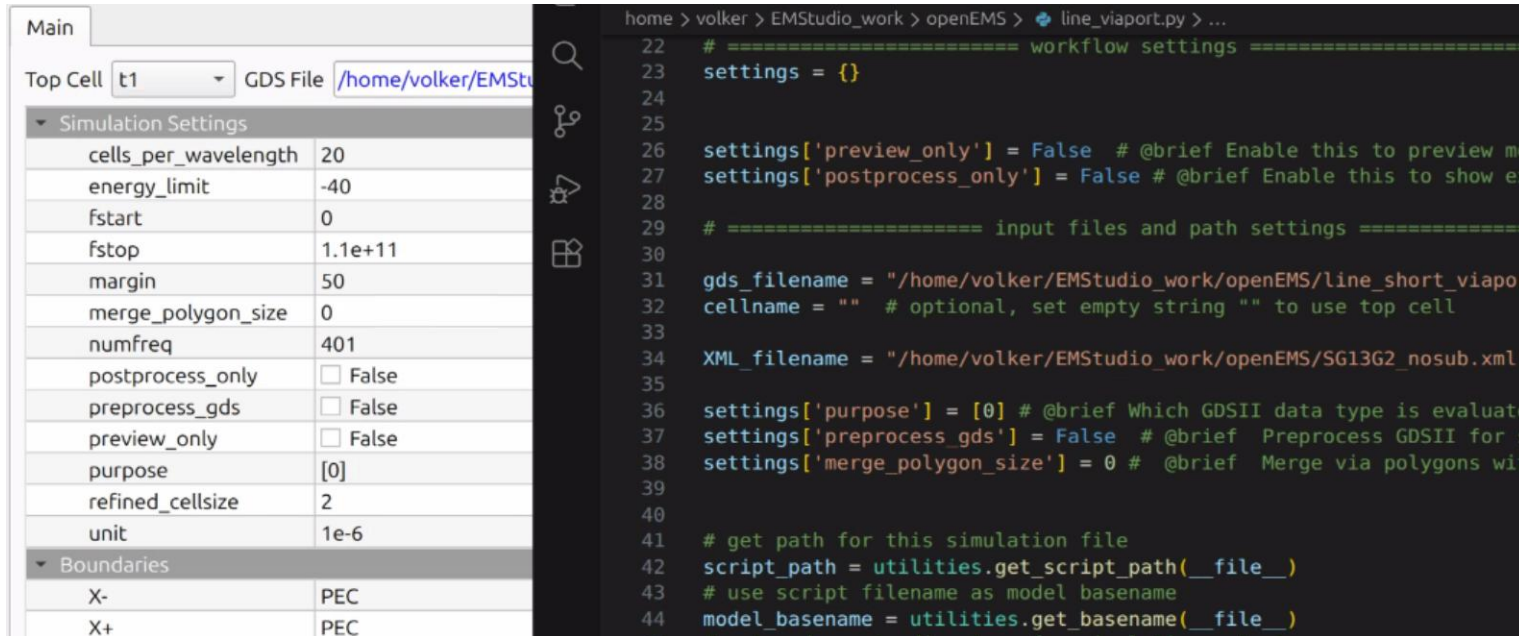
settings['refined_cellsize'] = 2 # mesh cell size in conductor region

# choices for boundary:
# 'PEC': perfect electric conductor (default)
# 'PMC': perfect magnetic conductor, useful for symmetries
# 'MUR': simple MUR absorbing boundary conditions
# 'PML_8': PML absorbing boundary conditions
settings['Boundaries'] = ['PEC', 'PEC', 'PEC', 'PEC', 'PEC']

settings['cells_per_wavelength'] = 20 # how many mesh cells per wavelength, must be 10 or more
settings['energy_limit'] = -40 # end criteria for residual energy (dB)

# port configuration, port geometry is read from GDSII file on the specified layer
```

- Create simulation model from scratch, choose openEMS or Palace template
- Can read existing Python model code (openEMS or Palace)
- Template scripts for openEMS and Palace, create SnP Touchstone output, no data plots
- In settings table on main tab, parameters with settings[] syntax can be edited

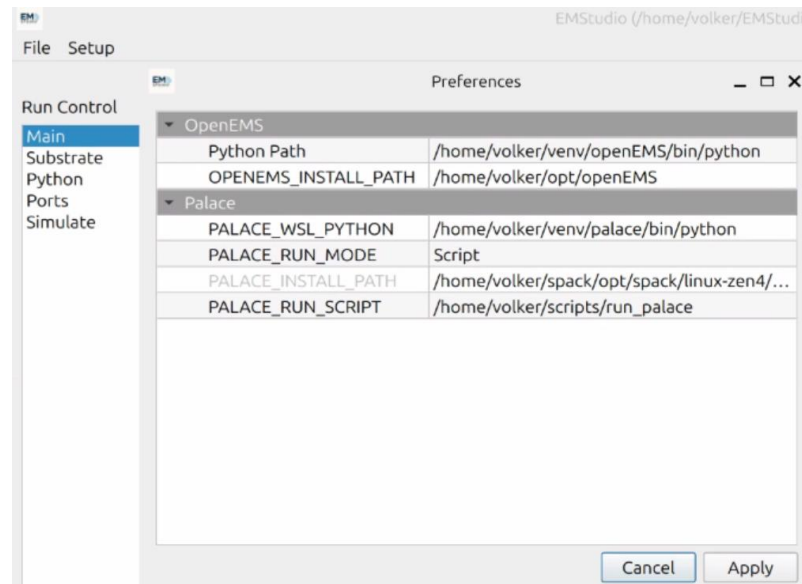


The screenshot displays the EMStudio interface. On the left, the 'Main' tab is active, showing a 'Simulation Settings' table. The 'Top Cell' is set to 't1' and the 'GDS File' is '/home/volker/EMStudio...'. The settings table includes parameters like 'cells_per_wavelength', 'energy_limit', 'fstart', 'fstop', 'margin', 'merge_polygon_size', 'numfreq', 'postprocess_only', 'preprocess_gds', 'preview_only', 'purpose', 'refined_cellsize', and 'unit'. Below this is a 'Boundaries' section with 'X-' and 'X+' set to 'PEC'.

On the right, a Python script is shown in a code editor. The script defines workflow settings and input files. Key lines include:

```
22 # ===== workflow settings =====
23 settings = {}
24
25
26 settings['preview_only'] = False # @brief Enable this to preview mo
27 settings['postprocess_only'] = False # @brief Enable this to show ex
28
29 # ===== input files and path settings =====
30
31 gds_filename = "/home/volker/EMStudio_work/openEMS/line_short_viapor
32 cellname = "" # optional, set empty string "" to use top cell
33
34 XML_filename = "/home/volker/EMStudio_work/openEMS/SG13G2_nosub.xml"
35
36 settings['purpose'] = [0] # @brief Which GDSII data type is evaluate
37 settings['preprocess_gds'] = False # @brief Preprocess GDSII for s
38 settings['merge_polygon_size'] = 0 # @brief Merge via polygons wit
39
40
41 # get path for this simulation file
42 script_path = utilities.get_script_path(__file__)
43 # use script filename as model basename
44 model_basename = utilities.get_basename(__file__)
```

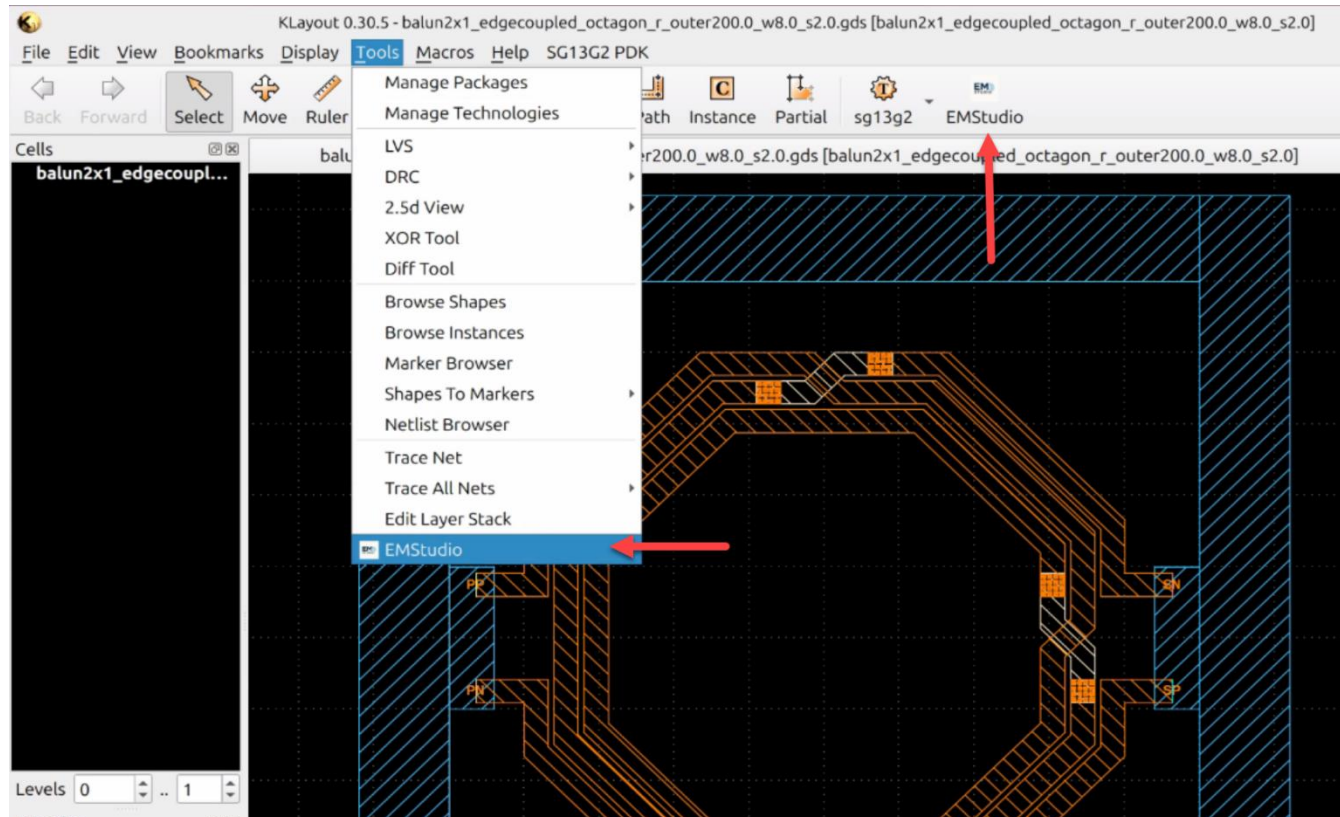
- <https://github.com/IHP-GmbH/EMStudio>
- For Linux, follow instructions in readme to build from source (no worries, it's automated!)
- For Windows, follow instructions to download pre-built Windows installer version
- openEMS supported native on Windows & Linux (but Linux version simulates faster)
- Palace not supported on Windows, requires Windows Subsystem for Linux (WSL) to run models



Start EMStudio from klayout



- EMStudio provides a plug-in for klayout to start EMStudio and load the current layout file
- Settings from previous run can be loaded (load Python model, File > Recent ...)



setupEM (developed for gds2palace flow)



- User interface to create, load and run model scripts for Palace flow
- GUI driven: resulting script is shown in setupEM, but no script editing
- No support for openEMS

setupEM Palace

File Simulator Help

Input Files | Frequencies | Ports | Mesh and Boundaries | Create Model | Code

GDSII Layout File

Browse ...

Read this datatype (purpose): (default=0, multiple values can be separated by comma)

Merge via arrays with spacing micron or more, value 0 disables via array merging

☐ Preprocess GDSII file (required for polygons with holes/cutouts)

XML Stackup File

Browse ...

Show Stackup

setupEM Palace

File Simulator Help

Input Files | Frequencies | Ports | Mesh and Boundaries | Create Model | Code

Mesh settings

Mesh refinement at metal edges μm

Mesh cells per wavelength (min 10)

Mesh cell maximum size absolute μm

Mesh basis function

Adaptive mesh refinement (AMR)

Adaptive mesh iterations (default is 0, no adaptive mesh refinement)

Boundary settings

Boundary conditions

Dielectric stackup: oversize by μm from metal drawing

Air layer thickness around stackup is μm

setupEM Palace

File Simulator Help

Input Files | Frequencies | Ports | Mesh and Boundaries | Create Model | Code

Port settings

Port geometry on layer number in GDSII file

Port direction (negative for reversed polarity)

Via port from layer to layer

Port impedance

Port voltage (1=active, 0=passive)

Apply ↓ Remove

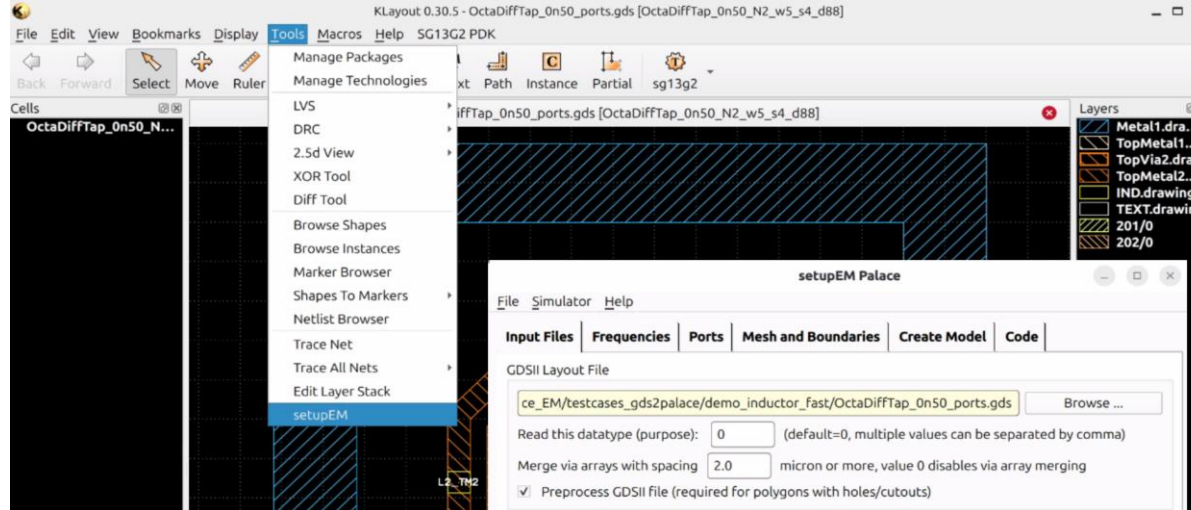
Port overview

	Z0	Voltage	Source layer	Target layer	From layer	To layer	Direction
1	50.0	1.0	201		Metal3	TopMetal2	Z
2	50.0	1.0	202		Metal3	TopMetal2	Z
3	50.0	1.0	203		Metal3	TopMetal2	Z
4	50.0	1.0	204		Metal3	TopMetal2	Z
5	50.0	1.0	205	Metal2			-X
6	50.0	1.0	206	Metal2			X
7							
8							
9							

setupEM



- Repository and readme: <https://github.com/VolkerMuehlhaus/setupEM>
- setupEM is Python code, easiest way to install into your Palace venv: `pip install setupEM`
- Create script wrapper to start setupEM from that venv where module is installed
- layout plug-in is available, similar to EMStudio: `klayout_setupEM.py`



Overview

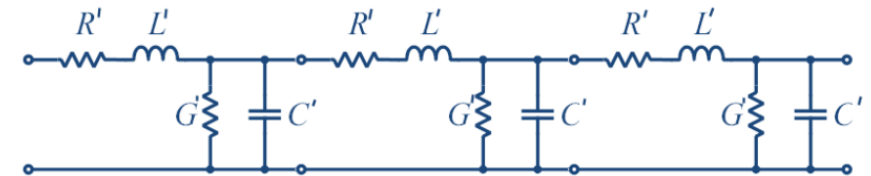
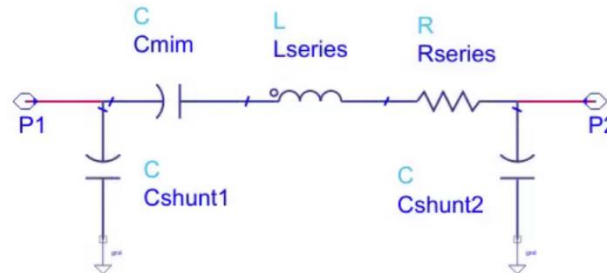
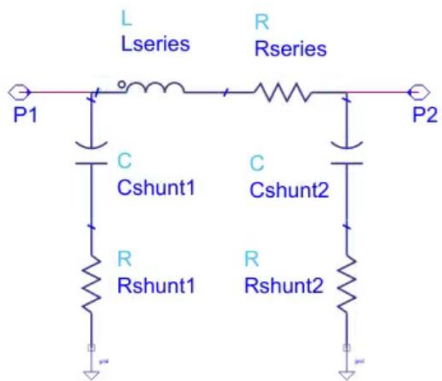


1. Why use EM simulation?
2. openEMS (FDTD) and Palace (FEM) overview
3. Python model workflow overview & example
4. MPA model setup in openEMS, port configuration, mesh and simulation time
5. MPA model in Palace (FEM simulation)
6. Graphical user interface to create & run models
7. Using S-Parameter results, optional lumped model extraction

Using results



- S-parameter output can be used in circuit simulation
- No noise simulation in ngspice with S-parameter data -> workaround is lumped circuit model extraction
- Lumped model extraction also useful for transmission line models
- Lumped model ensures fixed topology, true DC isolation, no leakage from small numerical errors
- Downloads: <https://github.com/VolkerMuehlhaus/lumpedmodel>



References



- openEMS:
<https://www.openems.de/>
- IHP workflow for openEMS
https://github.com/VolkerMuehlhaus/openems_ihp_sg13g2
- AWS Palace:
<https://awslabs.github.io/palace/stable/>
- IHP workflow for Palace:
https://github.com/VolkerMuehlhaus/gds2palace_ihp_sg13g2
- Lumped model extraction:
<https://github.com/VolkerMuehlhaus/lumpedmodel>

Thank you for your attention!

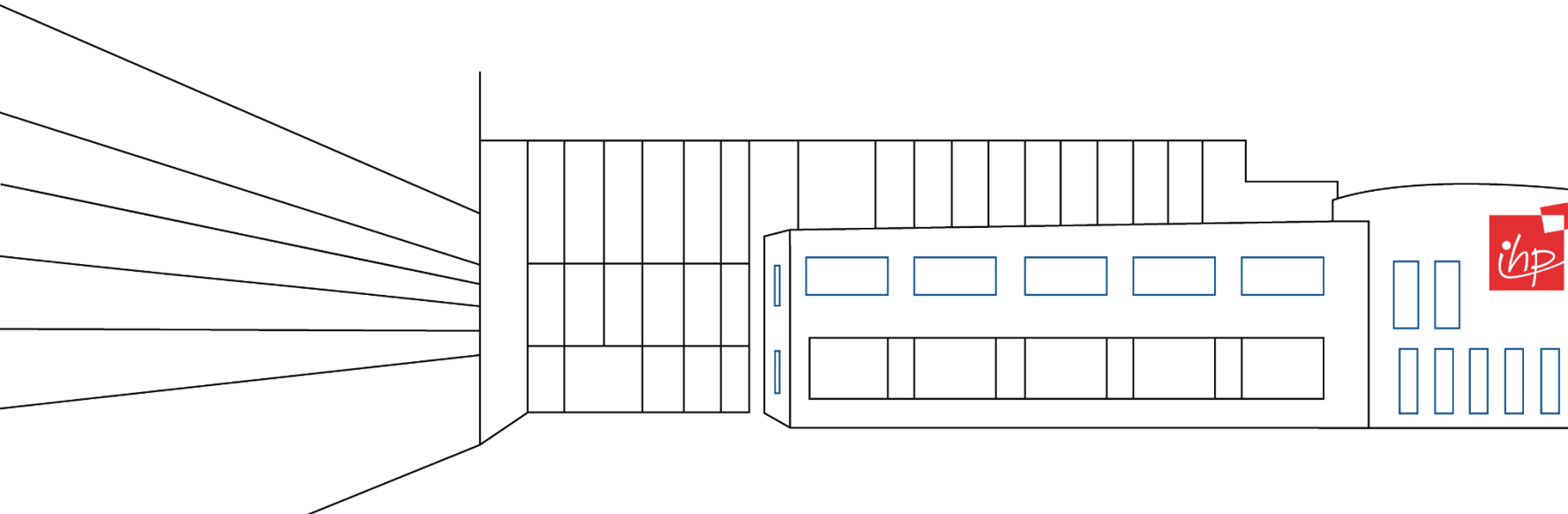
IHP GmbH – Innovations for High Performance Microelectronics

Im Technologiepark 25

15236 Frankfurt (Oder)

Tel.: +49 (0) 335 5625

E-Mail: volker@muehlhaus.com



Remaining Agenda for Today



- 0 EM simulate components for the PA and implement in the simulation refer to the following link for the guide:

https://github.com/IHP-GmbH/IHP-AnalogAcademy/blob/main/modules/module_2_50GHz_MPA/part_4_layout_EMsim/README.md

- 0 **Catch-Up:** Work on material from today or the other days to catch up or clarify points that wasn't clear
- 0 **Relax** 😊

What Will We Do Tomorrow?



- 0 **Xschem Practice:** Create small building blocks for a larger circuit and perform analyses such as Monte Carlo simulations based on your prior experience
- 0 **Verilog:** Write and simulate simple Verilog code using open-source tools.
- 0 **Analog-Mixed-Signal Integration:** Learn to integrate digital blocks into analog designs within Xschem.
- 0 **8-bit SAR ADC Simulation:** Build and simulate a basic 8-bit successive approximation register (SAR) ADC using the day's designs.
- 0 **Post-Processing with Python:** Plot and analyze transient simulation results using Python.

Bonus Material

Port Parasitics in openEMS Models

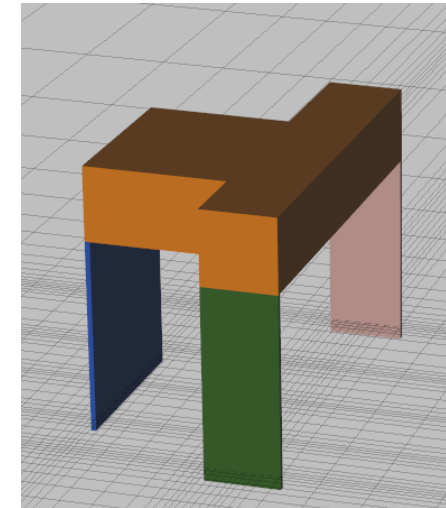
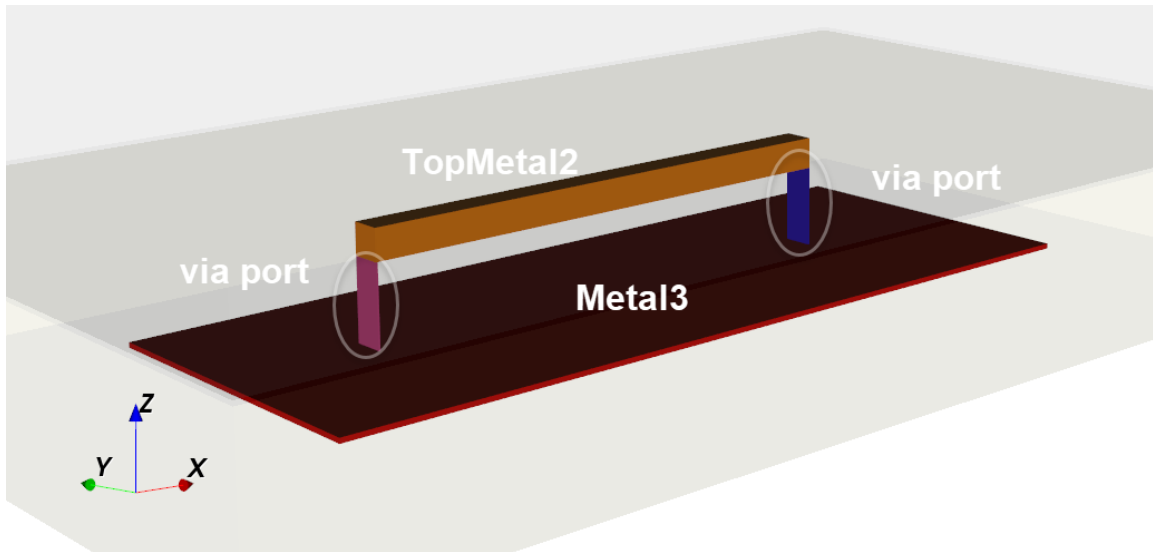
Volker Mühlhaus

29-Jan-2026

Effect of port size



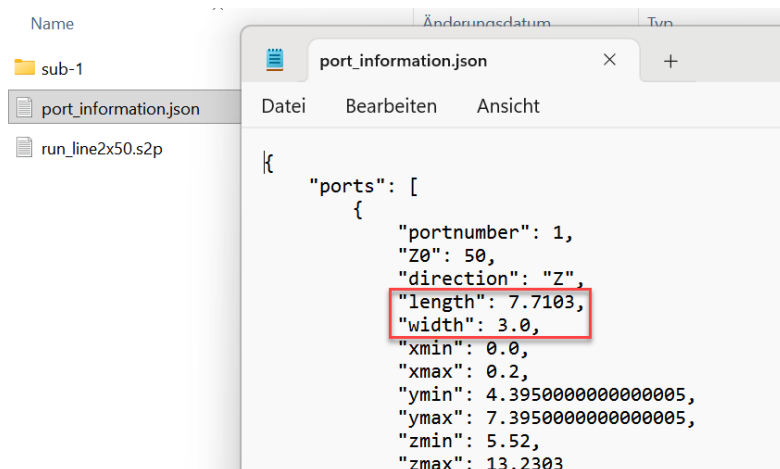
- Ideally, we want ports to be small compared to wavelength and DUT
- Sometimes this is not possible
- Ports then add physical length -> phase to results



Simple port de-embedding concept



- openEMS and Palace write port information file with geometry data
- Postprocessing script assumes „thin sheet“ current flow and estimated inductance of that sheet (=port) geometry
- Cascade negative inductance at each port
- Does not account for coupling between ports



```
def flat_strip_inductance(length, width, thickness, unit):  
    """  
    Flat Wire Inductor Calculator  
    The original for this equation is by F.E. Terman and can be found in the Radio Engineers Handbook, McGraw-Hill  
    """  
    return 2e-7 * length * unit * (math.log(2*length/(width+thickness)) + 0.5 + 0.2235*(width+thickness)/length)  
  
def port_deembedding (snp_filename, port_info_available, port_info_data):  
    if port_info_available:  
        print('Port de-embedding based on port geometry data')  
        unit = port_info_data.get("unit", 1e-6) # default dimension is micron  
  
        # calculate parasitic port inductance for all ports  
        Lport = {}  
        portlist = port_info_data["ports"]
```

De-embedding script



- https://github.com/VolkerMuehlhaus/openems_ihp_sg13g2/tree/main/scripts
- Run script with openEMS *.snp as input file
- Script creates new file with suffix _deembedded

```
D:\Workshop_Training\IHP_Workshop_Feb2026\port_parasitics\line_two_lengths\output\run_line2x50_data>deembed_openEMS run_line2x50.s2p

D:\Workshop_Training\IHP_Workshop_Feb2026\port_parasitics\line_two_lengths\output\run_line2x50_data>D:\venv\palace\Scripts\python d:\batch\deembed_openEMS.py run_line2x50.s2p
Found extra file with port information: port_information.json
Port Z0 values found: [50, 50]
Port impedance for Touchstone header: 50
Port de-embedding based on port geometry data
Cascading L= -3.43 pH at port 1
Cascading L= -3.43 pH at port 2
Created file with de-embedding (cascaded negative port L): run_line2x50_deembedded
```